MONSTER DUNGEON

My project is going to be a 2d dungeon crawler game, the aim of the game will be to hunt monsters in a dungeon to collect their parts to then get out alive so that you can sell them to upgrade your gear/stats. It will have procedural level generation and the spawn rate/behaviour of monsters will change depending on how many have been killed. Centre Number: 20570Candidate Number: 2290

KING EDWARD VI COLLEGE

Contents

Analysis 4
Problem definition 4
How the Game Works 4
Background Research 5
A* search Algorithm 5
Cult of the Lamb – Product Overview 6
Stardew Valley – Product Overview 8
Summary 10
End user interview 10
End user analysis 10
Interview 11
Interview summary 12
Objectives 12
Modelling 16
The core concepts 17
Pathfinding 18
Level Generation 22
Design 26
Mission Statement 26
Design Introduction 27
Introduction 27
Hierarchy Charts 28
..... 30
Data Structures 31
Key algorithms 33
A-star Algorithm 33
Map Generation 34
Collisions 36
Class Diagrams 38
..... 39
..... 39
..... 39
..... 39
..... 39

.....39
.....39
.....39
.....39
.....39
.....39
Functions.....40
Main.py 40
Character.py.....43
Generation.py 48
File Structure.....52
.....53
HCI (user interface) 53
Start Menu 53
Chest Menu.....53
Main Game.....54
Rest Screen.....55
.....55
.....55
Testing.....56
Test strategy.....56
Character Class.....56
Level generation.....56
Monster Class.....56
Menu`s 57
Test Plan.....57
The code.....61
Main.py 61
Character.py.....86
Generation.py 118
Evaluation 147
Objectives 147
End User Feedback.....151
End User Review 151
Questions 151
What should be taken away from the Feedback? 152

Future Improvements 152
Gold Counter 152
Health Bar 153
Vectorising Player Movement 154

Analysis

Problem definition

There are not many games that combine the genres of monster hunters and dungeon crawlers so my project will put those two genres together as well as this it will also be more realistic with the spawn rates of monster changing so that if the player hunts them enough, they will become extinct which is not a common trope in monster hunter games so this will be for people who want a more realistic game.

I am doing my project on this to for fill this niche.

My project will solve this problem by giving the monsters a complex AI so that they will start to change their behaviour if they are being killed a lot by running away from the player to show that they learn that to survive they must evade the player but on the other hand they would also start to hunt the player for food if they see them if their numbers are high because they are not scared of death when faced to the player. The generation of the levels will also help this by making the generation a lot like it would be in a dungeon crawler where the player starts off with their starting equipment and can find new gear through their journey through the dungeon it would also be procedural so the player could play it forever as no dungeon/level would be like the last.

How the Game Works

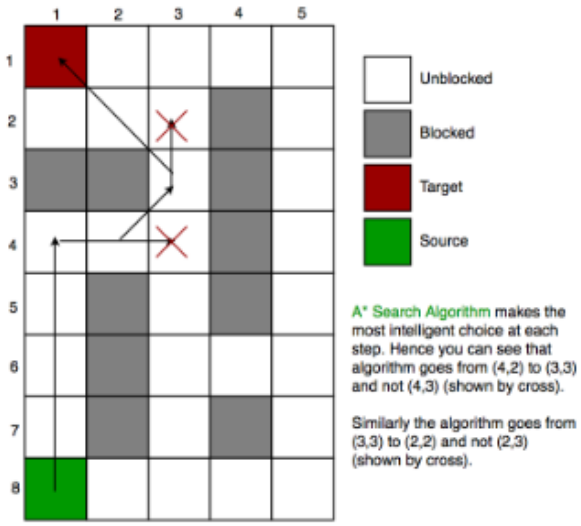
You will start the game at main menu where you can select your character or create a new one. After you have selected a new character you will be sent to a rest point which is like a main menu where you can upgrade your characters gear and rest to heal health points, it will also act as the save points of the game like checkpoints so if you die or load a character in from the main menu you will be sent to the last rest point you were at. Rest points can be reached by selecting that type of level when choosing what level you would like to go to next. After you have selected that you would like to leave the rest point you will be shown a map where you can select what level to go to and where that level will then allow you to go to so you can plan out what path you want to go on. There will be different level types being rest points, different dungeon types (based on different environments) and special rooms where you can get gear through treasure chests or find a travelling merchant to buy items from or ways to heal for a small amount without the need of a rest point. After selecting a level that is a dungeon a level will be generated which will have interconnected rooms that in one of them have a level exit. However, there will be monsters that inhabit the rooms will chase after the player at first and try to attack them however if the player continues to decrease their population they will start to run away from the player as they are close to extinction. If the player does successfully make one of the monster types extinct it will stop spawning in for that playthrough. However if the play dies while attacking the monster they will loose all progress they made after their last rest point and go back to that point. If the player manages to escape the monster the monster will start to wander around the room it is in until it sees the player again. The aim of the game is to survive as long as possible as each level gets harder with the strength and sometimes numbers of the monsters increasing depending on which level it is. If the player does end up making all monsters extinct they would of beat the game however it is not necessarily the aim as the player would not be able to get anymore stronger and would be stuck at that point so the player should learn to control the levels of the population and let them rise when they get low so that the player can keep getting stronger to increase their power level which is like the high score system of the game. So overall the aim of the game is to get your character to be the strongest. The Travelling merchant one of the special levels will allow the player to purchase new equipment outside of the rest points of the game however it will not allow them to spend there money to increase their core

stats. The treasure chests will give you a random piece of gear with a similar power level of that type of gear that you have equipped so that you cannot get really powerful early on for getting lucking from a treasure chest. The heal points will allow the player to spend some money to heal a quarter of their health and can only be done once so that they cannot just keep buying it until they are at full health. The map where you select levels will have an end rest point where you will get gear with a significantly higher power level to the gear that you have now so there is an incentive to reach the end after doing so you will enter a new map where the monsters are more common and more powerful and there are less rest points to act as save points so it will be more challenging to get past this map.

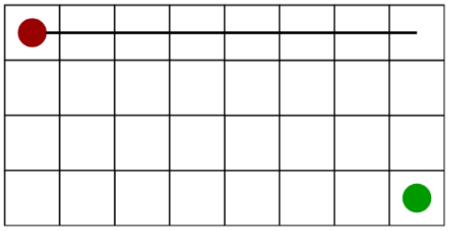
Background Research

A* search Algorithm

A* search algorithm is one of the most popular techniques used in pathfinding so I will be using it for the pathfinding of the monsters.



A* algorithm has 2 cost values to decide which node to visit. It has the actual cost between nodes this is the distance from one node to another (distance) as well as this it also has the approximate cost from the starting node to the final node (Heuristic). It uses the distance and the heuristic to find the shortest distance in the algorithm it does this by adding them together.



This diagram is a diagram of the Diagonal distance equation which is used for the Heuristics distance.

The data stored for this algorithm:

Node	From	Distance	Heuristic	Total distance
B	A	7	6	13
C	A	3	7	10
D	A	6	1	7

So, with this data D would be selected as it has the smallest total distance.

To get the Heuristic data I will be using an algorithm called diagonal distance this calculates the distance where the object can move in 8 directions on a grid which is how my game will be played. So, the equations are used:

Distance x = abs(current_X_position – goal_X_position)

Distance Y = abs(current_Y_position – goal_Y_position)

Heuristics = D * (Distance x + distance Y) + (D2 – 2 * D) * min(distance x, distance Y)

Where D is the length of each node and D2 is the diagonal distance between each node.

The algorithm starts out by initializing two lists, an open list and a closed list. The starting Node is appended to the open list. A while loop is then created for when the open list is not empty. It then pops the node with the smallest Total distance and appends it to the open list and calls it q. Of this list it generates the cells around it as its children and makes q their parent. Then a for loop is created for each child. It checks if this child is the goal if so the search stops, else it computes both the g and h for this child. Equations used are:

- Child.g = q.g+distance between child and q
- Child. Heuristic = distance from goal to child (Done using the equation stated above)
- Child.Total distance = child.g + Child. Heuristic

Then it checks if a node with the same positions as child is already in the closed list which has a lower f than the child if this is true it then skips this child if not the node of the child is appended to the open list. The for loops ends. And then q is added onto the closed list by being popped off of the open one.

To make the algorithm simpler I will be using the python module Heapq this makes it so that the starting element of the list (the root) is always the smallest value. This removes the need to code a loop to check for the smallest value in the whole list so the open list would be the one using this module. Heappop is a useful function and the one that will get used the most as it pops the smallest value in the heap so I will be using this for when I need to search for the smallest total distance.

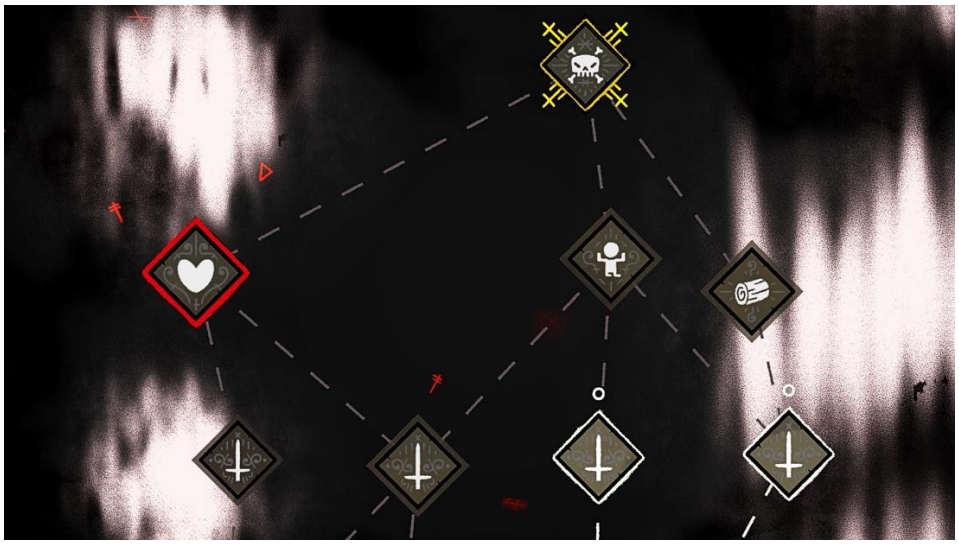
Cult of the Lamb – Product Overview

Cult of the Lamb is a dungeon crawler where you can also have a campsite which you can upgrade however for my research, I will be focusing on the part of the game that is the dungeon crawler.

Level generation:

The level generation is split up into two parts the overall map and the individual levels which is where the main part of the game is.

The overall map is where the player selects which route, they want to take and allows them to select which room they want to go into. I won't be using this aspect of the game as I feel it takes away from the exploration of the level as the player would not be able to go to every part of the map which will be an aspect of my game.



After you get select which your level you enter the main part of the level generation. In which there is a grid of rooms which you can enter finding enemies or chests in each room. I like how the rooms connect and that it rewards exploration however the rooms are just rectangles of varying sizes which is not very interesting for level generation so in my game I will use a more advanced generation for the individual rooms.

The combat:



I really like the combat in this game as it feels smooth which is very important for a combat-based game. There are different weapon types in this game an axe, dagger, sword, hammer and a gauntlet which all offer different playstyles, attack speeds and damage outputs. I would like to implement this into my game as I feel it keeps the combat fresh and new. Cult of the Lamb also has a combo system. The way it works is that there are 3 attacks that build up to a final attack so it rewards the player for taking risks and going in for the final attack as it deals more damage than the last .

In the game they also have area of effect attacks which are spells in the game, I really like how they look, and they feel to play, and I feel that my game could benefit from the use of AOE attacks.



These attacks are very destructive and good at getting rid of lots of enemies at once which is very satisfying when you are playing the game.

Stardew Valley – Product Overview

Stardew Valley is a farming game however there is a section of the game I enjoy and feel that my game could benefit from the insight of having. This section being the mine in Stardew Valley. The aspect that I would like to use for my game is the level generation of the mines.





It is procedurally generated so that every level is different from the last time you went in. I like that it is very open at parts but also has small sections of the map. Aswell as this as you go deeper down there are different styles which I could use to represent the difficulty of the area and the type of monsters that inhabit it.



Ladders are used to traverse further down the mine. In my game I feel I could do something similar like having stairs that go further down the dungeon. However, I do feel as it could stop the flow, I want with my game so when I use this feature it will be sparingly and only to get the player to the next stage. The way they are found are by mining rocks or killing monsters. In my game unlike Stardew Valley you cannot mine rocks, but I would also like a way for the player to still do something to find the entrances without killing monsters as it gives the player a chance to progress when their health is low while keeping it challenging. So, I will have to have find another way to implement this feature.

I like the top-down view the play is given throughout the game as it gives the player more freedom with how they would like to traverse the level. I feel this would work well in my game as it gives the player more freedom to walk around and the other game, I researched Cult of the Lamb also does this so feel that a top-down is one of the key components to make this style of game. However, there are still many rogue likes that do have a 2d platformer look but I do prefer the using the top-down view.



In Stardew valley you can also find different weapons with different attributes which adds a variety to all the weapons however the progression with weapons is very linear which is not always a bad thing as it makes it simpler for the player to understand. However, having a more complex combat system like seen in Cult of the Lamb adds more creativity to the player so I will have to chose which one to focus on for my game.

The combat in this game is very repetitive as there aren't many different attacks that someone can do as there are 2 weapon types in the game swords and slingshots. The slingshots are for ranged attacks and use stones as their ammunition which is something you can find easily in the

mines, the way this works is by shooting a stone to where the mouse is pointing this is very simplistic and easy to use so if I implement ranged attacks, I feel this will be a good aspect take from this game. The way swords are used in this game is you click to left swing your sword and right click to block or do a special attack depending on the sword you have this is quite simplistic which is a good thing for new players however I feel that the mechanics of swinging swords are clunky as your player is forced to stand still while swinging their sword and the animations with some of the weapons take ages to play out which is very annoying at times so I my game I would to make sure the combat is smooth and not annoying.

Summary

The other systems

I will be taking the good aspects from the two other systems I researched and put them into my program. From Cult of the lamb, I like the movement as it feels very fluid and would like my movement from my game to be similar, I also like the different attacks/combat from the different weapons that you can have so I will use that aspect of the game in mine however on a smaller scale with only a few different weapon types from the player to choose from. I also like the generation from cult of the lamb but more specifically the traversal through levels where you can select what type of level you want to go to which I will use in my game so that the player can strategically chose where they want to go and so that they can go into harder levels to get better rewards down the path. From Stardew Valley I researched the mines of the game as the other aspects would not be relevant to what I am coding. I really like the generation of the mines as I feel it fits the theme of my game and I would like the game to have a more natural feel which the generation of the levels in the mine of Stardew valley give. From its combat what I like is the attribute that weapons get as they are varied and can change up the playstyle however I would not like my game to end in the position of Stardew Valley where there is a best weapon in the game so I will have to make sure all weapons are balanced and there are a couple weapons that are powerful for the end game so that various playstyles are still viable at the end of the game.

The algorithms

The algorithm I researched was A* algorithm a pathfinding algorithm. I will use this algorithm for the pathfinding of the monsters in the game. I like that it is efficient and the base concept makes sense and is quite simple however when it comes to actually coding/implementing it, it can get complex with the costs being one of the core aspects of this. Overall, I feel that this algorithm is a good choice as the base concept is simplistic however it is complex enough to show an advanced pathfinding.

End user interview

End user analysis

The target demographic needs to be someone who likes to play video games, it would also be beneficial for them to like dungeon crawler and monster hunter games.

My end user does play video games at least once per week, so they are interested in the hobby. Their technical skill is quite advanced with their knowledge of how coding works and their approach to games is more casual than competitive. I feel this is good as I want to create a casual experience that is also challenging at times so having an end user that likes casual play but is also experienced so that they can enjoy more challenging gameplay is optimal. They are also interested in rogue likes which is the genre of the game I researched (cult of the lamb). The reason I chose them is because they also see this gap in the market where this type of game does not have a very big audience so

they would like to experience this type of game they can also give useful feedback on my game and other games so that I can take the parts they like from other game sand implement them into mine which will create a better game in the end. It will also increase the scope of the game as I will they will help me take aspects from other games so fans of those games will also like the same aspects they like from a game in my game. The feedback they will give on my game will be beneficial as well as from their way they like to play games they give a good idea of how other people that like to play video games, of the roguelike and monster hunter genre, will also react to my game.

Interview

With the combat in games do you prefer a system based around weapons and items you pick up or systems based around the abilities the character has and can get please explain your answer?

I prefer the items like in breath of the wild for example as it encourages and rewards exploration and offers a unique experience each playthrough.

How much do you think the stats of the player should influence the game do you think they should be purely combat based or that they should also affect things outside of combat please explain your answer?

All skills should influence combat but not directly for example movement speed effects combat indirectly and attack speed effects the combat directly.

How do you think the levels/setting should look do you think they should more natural and random like in Stardew valley in the mines or more uniform and in rooms like those of in a dungeon please give the reasons for which one you picked?

I feel like a more natural setting would work better for the game. I say this because the vibe I'm getting is "hunting" as opposed to "monster slaying", I feel like a dungeon created be people/monsters doesn't really lend itself to hunting, whereas a natural environment is more suited to hunting.

For the individual levels of the dungeon do you think that it will be better to have a choice for the player to a type of level to go down into or that it should be random so that the player can discover what will be on the next level?

I feel that the game would work best with the player given a choice between a few options, these being randomly generated, as it allows for strategy and planning, while still incorporating an element of randomness, for example you could set off to hunt with a certain monster in mind, and the choice of floors allows you to plan your adventure to eventually find said monster.

What do you think about games with different weapon types do you enjoy them, if so, what do u enjoy/ not enjoy about them?

With different weapon types, I think they are a good thing as it allows for customisation and a unique playstyle for each player, they also allow for a player to have a different experience with the game by choosing to play a different weapon type. However, this only works if the different types are balanced correctly, as having one type of weapon be objectively better than the rest leads to stagnation as it becomes the only viable choice, therefore rendering the game as if there is only one type of weapon.

Do you think a game could benefit from a more simplistic weapon system if so how or how could it not and do you think that it is better than a complex weapon system with different types?

I think that with the style of game you are creating multiple, simple weapons would work better (for reasons discussed above), but one weapon with very fleshed out mechanics, i.e., many combos/skills, can lead to the same effect as many different, simpler weapons. What you shouldn't aim to do is try to include many weapons that each have their own incredibly detailed systems, because this can lead to the player feeling overwhelmed and ending up in a place where, because they have so many options, they end up not making any decisions to vary their playstyle as it seems too daunting to have to learn a whole new system. To put is simply, you need to pick either quantity or quality, without choosing both.

Interview summary

From the interview I was given very useful answers which will help me in the objectives and design aspect of the game. For the combat system I feel that from the feedback given I should focus on very few weapon types because I feel the quality of the game will be better than having lots of different weapon types. This is because the End User feels the replay ability will increase as you can replay the game with a different fighting style so that it feels different and unique. The stats of the player should also not be the backbone for the combat it should be more focused on the gear and weapons the player finds as it makes the gameplay more unique as you don’t know what you could find next, and the End User prefers more loot focused games like breath of the wild. For the level generation it will have to be quite complex for it to look natural as the End User feels this style of game could benefit from that as you are hunting monsters and it makes more sense for them to be in natural environments.

Objectives

Character Creator

- A simplistic character creator where players chose:
    - Their name
    - Their starting stats through spreading them out from a pool of points (20)
    - Select different avatars for their character.
    - This information will be used to create a class for the character.

Character Class

- A class consisting of the stats of the player and all the actions the player can do.
    - Allowing the player to move on the screen.
        - With their images
        - With their rect boxes
    - Allowing the player to attack
        - TO show a visualisation of them attacking
        - To have it deal damage to an enemy if it would hit them.
    - Add a dash that the player can do.
        - When the player pressed left control
        - The player will jump
        - Towards to where the mouse curser is

Level Generation

- A procedural generated dungeon
    - A class consisting of the objects used for the walls and floors.
        - Walls class
            - I must be able to have a position given to it
            - It must have the image/texture that the wall is
            - Have it so that the player cannot walk through
                - Use rect.collide for the collisions
        - The floor will be the background of the level
        - There will be environmental class
            - Must have positional arguments to show where to put it on the map
            - Different textures/images to pick from
            - Have collisions
                - Use mask collisions
                    - Create a mask of the texture for the collision
                    - When checking for collisions check the rect.colide before mask as mask take sup more processing time
                - When they have collided allow the player to walk through them so that can hide behind the environmental items
        - There will be an exit class
            - It must have a unique image
            - It must have collisions
                - Rect collisions
            - Always looking for an enter input
            - When there is a collision and enter input if the exit clause is fulfilled
                - The player will exit the level and go into the next level.
            - The exit clause
                - Will be to solve puzzle
                - Will be to kill every monster
    - An algorithm to randomly create a map of the dungeon which will be populated with the objects.
        - Map class

- This class starts of with creating a blank list of lists of 0`s (2d array) that represents the screen
- Function to give map away
    - Return map
- Function to receive maps
    - An update of the self.__map variable
- The Map class will then create a room class to the number of rooms in the class
    - Added to a list called rooms
- Function called connections to connect the rooms together
    - Connects the room its looking at to the next room in the rooms list
    - Find out which room is bigger
    - The centre of the smaller rooms wall facing the other room is used as the centre of the pathway between the rooms
    - Remove the space for the pathway
    - Add the top and bottom walls of the pathway into the other room
    - Have the gap/ width of the pathway be 2 wide so that the player can pass through but the monsters can`t

- Room Class
    - Receive the map from the Map class
        - Using the map.give_map function to receive the map
    - Find the starting point of this room by using a parameter of starting point which will be [0,0] for the first room
    - Start off by deciding the size of the room
        - Using random.randint to produce a random number between the range of 10 and 25 for the width and height
    - Make a basic shape of the rectangle of that size onto the map add 3 to both the starting height and width so that there is room for indents in the room in the future
    - Make the corners
        - Remove the original corners from the room
        - Select one of the 3 corners to map onto the room
            - The different corners are a set of instructions of how the program can produce the corners so they can be done from any point.
    - Make Indents
        - Top
            - Working Room is defined by how much room the top wall has above it.
                - This is by if it hits the top (index 0) of the map.
                - This is by if it hits another wall.
        - Bottom
            - Working Room is defined by how much room the bottom wall has bellow it.

- This is if it hits the bottom (index Len(map)) of the map.

- Left
  - Working Room defined by how much space is to the left of it.
    - How far the wall can go back until it hits the left edge of the map.
    - How far it can go left until it hits another wall.
- Right
  - Working Room defined by how much space is to the right of it.
    - How far the wall can go back until it hits the right edge of the map.
    - How far it can go right until it hits another wall.
- How they will all work
  - A gap is made in wall depending on the gap variable which will be random number between 5 and 15.
  - It will rise to the working room in stages of two blocks being pushed up by 1 and moved to right by 1 in every increment.
  - When at working room it will stay level until there is enough room to go down in same increments
  - It will go down in same increments.

- Implementation of an algorithm like Dijkstra's Algorithm or A* algorithm to make sure that it can be completed.
  - Run the algorithm
    - Until a path is found
      - To the exit
      - If it is found then exit function
        - Level can be completed
    - Or all paths have been explored
      - Level cannot be completed remake another level

Monsters AI

- A class for the stats of the monsters and the algorithms they will follow.
  - Implementation of A* algorithm for their pathfinding to or away from the player
    - The nodes
      - Will be each index of the map used to generate the level
        - Map will be gained from the map.give_map function
      - A target node will be set
        - Getting the pixel the rect positions are on
        - Then finding which node this lands into
          - Dividing it by 100 with //
          - This will give the index of where the pixel is in.
      - The starting node will be set

        - Same way as target node is set expect with the monsters rect
    - Make a visited list
    - Make an unvisited list
    - Set starting node as current node
    - From current node
        - Check all nodes around it
            - Check if it has nodes around it
                - Some hypothetical node areas may be out of the range of the map
                - Therefore you must check whether it is in range of the map first
        - Check if each node is traversable
            - If it is a 0 on the 2d array
            - If it is not then it is excluded from the algorithm
        - Visit one with lowest distance
            - One distance is distance of travel to that node
            - One distance is the heuristics to the target node
        - Visited node is added to visited list
        - Visited node is made current node
    - Loop this until current node is target node

- Allowing the enemies to attack
    - with an animation to represent it to the player
    - Make sure the attack collides with the player mask.
        - If it does take away hp from player with damage taken function
- To make it so that they can take damage and die.
    - Have their hp checked every tick so that if they are bellow 0 there death starts
    - To have a death animation and or body left on the ground so they player knows when they have been killed.
    - To have them drop their loot when they have been killed.
- Have the spawn rate of the monsters drop depending on how many monsters the player has killed.
    - Track how many monsters of one type the player has killed
        - An array inside of the player class
            - That is passed during generation
                - With a give_array function
    - Have the spawn rate of the monsters be changeable.
        - Take away from the percentage depending on how many monsters of that type have been killed

Modelling

The core concepts

In this game the player will enter a level and then explore the procedurally generating map until they find a monster which they will fight to kill to get its monster parts. This loop continues until the player dies or they find an exit. If they do exit, then they will get a score based on how many monster parts they collected which will be converted into gold which they can use to by upgrades for there stats. Throughout the level they will be able to find chests which will house new weapons so that the gameplay doesn`t get boring.

Pygame is the plugin I will be using to display what happens to the player so I will produce a basic pygame program:

```
import pygame

1 usage
class Player(pygame.sprite.Sprite):

    def __init__(self):
        super().__init__()

        self.image = pygame.Surface([25,25])
        self.image.fill('#FFFFFF')
        pygame.draw.rect(self.image, color: '#8DD8CC', rect: [0,0, 25, 25 ])
        self.rect = self.image.get_rect()
        self.rect.x = 0
        self.rect.y = 0

    1 usage
    def Movement(self):
        input = pygame.key.get_pressed()
        if input[pygame.K_LEFT] and self.rect.x > 0:
            self.rect.x -= 1

        if input[pygame.K_RIGHT] and self.rect.x < 775:
            self.rect.x += 1

        if input[pygame.K_UP] and self.rect.y > 0:
            self.rect.y -= 1

        if input[pygame.K_DOWN] and self.rect.y < 575:
            self.rect.y += 1

    def update(self):
        self.Movement()

pygame.init()


Display = pygame.display.set_mode((800,600))
player1 = Player()
players = pygame.sprite.Group()
players.add(player1)
run = True

while run:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False

    Display.fill('#FFFFFF')
    players.draw(Display)
    players.update()
    pygame.display.update()

pygame.quit()
```

At the top of the code a class is created which is made the child of Sprite class from pygame. Inside of this class the image and rects are created by overwriting the code in the pygame.sprite.Sprite class. There is then a function movement which waits for the inputs of the player and then if it corresponds with one of the inputs and then moves the rect and image of the sprite in that direction but making sure it cannot exist the display. The update function is then overwritten with the movement function inside of it so that it is always being ran. Pygame is then initialised and the display, player1 and players group is assigned a loop is then made but to make sure that an infinoite loop is not created there is a for loop that loops through the events in pygame and if thequite event is fulfilled the loop stops and the program closes after this the display is made white and the players group and display is updated so that the last tick/frame is saved onto the display.

Pathfinding

For the pathfinding of the enemies, I will be using A* algorithm. This aspect of the code needs to be modelled before I start to code my game because if it can`t be done then the game would not work as intended as the enemies would either stand still or move completely randomly. This algorithm will be implemented inside of the Monster class for its pathfinding towards the player. The way this will be implemented is that a grid will be placed over the level, that the player cannot see, each square will be the size of one player/enemy/sprite to speed up the searching as it would be quicker to search through this grid than to search through all the pixels on the canvas. After the search is completed and the enemy knows where the player is it will start moving towards them by seeing where they must move based off their X and Y cords which will be done using if statements.

Prototype code:

```
import math
import heapq
#NOTE: never change formating halfway through coding
#NOTE: Avoid hardcoding something that you need to test with different values
1 usage
class Node:

    def __init__(self,row, colum, target):

        self.__parent = None
        self.__row = row
        self.__colum = colum
        self.__start_to_cell = float('inf')
        self.__Heuristic = Get_heuristic(self.__row, self.__colum,target)
        self.__cost = float('inf')
        self.__target = target



    def getcolum(self):
        return self.__colum
    def getrow(self):
        return self.__row
    1 usage
    def get_start_to_cell(self):
        return self.__start_to_cell
    1 usage
    def get_Heuristics(self):
        return self.__Heuristic

    2 usages
    def get_cost(self):
        return self.__cost


    6 usages (6 dynamic)
    def get_parent(self):
        return self.__parent

    def set_colum(self, data):
        self.__colum = data

    def set_row(self, data):
        self.__row = data

    2 usages
    def set_start_to_cell(self,data):
        self.__start_to_cell = data

    1 usage
    def set_cost(self, data):
        self.__cost = data



    3 usages
    def set_parent(self, data):
        self.__parent = data
```

```
class Grid:
    def __init__(self):
        self.__ROW = 9
        self.__COLUM = 10
        self.__grid = [
        [1, 0, 1, 1, 1, 1, 0, 1, 1, 1],
        [1, 1, 1, 0, 1, 1, 1, 0, 1, 1],
        [1, 1, 1, 0, 1, 1, 0, 1, 0, 1],
        [0, 0, 1, 0, 1, 0, 0, 0, 0, 1],
        [1, 1, 1, 0, 1, 1, 1, 0, 1, 0],
        [1, 0, 1, 1, 1, 1, 0, 1, 0, 0],
        [1, 0, 0, 0, 0, 1, 0, 0, 0, 1],
        [1, 0, 1, 1, 1, 1, 0, 1, 1, 1],
        [1, 1, 1, 0, 0, 0, 1, 0, 0, 1]
    ]

    1 usage
    def valid_check(self, row, colum):
        if row>=0 and row<self.__ROW and colum>=0 and colum<self.__COLUM:
            return True
        else:
            return False

    1 usage
    def cell_blocked(self,row,colum):
        if self.__grid[row][colum] == 1:
            return True
        else:
            return False
```

```
def run(self):
    open_list = []
    closed_list = []
    starting = [8,2]
    target = [0,3]
    target_found = False
    Nodeslist = [[Node(i,j,target) for i in range(len(self.__grid[1]))] for j in range(len(self.__grid))]
    #sets up starting node
    heapq.heappush(open_list, (0.0, starting[0], starting[1]))

    Nodeslist[starting[0]][starting[1]].set_start_to_cell(0)

    Nodeslist[starting[0]][starting[1]].set_parent((0.0,starting[0],starting[1]))

    while len(open_list) > 0:
        parent = heapq.heappop(open_list)

        closed_list.append(parent)

        movements = [(0,1),(0,-1),(1,0),(-1,0),(1,1),(-1,-1),(1,-1),(-1,1)]
        for direction in movements:
            row = parent[1]
            colum = parent[2]
            new_row = row + direction[0]
            new_colum = colum + direction[1]
            child = [new_row,new_colum]

            if self.valid_check(new_row, new_colum):
                if self.cell_blocked(new_row,new_colum) and not is_closed(closed_list, child):

                    if is_target(new_row, new_colum, target):
                        Nodeslist[new_row][new_colum].set_parent(parent)
                        print('target found')
                        trace_path(Nodeslist, target)
                        return
```

```
                    return

                else:
                    Cell_cost = Nodeslist[row][colum].get_start_to_cell() + 1
                    new_cost = Cell_cost + Nodeslist[new_row][new_colum].get_Heuristics()

                    if Nodeslist[new_row][new_colum].get_cost() == float('inf') or Nodeslist[new_row][new_colu
                        heapq.heappush(open_list, (new_cost, new_row, new_colum) )
                        Nodeslist[new_row][new_colum].set_start_to_cell(Cell_cost)
                        Nodeslist[new_row][new_colum].set_cost(new_cost)
                        Nodeslist[new_row][new_colum].set_parent(parent)

        if not target_found:
            print('target not found')
```

```
1 usage
def is_target(row,colum,target):
    if target[0] == row and target[1] == colum:
        return True
    else:
        return  False
```

```
1 usage
def Get_heuristic(row,colum,target):
    t_row = target[1]
    t_colum = target[0]
    Distance_X = abs(row - t_row)
    Distance_Y = abs(colum - t_colum)
    Heuristics = (Distance_X + Distance_Y) + (math.sqrt(2) - 2) * min(Distance_X, Distance_Y)
    return Heuristics
```

```
def trace_path(Nodeslist, target):
    print("Found: ")
    path = []
    row = target[0]
    colum = target[1]
    P_row = Nodeslist[row][colum].get_parent()[1]
    P_colum = Nodeslist[row][colum].get_parent()[2]

    while  P_row != row or P_colum != colum:
        path.append((row,colum))
        new_row = Nodeslist[row][colum].get_parent()[1]
        new_colum = Nodeslist[row][colum].get_parent()[2]
        row = new_row
        colum = new_colum
        P_row = Nodeslist[row][colum].get_parent()[1]
        P_colum = Nodeslist[row][colum].get_parent()[2]

    path.append((row, colum))
    path.reverse()
    for i in path:
        print(i)
```

```
1 usage
def is_closed(closed_list, coords):
    for i in closed_list:
        if i[1] == coords[0] and i[2] == coords[1]:
            return True
    return False

grid = Grid()

grid.run()
```

Output:

```
target found
Found:
(8, 2)
(7, 3)
(7, 4)
(6, 5)
(5, 4)
(4, 4)
(3, 4)
(2, 4)
(1, 4)
(0, 3)
```

Level Generation

The level generation is an important aspect of the project as well as without it the game would not be procedurally generating. So I will be prototyping a simple algorithm that joins together 2 rooms by using a list of lists to generate a map that will then be used to make a block if the data inside of one slot is a 1 and if it is it will then be given an x cord and a y cord to be put onto the display of the pygame so that when they all come together it produces the map onto the grid.

At the top of the code there is a block class which is a block that is black and 40 pixels long and 30 pixels high. After this the map is set up which is a list of lists of 1's and 0's where 1 means that a block and 0 means that empty space is going to be left there as this list is used to represent the display as each individual list is a space 40,30 it puts it as zero as default after this it then makes the first room from randomly generating the width and length and then doing a for loop to change the 0's into 1's on the map after this a decision is made whether to make the second room to the side or bellow by whether a random number is a 1 or a 0. After this the same code is used to generate the second room except the cords are changed so that it is built to the side of or below by having a zero variable which is added onto it. Then the middle of the smallest room is found and a pathway is created between them so that they are connected. The map is then read using a for loop and checks whether that space should have a block or not, whether or it's a 1 or not, which creates the map on the pygame display.

```
import random
import pygame
#Classes

1 usage
class Block(pygame.sprite.Sprite):
    def __init__(self,x,y):
        super().__init__()
        self.__colour = '#000000'
        self.__width = 40
        self.__height = 30
        self.image = pygame.Surface([self.__width, self.__height])
        self.image.fill((255, 255, 255))
        pygame.draw.rect(self.image, self.__colour, rect: [0, 0, self.__width, self.__height])
        self.rect = self.image.get_rect()
        self.rect.x = x
        self.rect.y = y

Map = []

#set up of layout
map_size = [20,20]

for row in range(map_size[0]):
    Map.append([])
    for slot in range(map_size[1]):
        Map[row].append([0])
#first room

num = random.randint( a: 3, b: 10)
num2 = random.randint( a: 3, b: 10)


for slot in range(num):
    Map[0][slot] = [1]
    Map[num2][slot] = [1]

for slot in range(num2):
    Map[slot][0] = [1]
    Map[slot][num] = [1]

Map[num2][num] = [1]

# sets up second room
random_number = random.randint( a: 0, b: 1)
if random_number == 1:
    Zero = num + 2
    checking = True
    while checking:
        N_num = random.randint( a: 3, b: 10)
        N_num2 = random.randint( a: 3, b: 10)
        if N_num + Zero < 20:
            checking = False

    for slot in range(N_num):
        Map[0][slot + Zero] = [1]
```

```
57
58      for slot in range(N_num2):
59          Map[slot][0 + Zero] = [1]
60          Map[slot][N_num + Zero] = [1]
61
62      Map[N_num2][N_num+ Zero] = [1]
63
64      if num2 < N_num2:
65          centre = num2//2
66
67      else:
68          centre = N_num2//2
69
70      Upper = centre - 1
71      Lower = centre + 1
72      middle = Zero - 1
73      Map[Upper][middle] = [1]
74      Map[Lower][middle] = [1]
75      Map[centre][Zero] = [0]
76      Map[centre][num] = [0]
77
78  else:
79
80      Zero = num2 + 2
81      checking = True
82      while checking:
83          N_num = random.randint( a: 3, b: 10)
84          N_num2 = random.randint( a: 3, b: 10)
85          if N_num2 + Zero < 20:
86              checking = False
87
88      for slot in range(N_num):
89
90          Map[Zero][slot] = [1]
91          Map[N_num2 + Zero][slot] = [1]
92
93      for slot in range(N_num2):
94          Map[slot+ Zero][0] = [1]
95          Map[slot + Zero][N_num] = [1]
96
97      Map[N_num2+ Zero][N_num] = [1]
98
99      if num < N_num:
100         centre = num // 2
101
102
103
104     else:
105         centre = N_num // 2
106
107     middle = Zero - 1
108     Upper = centre - 1
109     Lower = centre + 1
110
111     Map[middle][Upper] = [1]
112     Map[middle][Lower] = [1]
```

```
    Map[middle][Upper] = [1]
    Map[middle][Lower] = [1]
    Map[Zero][centre] = [0]
    Map[num2][centre] = [0]



# displaying of the room
for i in Map:
    print(i)

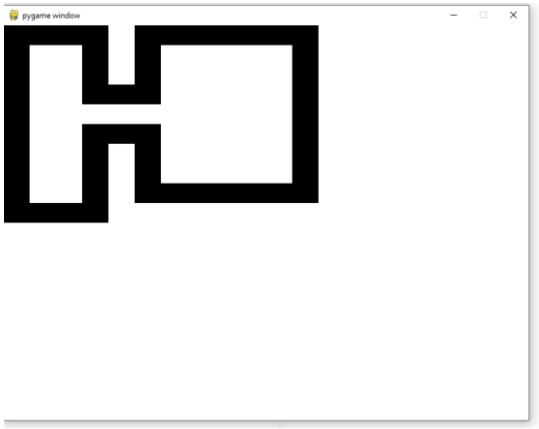
pygame.init()

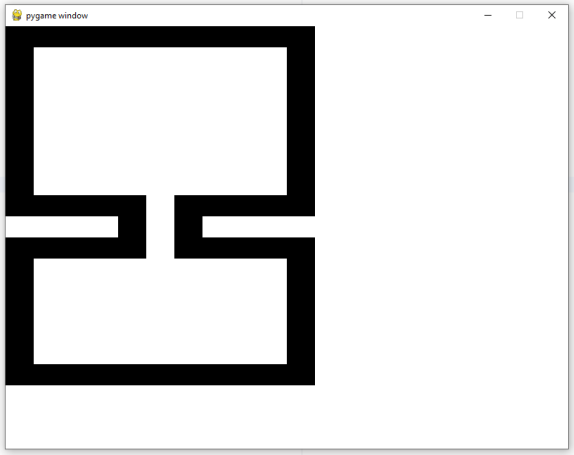
surface = pygame.display.set_mode((800,600))
blocks = pygame.sprite.Group()

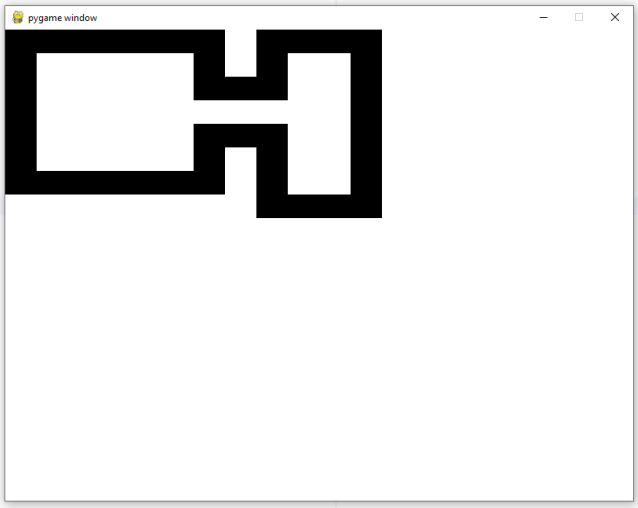
for i in range(len(Map)):
    for x in range(len(Map[0])):
        if Map[i][x] == [1]:
            slot = Map[i][x]
            x_cord = x * 40
            y_cord = i * 30
            block = Block(x_cord, y_cord)
            blocks.add(block)

run = True
while run:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
    surface.fill((255, 255, 255))
    blocks.draw(surface)
    blocks.update()
    pygame.display.update()
```

Output:







Design

Mission Statement

This is a game where you will go into a dungeon to kill monsters to sell their parts if you are to get to a rest point, at these rest points you will be able to purchase items from a shop and sell the items you have gained throughout the levels. The player will be able to attack using their fists or a sword which will have unique playstyles and they will have movement where they can walk, sneak and dash which have unique sound levels for the monsters to hear and different speed levels with dash being a burst of speed. When you die you go back to where you were at the last rest point you got to so they will start to get rarer the further into the game you get. The monsters will try to attack the player if they it sees them or hears them. The player will be able to find loot/chests throughout the levels to get better gear. The dungeons will be more themed towards caves and forests than a classic

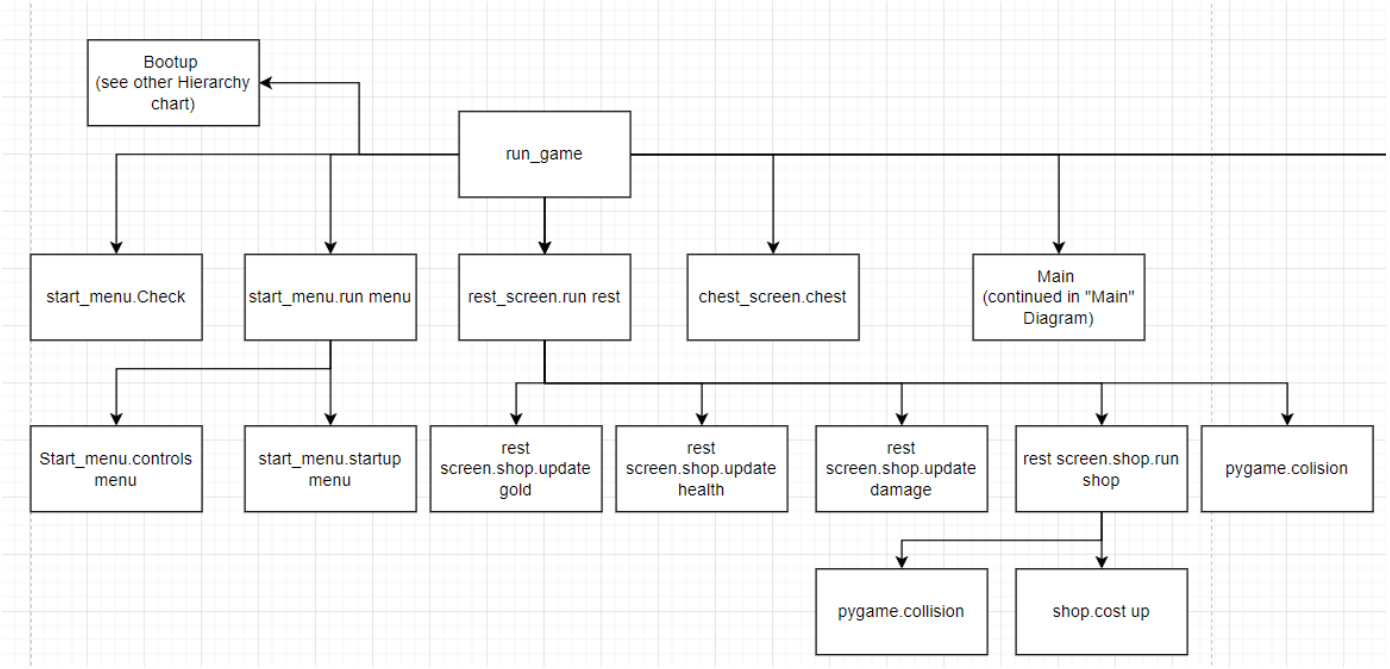
dungeon so that the game is more interesting and makes it unique to the other games in similar genres.

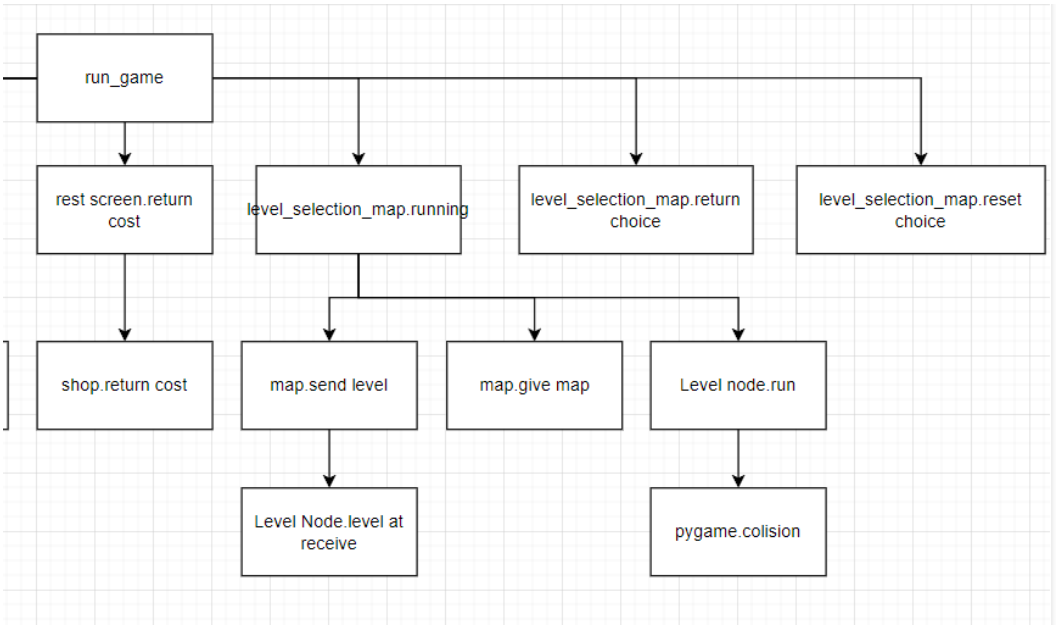
Design Introduction

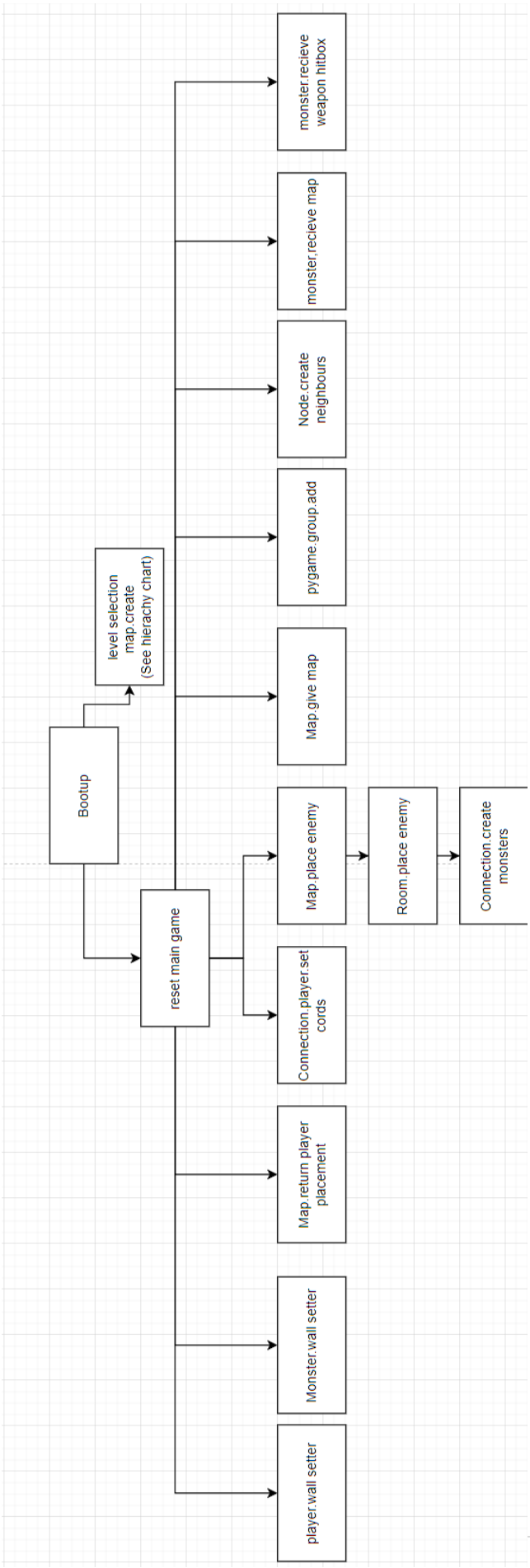
Introduction

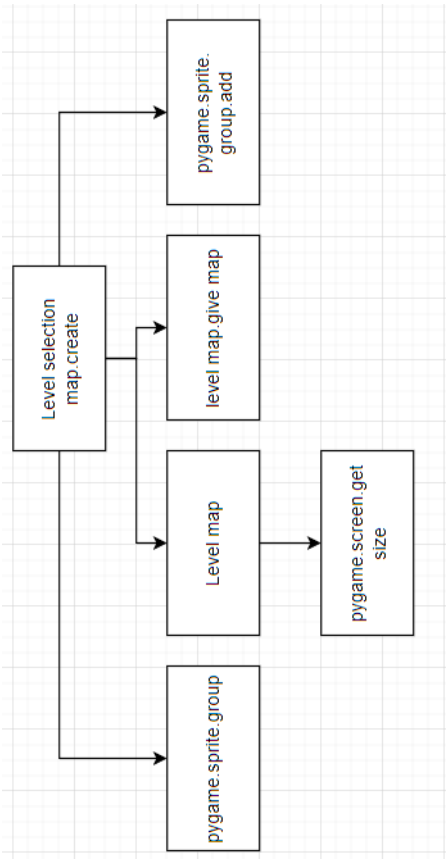
The way I will design my game is by splitting up into 3 core aspects of the game which are The characters and the AI`s / enemies, the generation and the main game for most aspects of the design they will be separated except for when I am looking at the game as a whole. Firstly, I will create a Hierarchy diagram to look at the program as a whole. I will go over the key algorithms of the game in depth so that they can be understood. This will be done using pseudocode, thorough explanations and diagrams if needed. For the classes I will be creating a class diagram which will go through the parameters/returns of methods as well as the methods themselves. It will also show the relationships between the other classes that they have. Then every function will be listed in a table to explain what it does and the returns and parameters of each function. Then I will go through how the project folder is organised and what the file structure looks like.

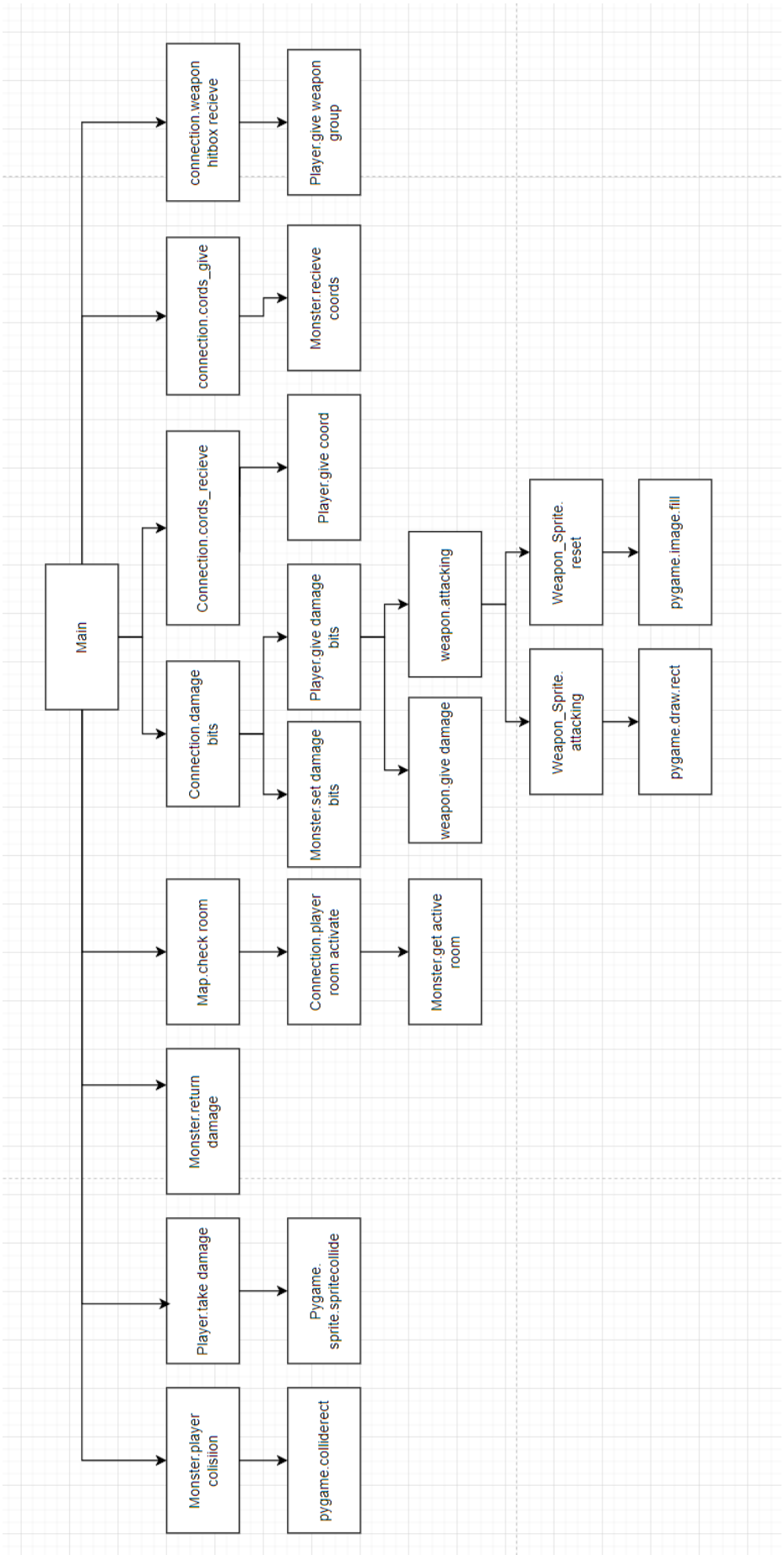
Hierarchy Charts

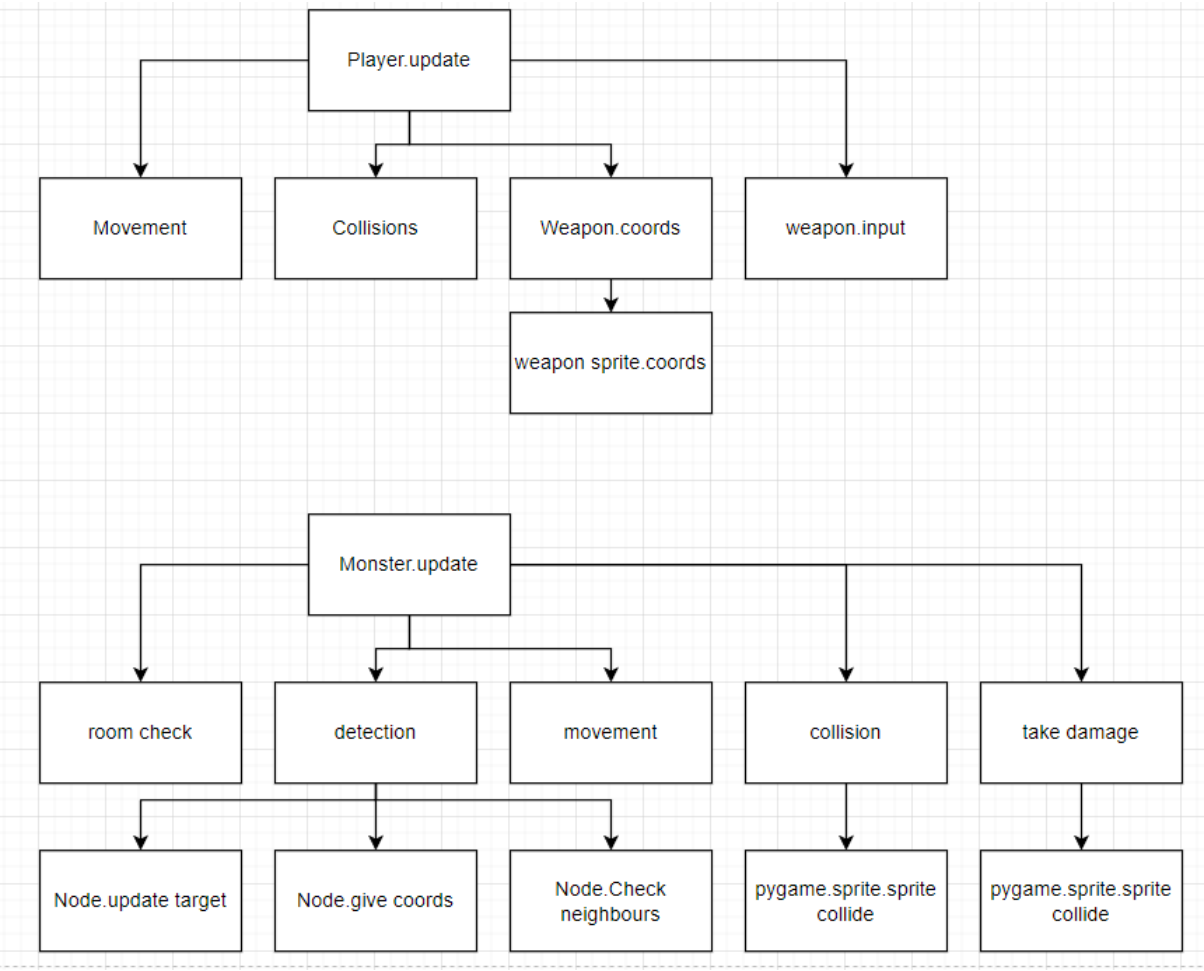





Bootup
level selection map.create (See hierachy chart)
reset main game
player.wall setter
Monster.wall setter
Map.return player placement
Connection.player.set cords
Map.place enemy
Room.place enemy
Connection.create monsters
Map.give map
pygame.group.add
Node.create neighbours
monster,recieve map
monster.recieve weapon hitbox


Level selection map.create
pygame.sprite.group
Level map
pygame.screen.get size
level map.give map
pygame.sprite. group.add


Main
connection.weapon hitbox recieve
Player.give weapon group
connection.cords_give
Monster.recieve coords
Connection.cords_recieve
Player.give coord
Connection.damage bits
Player.give damage bits
Monster.set damage bits
weapon.attacking
weapon.give damage
Weapon_Sprite. reset
pygame.image.fill
Weapon_Sprite. attacking
pygame.draw.rect
Map.check room
Connection.player room activate
Monster.get active room
Monster.return damage
Player.take damage
Pygame. sprite.spritecollide
Monster.player collision
pygame.colliderect



Data Structures

In my program for the generation, it will be built using a 2d array that is the size of 70 by 70 which leaves the screen to split up into sections of 20by15 pixels. In this array the first number in the index is the y coordinate as that is to select each row in the array. The second index is the x coordinate as it is used to select which column in the array. The data stored in the array are the integers either 0 to 4 0 is an empty slot, 1 is that there is a wall there, 2 is an environmental(A) piece is placed, 3 is an environmental piece(B) is placed and 4 is the exit of which there will only be one of. This 2d array is used for the generation of the level

This array also has another very similar to it where instead of integers it has objects of the node class for each of the slots in the array and is used for the pathfinding of the enemies.

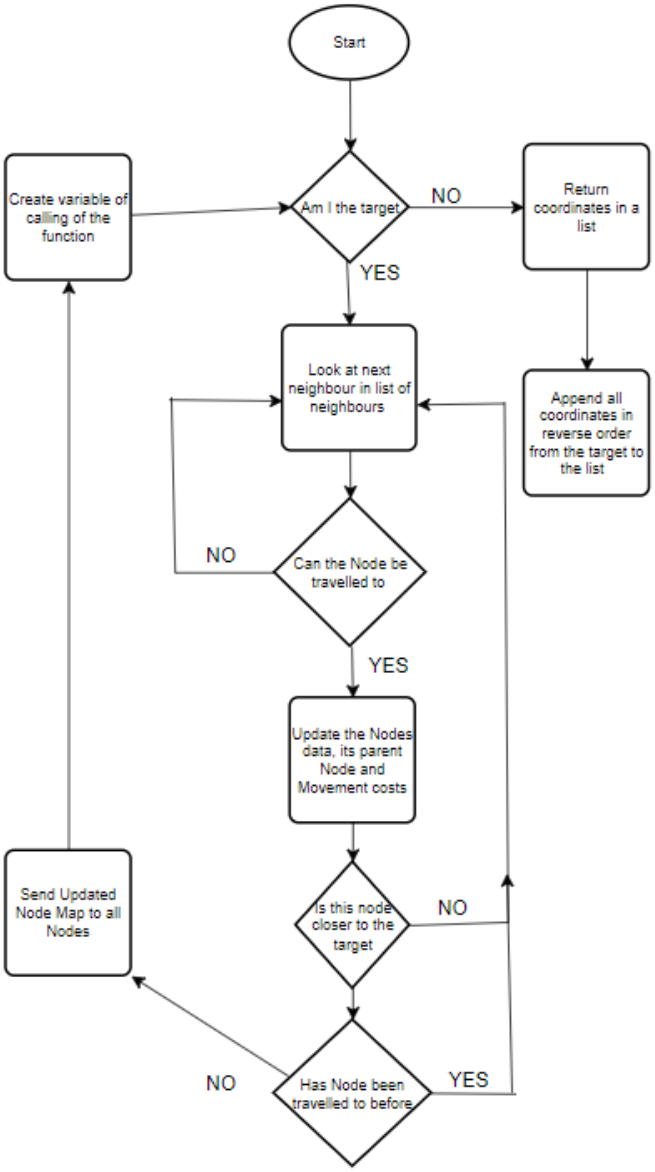
When ever coordinates are being used they will be put into a list of [y cord,x cord] this is because of the formatting of the level map 2d arrays.

In the connections class there will be a list of all the enemies so that when you need to send something to all enemies you can just loop it through that list. The items inside of it will be objects of the Monster class.

In the character class there will be a list of the last 5 previous moves the player has done so the data stored will be “W” , “A”, “S” and “D” and will be used to backtrack the player when they have collided with the wall.

Key algorithms

A-star Algorithm



The A* algorithm is an algorithm that is used for my enemies to track the player I have created a flow chart to show how the algorithm works. This algorithm is very important as without it the enemies would not be able to track the player.

Pseudo Code:

Function CheckNeighbours():

```
Travelled()  // This is used to change a variable to state that it has been traversed//
newDistance = giveDistance()
if Own coordinates == targets coordinates
self: Traversed = False
return [own coordinates]
end if
for neighbour inside list of neighbours
Node = NodeMap(position: neighbour) // Neighbour is a list of a coordinate//
If Node.checkIfTraversable()
Node.setParent(own coordinates)
Node.UpdateMovement(neighbour(position: 2)
Node.UpdateTarget(target coordinates)
If ownDistanceToTarget < NodesDistanceToTarget
nextToBeTraversed = Node
Map(position of neighbour) = Node //Updates the node on the map//
For loop each NewNode in Map
NewNode.updateMap(Map)
List = Node.CheckNeighbours()
If list == None
List = []
End if
Self.traversed = False
Append (own coordinates) to list
Return list
End if
End if
```

Map Generation

For the map generation the main algorithm used is the Room class's initializer. It uses a 2dlist full of 1's and 0's or other numbers for objects. A 0 signifies a blank space and a 1 signifies a wall. Firstly, it first randomly chooses a width and height by generating random numbers. A square of 1's is put onto the 2d list with the top left corner being a parameter sent in when creating the room class. It then cuts off the corners by making the 5 1's on the corners into 0's. Then randomly selects one of

the preset corners for each corner. The presets are like step-by-step instructions on adding 1's back onto the 2d list to create different types of corners. It then takes away the 1's on the left side and then creates a smaller line than the gap away from the side up to 3 spaces away it then creates 1's in 2 by 1 blocks going from the edge of the gap to the new line on both sides. It then does this to the right side.

For when objects/ enemies are placed into the room. It generates a random number within the width and height of the room, and it checks if that spot is a 0 if it is then it places it there if not then it is not placed there. The exit is only placed into one of the rooms.

The 2 corner presets

idea 1
[1][1][0][0]
[0][1][0][0]
[0][1][1][0]
[0][0][1][1]

```
map[0 + self.__zero_height][0 + self.__zero_width] = 0
map[0 + self.__zero_height][1 + self.__zero_width] = 0
map[1 + self.__zero_height][2 + self.__zero_width] = 1
map[1 + self.__zero_height][0 + self.__zero_width] = 0
map[2 + self.__zero_height][0 + self.__zero_width] = 0
map[2 + self.__zero_height][2 + self.__zero_width] = 1
map[2 + self.__zero_height][1 + self.__zero_width] = 1
map[3 + self.__zero_height][1 + self.__zero_width] = 1
```

idea 2
[1][1][0][0]
[0][1][1][0]
[0][0][1][0]
[0][0][1][1]

So a room would look like this:

[0][0][0][1][1][1][1][1][1][1][1][1][1][1][1][1][1][1][1][0][0][0][0][0]

[0][0][1][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][0][0][0][0]

[0][0][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][1][0][0][0]

[0][1][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][1][0][0][0]

[0][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][0][0][0]

[0][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][0][0][0]

[0][1][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][1][0][0][0]

[0][0][1][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][1][0][0][0]

[0][0][0][1][0][0][0][0][0][0][0][0][0][0][0][0][0][0][0][1][0][0][0][0]

[0][0][0][1][1][1][1][0][0][0][0][0][1][1][1][1][1][1][1][0][0][0][0][0]

[0][0][0][0][0][0][1][1][1][1][1][1][1][1][0][0][0][0][0][0][0][0][0][0]

When 2 rooms are connected together a gap in the sides of 2 walls closest to each other is made of equal length then the top row of 1's to bridge the gaps is made then the bottom this links the 2 rooms together. This happens once horizontally and once vertically due to the way that rooms are

placed onto the map where one is placed underneath the last created room and the other is placed next to the last created room. The order in which that happens alternates.

Table of what array would be like each slot in the table would be an index in the array

0	0	0	1	1	1	0	0	0	0
0	0	1	1	0	1	1	1	0	0
0	1	1	0	0	0	0	1	1	0
0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0
0	1	1	0	0	0	0	1	1	0
0	0	1	0	0	0	0	1	0	0
0	0	1	1	1	1	1	1	0	0

Table of shaded cells to show what the binary represents for the generation and in the program.

Collisions

The way that collisions are done is mainly through the sprite collide function between the sprite colliding and the group “walls” which is for the walls of the level. However, there is a list of the last moves that the enemy or player has done. So, when a player or enemy collides with the wall it will then do the reverse of its last moves until it is no longer colliding with the wall.

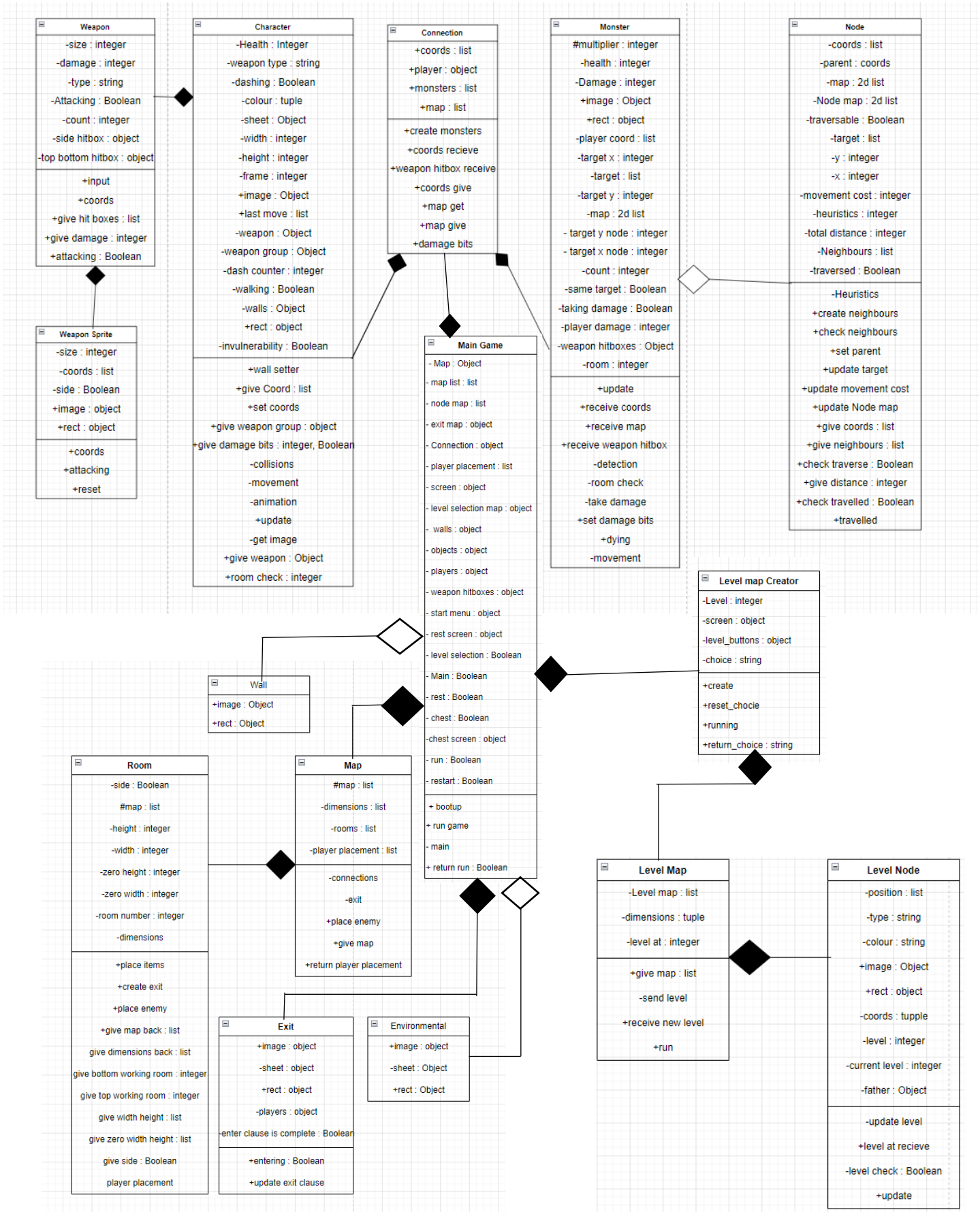
The list would look like this: [“W”,”S”,”D”,”W”, Ext.] where the letter corresponds to the key that was pressed to make that movement happen with monsters it is the key that would have been pressed to make that movement happen.

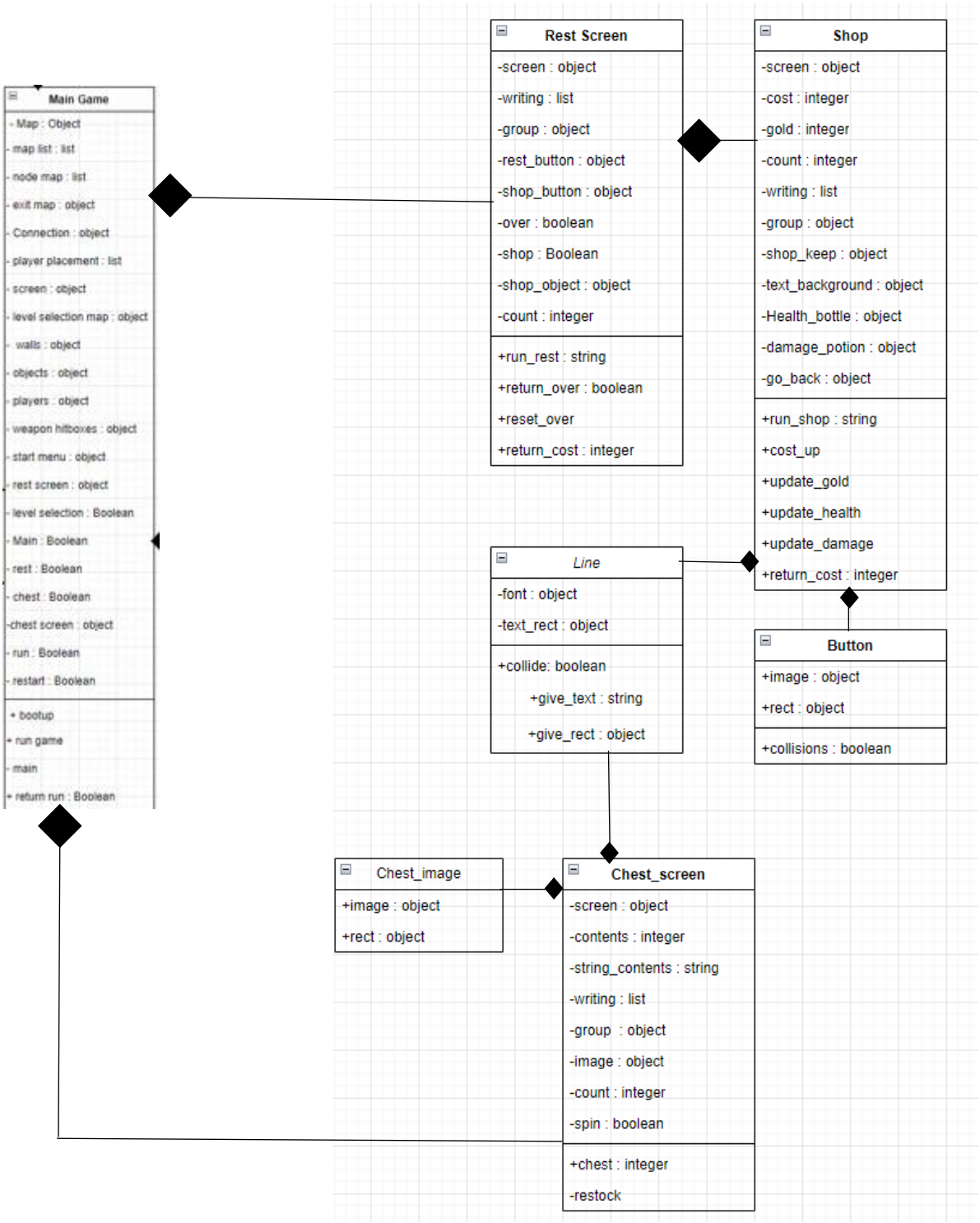
Pseudo code:

```
If Collision:
    For i in Movement list:
        If i is “W”:
            Y coord += 1
        If i is “S”:
            Y coord -= 1
        If i is “A”:
```

```
            X coord += 1
        If i is “D”:
            X coord -= 1
```

Class Diagrams




Main Game
- Map : Object
- map list : list
- node map : list
- exit map : object
- Connection : object
- player placement : list
- screen : object
- level selection map : object
- walls : object
- objects : object
- players : object
- weapon hitboxes : object
- start menu : object
- rest screen : object
- level selection : Boolean
- Main : Boolean
- rest : Boolean
- chest : Boolean
-chest screen : object
- run : Boolean
- restart : Boolean
+ bootup
+ run game
- main
+ return run : Boolean
Rest Screen
-screen : object
-writing : list
-group : object
-rest_button : object
-shop_button : object
-over : boolean
-shop : Boolean
-shop_object : object
-count : integer
+run_rest : string
+return_over : boolean
+reset_over
+return_cost : integer
Shop
-screen : object
-cost : integer
-gold : integer
-count : integer
-writing : list
-group : object
-shop_keep : object
-text_background : object
-Health_bottle : object
-damage_potion : object
-go_back : object
+run_shop : string
+cost_up
+update_gold
+update_health
+update_damage
+return_cost : integer
Line
-font : object
-text_rect : object
+collide: boolean
+give_text : string
+give_rect : object
Button
+image : object
+rect : object
+collisions : boolean
Chest_image
+image : object
+rect : object
Chest_screen
-screen : object
-contents : integer
-string_contents : string
-writing : list
-group : object
-image : object
-count : integer
-spin : boolean
+chest : integer
-restock

Functions

Main.py

Main game

Function Name	Parameters	Returns	Description
Initializer	None	None	This function sets up everything that stays constant in the game and creates all the variables this class uses.
__Main	None	None	This function is used when in a main level of the game and updates and draws all pygame groups needed. It also updates the display and runs the functions: C.C.coords_recieve() C.coords_give() C.weapon_hitbox_recieve() C.damage_bits()
bootup	None	None	This resets the game this includes the level selection screen as well as the main game levels. It does this by calling Reset Main Game and creating the level selection map object
Reset Main Game	None	None	This function is used to reset the dungeon levels so that another dungeon is generated. So, it assigs self.__map = Map() Which generates a new map and then adds that map to all of the pygame groups
Run_game	None	None	This function is the main loop of the game and where everything runs from
Return_run	None	Boolean	Returns the class variable run.

Chest screen

Function Name	Parameters	Returns	Description
Initializer	Object	None	Creates the text and image needed for the chest screen it also creates all the variables the class uses.
Chest	Boolean	Integer	This is the main code for the chest screen it creates a new value for the chest 200 times then stops this creates the illusion that the amount in the text is spinning to select an amount it then checks for if a collision and the mouse has clicked and then returns the value (amount of gold the chest has) and restocks the chest.
__restock	None	None	Creates a random number (0,100) this new value is now the new contents it then

			modifies the text according to the new amount of gold the chest has.

Chest image

Function Name	Parameters	Returns	Description
Initializer	None	None	Loads the image chest and creates the rect/hitbox for the image and sets the coordinates of where the image is placed.

Level map creator

Function Name	Parameters	Returns	Description
Initializer	Object	None	Creates the variables the class uses and assigns the pygame screen to a variable.
create	None	None	Resets the buttons and deletes the previous ones and then creates a new level map class and adds the buttons of that class to pygame group level buttons. Then the difficulty variable is increased by 1.
Reset_choice	None	None	assigns the class variable of choice to None/nothing.
running	None	None	Loops through every level node in the level map and checks if they return a choice if so that choice is assigned to the class variable choice It then runs the function send_level and makes the screen white and puts the buttons onto the screen and then updates the display.
Return_choice	None	String	Returns the class variable choice.

Shop

Function Name	Parameters	Returns	Description
Initializer	Object	None	It creates the text of the shop with the Line class loads the images that are used on this screen sets up the variables that the class uses.
Run_shop	Boolean	String	The main code for this class where to make sure no collision issues occur doesn't let the player input anything until the code is ran 100 times which is done quickly this happens every time an icon is clicked so that it is only activated once. It checks whether the icons have been clicked and returns a string according to the icon clicked it then updates and draws on the screen.

Cost_up	None	None	Increases the class variable cost and updates the text in the class variable __writing according to the new cost.
Update_gold	integer	None	Updates the class variable gold with the parameter and updates the text in the shop accordingly.
Update_health	integer	None	Takes the parameter and updates the text in the shop for the health of the player.
Update_damage	integer	None	Takes the parameter and updates the text in the shop for the damage of the player.

Start Menu

Function Name	Parameters	Returns	Description
Initializer	Object	None	Creates all the writing using the Line class for the main menu as well as a separate list for the controls screen.
Startup_menu	Boolean	None	Updates and draws the text onto the screen as well as making the background light grey it then checks for if any collisions have happened if they have a variable is set to True depending on which one it was.
Controls_menu	Boolean	None	Makes the background of the screen light grey and then puts the text onto the screen and checks for whether the text go back has had a collision if so then sets the Boolean variables to say that the controls menu is false and the start menu is true.
Run_menu	Boolean	None	checks if the variable quit, controls and start is true if quit is true program ends if controls is true it runs the function Controls_menu and if start is true it runs the startup_menu function.
check	None	Boolean	If controls is false return start or return True this is used to check whether the start menu is in use.

Button

Function Name	Parameters	Returns	Description
Initializer	Integer, integer, string	None	Runs the initializer of the pygame.sprite.Sprite and then loads the image of the string and creates a rect/hitbox for that image and sets the x and y to those integer parameters.
collision	None	Boolean	detects whether the mouse is over the button if it is it returns True if not then it returns False.

Rest screen

Function Name	Parameters	Returns	Description

Initializer	Object	None	Assigns the parameter screen to screen and then creates the text for the screen as well as loads the images they are then added to a pygame group and all the variables needed for the class is assigned a variable count is set to 1.
Run_rest	Boolean, Object	string	If the count is not 0 it checks whether it is 20 if it is then count is 0 if not 1 is added if the count is 0 then it checks for whether the shop is true if it is then functions update_gold, update_health, update_damage and run_shop are ran If the string return from run_shop is quit then the shop is False otherwise the string is returned. Or the function button.colision for the rest button is ran if that return is True and click is True then over is set to True The function button.colision for the shop button is ran and if they is a click from the mouse then shop is set to True.
Return_over	None	Boolean	Returns the Boolean variable __over.
Reset_over	None	None	Boolean variable over is set to False.

Line

Function Name	Parameters	Returns	Description
Initializer	String, integer, integer, string, string, integer	None	It takes the parameters and renders the text with the colours and the text and size due to the parameters given with the font castellar and then the rect/hitbox is created and the position of the text is set according to the parameters.
Collide	None	None	Checks whether the mouse has collided/ is inside of the rect/hitbox of the text.
Give_text	None	String	Returns the string variable text.
Give_rect	None	Object	Returns the object text_rect.

Character.py

Node

Function Name	Parameters	Returns	Description
Initializer	List, list, int, list, list, list	None	Assigns everything a variable and then checks whether it is traversable and it also splits up the 1st parameter: coords into and x and y integer it then runs the self.__Heuristics function. Calculates the total distance and creates a variable to store it (movement_cost+heuristics) creates a variable with a list and a variable to check when it has been traversed.

__Heuristics	None	None	Does the Heuristics equation which is (x + y) + ((2**0.5) * x or y) depending on which one is smaller this is saved as one of the classes variables.
Create neighbours	None	None	Creates a list of the coords of the neighbours next to it
Check neighbours	None	list	This is the A* algorithm. It first checks if it is the target if not it looks through its neighbours. It then checks if it can be traversed if it can it checks that it gives a smaller distance to the target if it does it runs the function update node map appends its coords to the a list and then returns that list the list is the return of a check neighbours function of the neighbour with a smaller distance.
Set parent	list	None	Takes parameter and assigns it to a class variable (parent)
Update target	List	None	Takes parameter and assigns it to a class (target) variable it then runs the __Heuristics function and then recalculates the class variable "total distance"
Update movement cost	int	None	Takes parameter and assigns it to class variable "Movement cost"
Update node map	list	None	Takes parameter and assigns it to the class variable "Node map"
Give coords	None	list	return class variable "coords"
Give neighbours	None	list	return class variable "Neighbours"
Check traversable	None	Boolean	Return class variable "traversable"
Give distance	None	int	returns class variable "total distance"
Check travelled	None	Boolean	Returns class variable "traversed"
Travelled	None	None	Assigns True to class variable "traversed"

Connection

Function Name	Parameters	Returns	Description
Initialization	None	None	Creates class Variables: "coords”: None "player”: object of character class "map”: list It then runs the function cords receive and weapon hitbox receive
Create monsters	Int,list,int	None	loops number of times for the 1st parameter and appends the class variable "monsters" with an instance of the monster class for so many times it loops for.

Cords receive	None	None	sets the class variable "coords" to the return value of the object player`s return of give cord.
Weapon hitboxes receive	None	None	sets the class variable "weapon hitbox" to the objects player return of give weapon group
Cords give	None	None	loops through class variable list of monsters and each loop it gives that monster the coords via the function receive coords from the monster class
Map get	list	None	Sets parameter to class variable "map"
Map give	None	None	loops through every monster in its list and gives them all a map (the class variable “map”) via the receive map function from the monster class.
Damage bits	None	none	gets 2 variables form the player class function give damage bits. It then loops through monster list and runs function from monster class set damage bits and if they died it removes them from the game.

Character class

Function Name	Parameters	Returns	Description
Character Class Initializer	None	None	This sets up the character class`s Variables and loads the image that is a sprite sheet it also initialises the weapon class as a variable
Wall setter	Object	None	Sets parameter to class variable "walls"
Give coord	None	list	returns a list of its y coord and x coord
Set coords	list	None	sets the parameter[0] to the y coord and parameter [1] to the x coord It also multiplies the y coord by 15 and x coord by 20 so that it is the coords of specific pixels.
Give weapon group	None	Object	Returns class variable "weapon group"
Give damage bits	None	Integer, Boolean	This returns the damage of the weapon from weapon.give damage and returns if the player is attacking from weapon.attacking.
collisions	None	None	checks if a collision is made with the player and the wall

			If it has it uses the class variable "last move" to determine which direction the players icon is moved so that it is no longer colliding with the wall.
movement	None	None	creates a variable of a Boolean list of all inputs it gets the position of the mouse sets class variable invulnerability to false checks if player is dashing if it is dash counter goes up 1 and if dash counter is 100 then dashing is false dash counter is set to 0 then it checks if player is holding left control if so then the player dashes 30 pixels towards the mouse after this it checks for every input of W A S or D and then moves the player in that respective direction (0.5 pixels if holding shift or 1 if not).
animation	None	None	A function that swaps between pictures on the sprite sheet to make it look like the character is walking
update	None	None	Where every function that runs every loop is ran from that are to do with the character class.
Give gold	integer	None	takes the variable and adds it the character class variable "gold".
Get image	integer	object	selects the image of the sprite sheet of the specific frame that is needed in that animation at that time and sets it as the variable of “image”
Give weapon	None	object	Returns the sprites of the hitboxes of the weapon.
Room check	None	None	Checks what room the player is in and then sets that as the variable "room in".

Weapon

Function Name	Parameters	Returns	Description
Weapon class initialiser	Integer, string, integer	None	Sets up all the variables of the weapon class and initialises to instances of the weapon sprite class.
input	None	None	Checks if the R key has been pressed and if "count" is 0 if it has it sets "attacking" to true and "count" is set to 100. While Attacking is False, and "count" is not 0. 1 is taken away from "count".

coords	list	None	Sets the coords of the 2 weapon sprite classes inside of weapon by running the .coords function of the weapon sprite class.
Give hit boxes	None	list	Returns the 2 instances of the class weapon sprites.
Give damage	None	integer	return the damage of the weapon.
attacking	None	Boolean	If "attacking" is true then the weapon sprite function attacking is run If count is 0 the hitboxes reset with the reset function of weapon sprite Then "attack" is returned.

Weapon Sprite

Function Name	Parameters	Returns	Description
Weapon Sprite Class initialiser	Integer, list, Boolean	None	It sets the weapon Sprites classes variables then it checks the Boolean parameter "side" if it is True or False it will take a different surface of the inverse of each other. This creates the two hitboxes
coords	list	None	It checks where the mouse is on the screen. It sets the list parameter "coords" to the coordinates of the rect. It then changes the coords of the Sprite to face to where the mouse is.
attacking	None	None	Depending on the "side" Boolean variable it changes the colour of the rect to red or yellow.
reset	None	None	Fills the image with the colour light grey.

Monster

Function Name	Parameters	Returns	Description
Monster Class initialiser	List, list, integer	None	Sets up the variables of the Monster Class.
update	None	None	Where all the function of the monster class are run from each loop. However, detection only occurs when Room check returns a True
Receive coords	list	None	it takes the "coords" list parameter and checks if coords are the same if they are a Boolean variable "Same

			Target" is set as True if not "target coords" are set as coords(the list variable) and "Same Target" is set to False and integer variable "count" is set to 0.
Receive map	2d list	None	Sets 2d list parameter "map" to the class variable "map".
Receive weapon hitbox	object	None	Assigns the object parameter "group" to the class variable "weapon hitboxes".
detection	None	None	It checks if it has the same target if not it runs the check neighbour function of the class node to detect where the player is. It then sets the target coords the first node/section of the map it needs to travel to.
Room check	None	Boolean	It checks whether it is in the same room as the player if it is it returns True if not it returns False.
Take damage	None	None	It checks if a collision has occurred between itself and the weapon sprites and if so then it checks whether it should be taking damage if so, it takes damage up to the damage of the characters damage.
Set damage bits	Boolean, integer	None	Assigns the parameters to "taking damage" a Boolean variable to check whether the monster should be taking damage and "player damage" and integer variable that stores how much damage the player does.
dying	None	Boolean	If health drops below 0 the monster has died so it returns True. This will then start the monster to be deleted from the game so that it has died.
Movement	None	None	Moves the monster toward the target coordinates by adding or subtracting its y and x values. And if the object it is walking into is traversable.

Generation.py

Wall

Function Name	Parameters	Returns	Description
Wall Class Initialiser	Integer, integer	None	Loads the image of wall.png into the self.image class variable and sets the coordinates according to the integer parameters.

Environmental

Function Name	Parameters	Returns	Description
Environmental Class Initializer	Boolean, integer, integer	None	Depending on the Boolean parameter it will load the image of either rock1 or rock2 .png it then sets the coords of these images according to the integer parameters.

Exit

Function Name	Parameters	Returns	Description
Initializer	Integer, integer, object	None	The image hatch is loaded and the rect/hitbox is created of the image and the coordinates are set using the parameters and the other variables the class uses are created.
Entering	Integer, integer, object	Boolean	Checks for a collision between the player character and the rect/hitbox of the image it then checks for if the enter clause is complete and if the enter key has been pressed if that is all True then it returns True if not then it returns false.
Update_exit_clause	object	None	It checks whether the length of the parameter object players is 1 which means only the player is now alive If that is true the enter clause is set to being True.

Map

Function Name	Parameters	Returns	Description
Initializer	None	None	It creates a 2d list of the size 70 by 70 by determining a random number 1 or 0 it selects which side the second room will come from the first room is created using the room class room 2 is then created with the starting coordinates either below or next to room 1 room 3 is then created with the opposite direction to room 2 than room 2 was to room 1. The coords of each room are then assigned to variables.
__connections	None	None	A hallway is created between 2 rooms in the direction that they are next to each other by this is done by changing the 1's and 0's to make the shape of a hallway onto the map that is one block wide which is enough room for the player. It also checks for if there are any indents and then changes the where the hallway is put.
__exit	None	None	It chooses a room at random it then runs the function create exit on the room that was chosen.

Place_enemy	Object	None	It loops through all of the rooms if the player doesn't spawn in that room an enemy is placed by the place_enemy function.
Check_room	Object	None	Checks if the player is in any of the rooms by looking at the coordinates of the player and looking at the coordinates of which the rooms span This is then put into the function player_active_room.
Give_map	None	2d list	returns the list map.
Return_player_placement	None	integer	returns the integer player placement.

Room

Function Name	Parameters	Returns	Description
Initializer	Integer, integer, list, 2dlist, Boolean, integer	None	Creates a random height and width this is then put onto the map with the positions having the starting coords added onto it Through a random number being generated a corner is chosen for each corner and then the original corner is removed and a new one is put onto the 2d list it then checks if there are space for indents and if there is space if they is it does an indent to how much space it has after this calculates the starting coords of the next room.
Place_items	Integer	None	Loops amount of times as the parameter amount and randomly choses the coordinates of the object to be placed it then either places a 2 or a 3 on the 2d list.
Create_exit	None	None	Loops until the coords chosen to place the exit is an empty slot and if it is then on the 2d list it is represented as a 4.
Place_enemy	Object	None	places a monster in the middle of the room using Connection: create monsters.
Give_map_back	None	2d list	Returns the 2d list map.
Give_dimensions_back	None	list	returns the list of the dimensions of the room.
Return_room_coords	None	list	Returns a list of the starting width and height and the width and height of the room.
Give_B_working_room	None	integer	Returns the amount of space the room has underneath it.
Give_T_working_room	None	integer	Returns the amount of space the room has above it.
Give_width_height	None	list	returns a list of the width and height.
Give_zero_width_height	None	list	returns a list of the width and height.
Give_side	None	Boolean	Returns the Boolean value side this is used to determine what side the next room will go from.

Player_placement	None	list	Returns a list of the coords of the middle of the room to place the player there.

Level map

Function Name	Parameters	Returns	Description
Initializer	Integer, object	None	creates all the variables the class uses then it loops for 4*difficulty, and it starts by creating one node on each layer but it increases by 1 each time until the last node.
Give_map	None	2d list	returns the 2d list level map.
Send_level	None	None	it loops through every level node in level map and run the function level_at_receive where the level at variable is the parameter.
Receive_new_level	integer	None	assigns parameter to level at variable.

Level Node

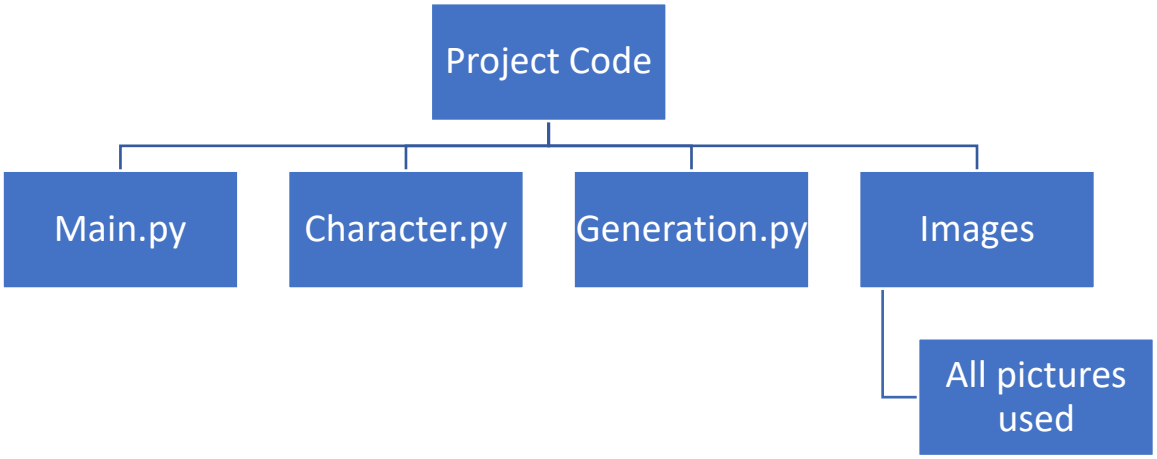
Function Name	Parameters	Returns	Description
Initializer	List, Boolean, integer, object, integer	None	checks if it is the last node if it is the its type is end and its colour is black with its width and height being 100 by 40 pixels if not then it is randomly assigned as either a dungeon, chest or rest node with the size always being 35 / total level halved squares if its a dungeon then its red, a chest then blue and a rest then yellow it then creates its rect/hitbox and assigns the position coordinates and creates other variables needed for the class.
__collisions	None	Boolean	runs the __level_check function and if it returns True then it checks whether a collision is made between the mouse and the node if so its checks if a mouse click has happened if it has the __update_level method is ran and it returns True.
__update_level	None	None	new_level is assigned as the level + 1 and then runs the function recieve_new_level to the father object with the new_level as the parameter.
Level_at_recieve	integer	None	assigns the parameter to current_level.
__Level_check	None	Boolean	Checks if the level at is this nodes level if so then Return True if not Return False.
run	None	string	Checks whether a collision has been made from the function __collision and

			if it returns True then it returns the string of its type.

File Structure

For my programming files I have split it up into Character.py , Generation.py and Main.py. Inside the code for Character.py there is the code for the character class as well as the enemy class and the other classes needed for the pathfinding and for the characters attacks. In Generation.py it has everything to do with the generation so the Level class as well with its associating classes. It also has the level select class where the player selects their level. The Main.py file it is where the program runs from, so it has all the menu`s and is where everything is called from. There is also a folder called images which is where the all the images the game uses are stored.

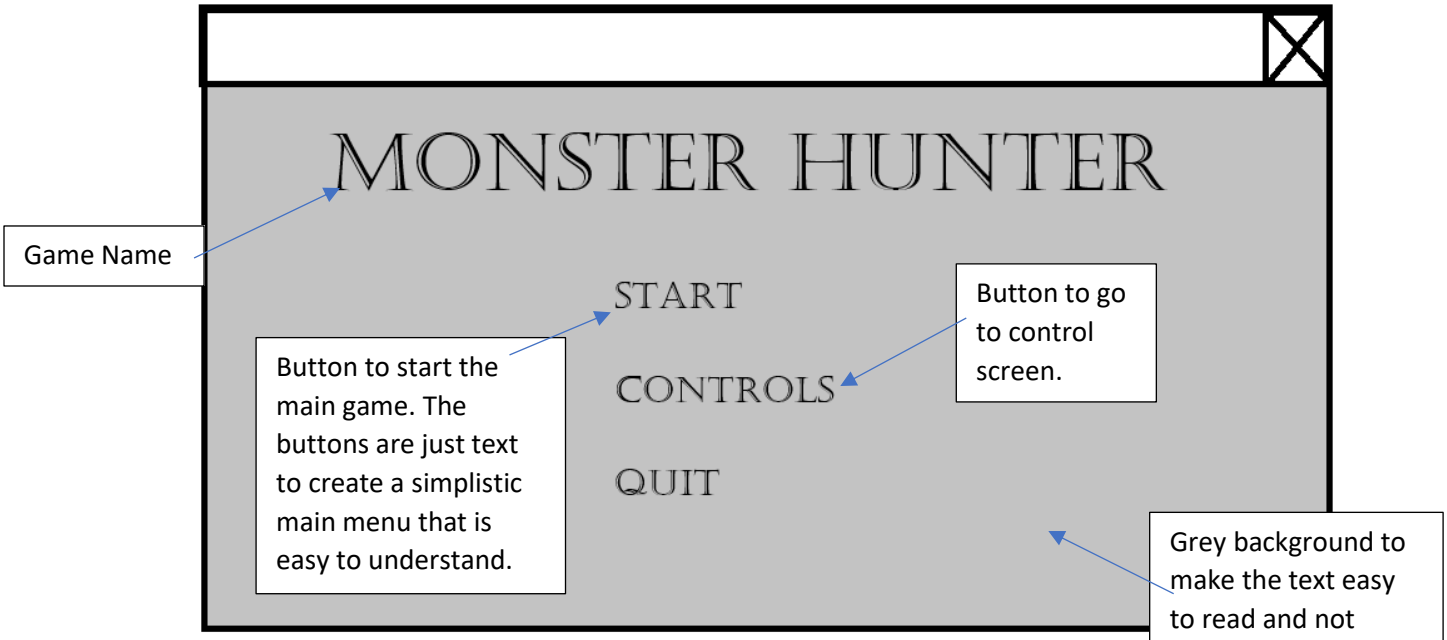
Here is a diagram to show the file structure:



HCI (user interface)

Start Menu

The font is castellar as it is quite a gothic font which fits the theme of my game, and it is the style that I want the game to be.

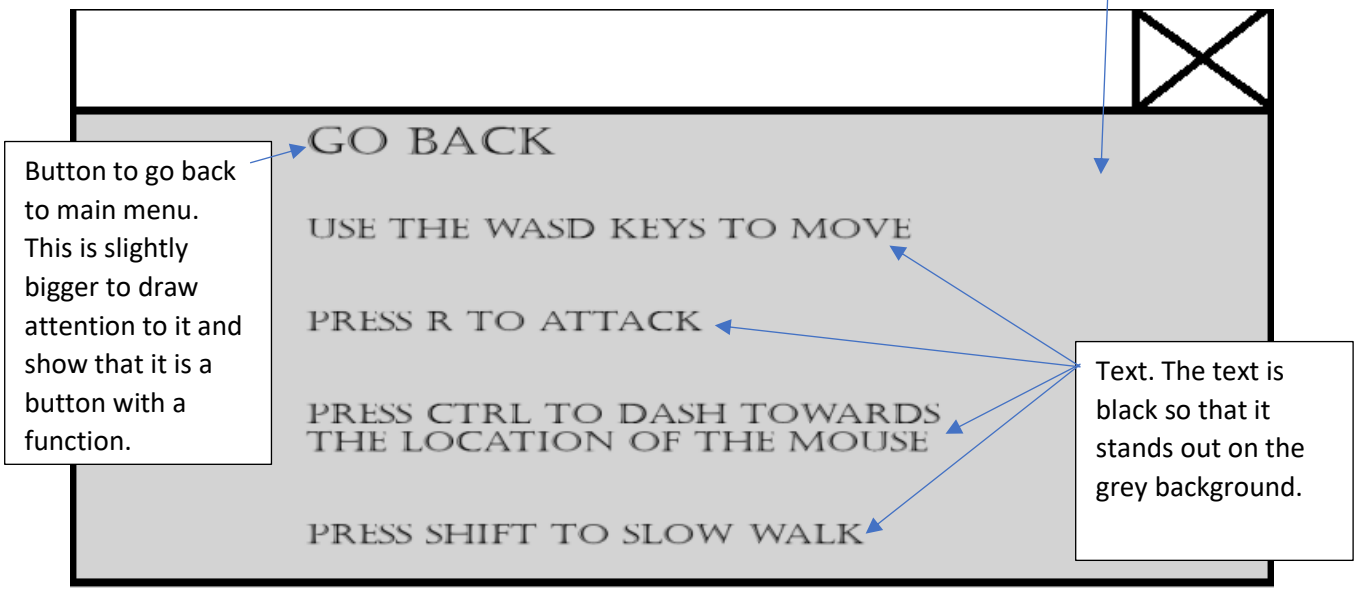


Game Name

Button to start the main game. The buttons are just text to create a simplistic main menu that is easy to understand.

Button to go to control screen.

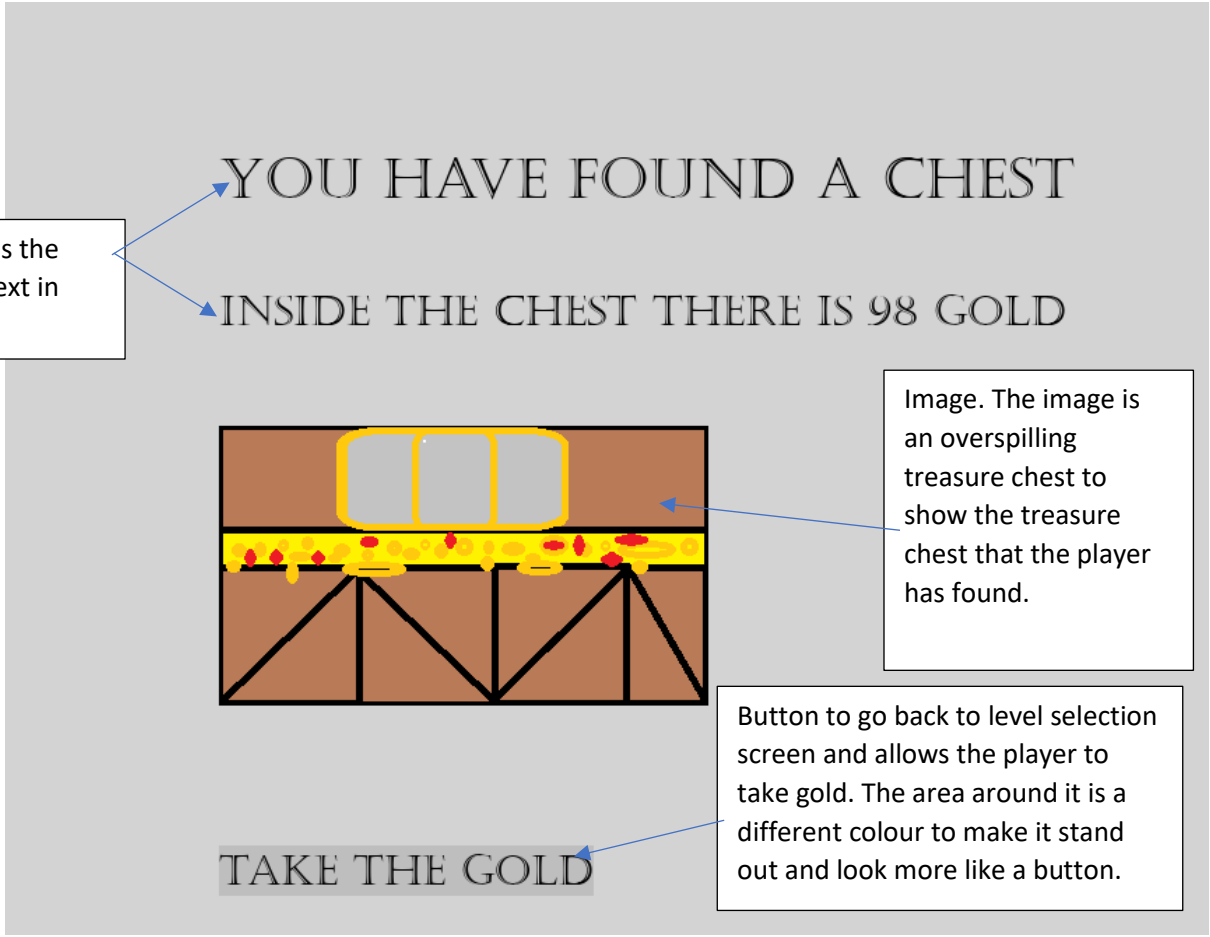
Grey background to make the text easy to read and not distract from the rest of the game.



Button to go back to main menu. This is slightly bigger to draw attention to it and show that it is a button with a function.

Text. The text is black so that it stands out on the grey background.

Chest Menu

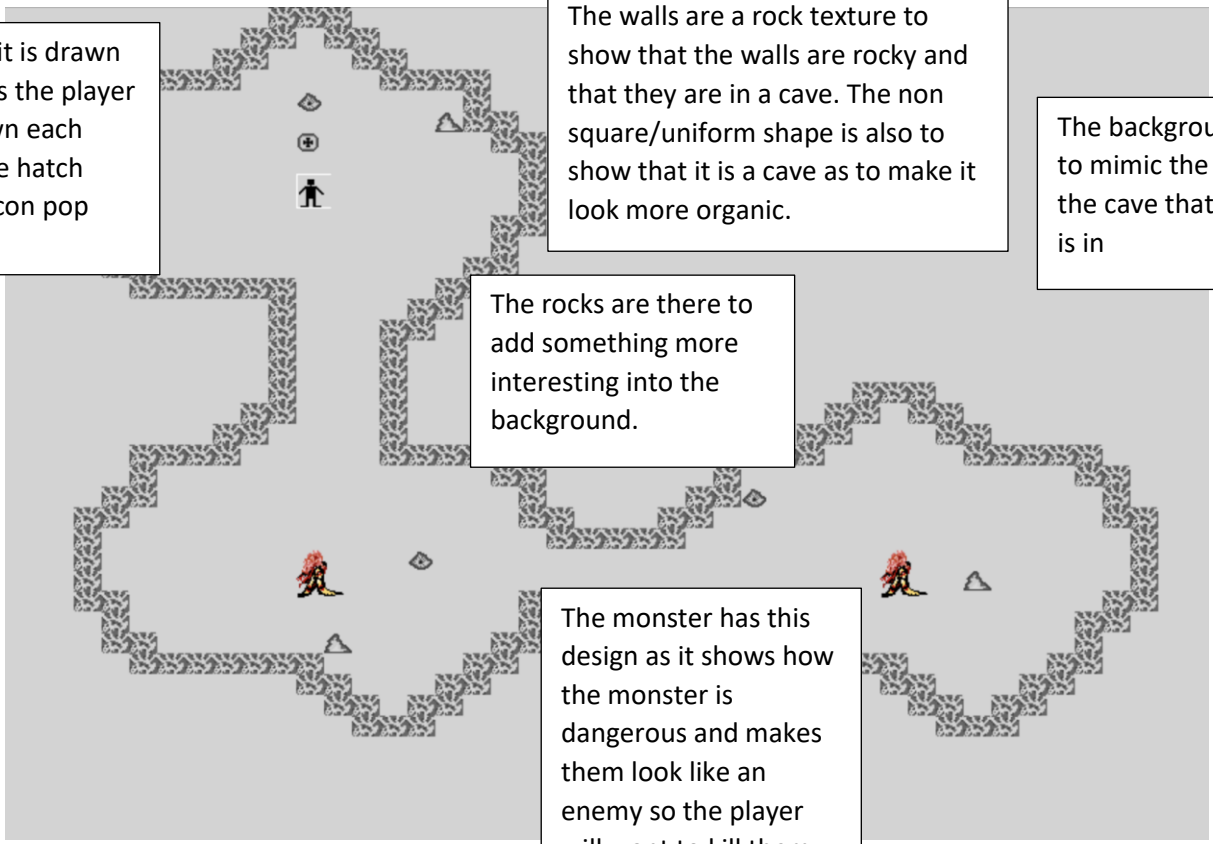


Text matches the rest of the text in the game.

Image. The image is an overspilling treasure chest to show the treasure chest that the player has found.

Button to go back to level selection screen and allows the player to take gold. The area around it is a different colour to make it stand out and look more like a button.

Main Game



The level exit is drawn as a hatch as the player is going down each level and the hatch makes the icon pop out.

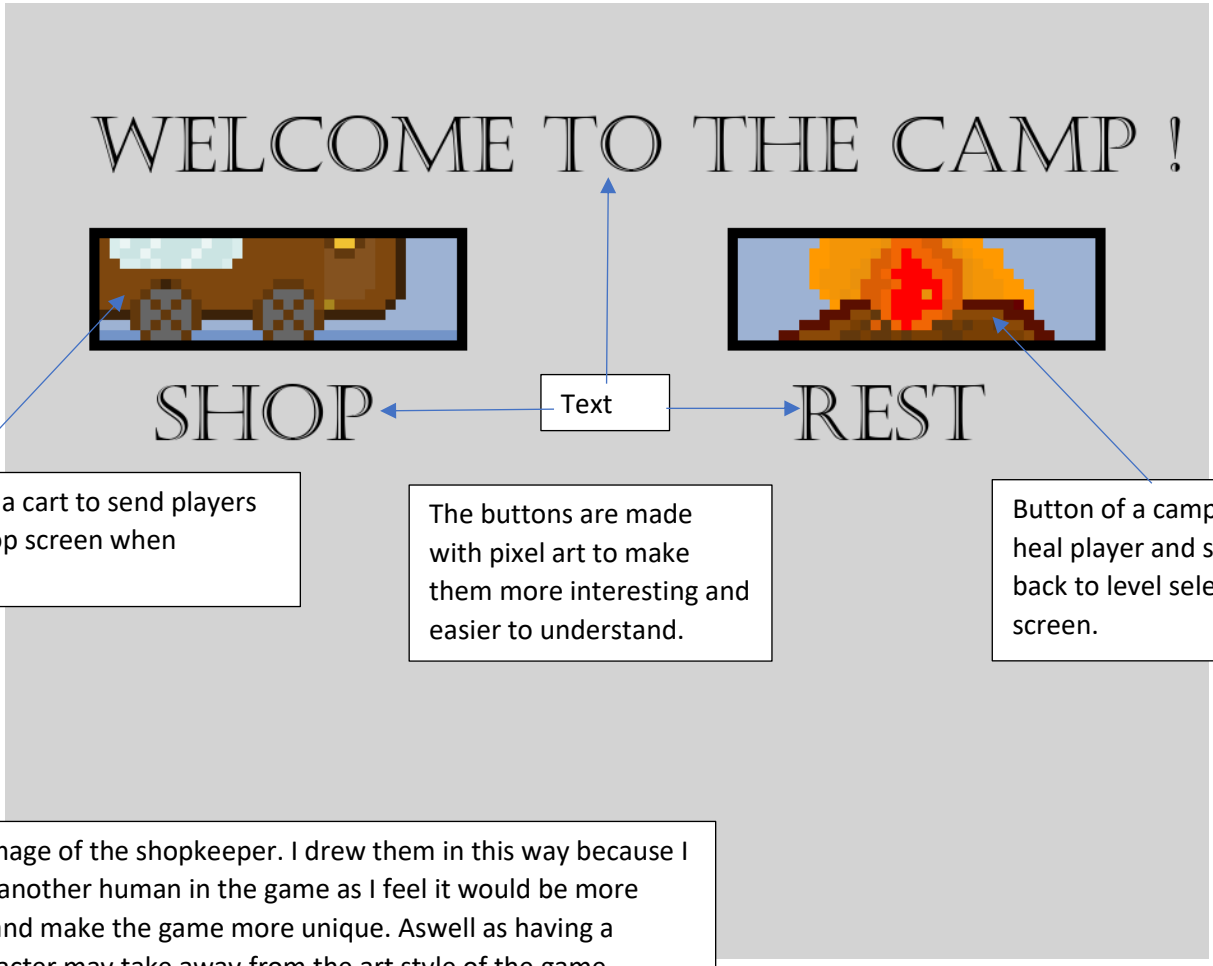
The walls are a rock texture to show that the walls are rocky and that they are in a cave. The non square/uniform shape is also to show that it is a cave as to make it look more organic.

The background is grey to mimic the floor of the cave that the player is in

The rocks are there to add something more interesting into the background.

The monster has this design as it shows how the monster is dangerous and makes them look like an enemy so the player will want to kill them.

Rest Screen

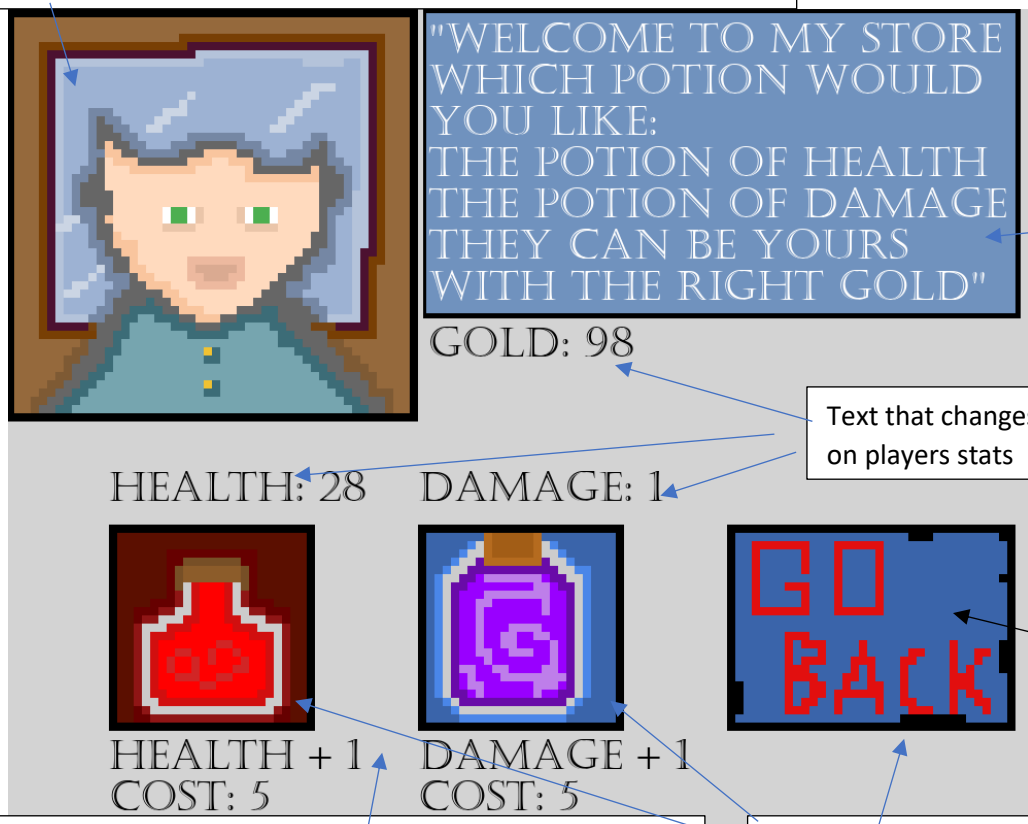


Button of a cart to send players to the shop screen when pressed.

The buttons are made with pixel art to make them more interesting and easier to understand.

Button of a campfire to heal player and send them back to level selection screen.

This is the image of the shopkeeper. I drew them in this way because I didn't want another human in the game as I feel it would be more interesting and make the game more unique. Aswell as having a human character may take away from the art style of the game.



Text, it is white so that it stands out on the blue background better than black would. The background is blue to make it different from the other text to make it stand out as speech and not a button.

Text that changes depending on players stats

The Go Back button is designed to look like a handmade sign of the shopkeeper. The use of the red and blue is to make is stand out so that the player does not miss it.

where number changes depending on the cost. The cost will increase after each purchase. The health potion is red because that's the colour in most RPG settings/games. The damage potion has a different shape and colour so that it can be easily distinguished between the health potion by players.

Buttons, they have black outline to make them stand out.

Testing

Test strategy

Character Class

This class will be tested mostly by checking if inputs work. This will be for movement and attacking which are the main aspects of this class.

The “WASD” keys are the keys used for movement so they should work in their respective movements the move the player across the screen, The dash which uses the “Ctrl” key jumps the player towards where the mouse curser is and then it has a cooldown for about 10 seconds in which the player can`t dash. The player can also attack using the “R” key which make the player attack when pressed and it will deal damage every couple seconds so that the enemy isn`t instantly killed.

There are also the aspects of this class that work behind the scenes in which I will show by just playing the game or trying to break it and or cause bugs in the game.

The player class can detect a collision with the wall and bounce the player off it, this means that the player cannot exit the dungeon. The player can also take damage which I will show by allowing the player to take damage from an enemy and then I will continue this until the player dies to show what that looks like.

Level generation

The level generation will be tested when heading into the level and will mostly be observations, alongside the other aspects of the game that must be tested.

To show that the generation is random and different each time I will load into different levels multiple times. This will also show how the placement of the objects in the levels is different. This will show that the enemies will always be placed into the middle of the rooms so that they cannot be put out of bounds. The rooms are always connected which can be shown using this method. One exit will spawn into a room in each level so you will be able to observe that there is only one and that it is inside one of the rooms.

For each of the rooms I will test that the corners are randomly chosen between the pre-sets that are in the game and that the width and height are chosen randomly within the range that they can pick from. The indents that push the wall out to make the room more complex will also be tested that they push out the most that they can and if there isn’t room that they don`t push out past the screen. I will also test that the rooms never intersect one another. This will use the same method of keep reloading a level to see all the different generations that are produced I will do this by reloading the level multiple times in the video and having screen shots of the different generations so that the generation can be thoroughly tested for bugs.

For the more mechanical aspects of the code with level generation I will run a debug to show that the list that is used as the base of the level is used and given to the other classes need it. This can also be used to show the aspects of the generation that are random.

Monster Class

Some aspects of this class will be tested using observation like prior.

That the enemy will be able to track the player using A* algorithm. For the A* algorithm I will also compare it to another A* algorithm to compare them to see if they work in the same way. That the enemy cannot clip through the wall will also be down from observations as there is no way to control where enemy goes without the player being able to get there and in the same room as the enemy this is because the enemy will only track the player if they are in the same room.

The other aspects can be shown with interference from the player.

For example, showing that the enemy can take damage will be tested when the player is testing if they can attack and when the enemy should take damage.

To go more in depth about the testing for the A* algorithm I will need to make sure that the nodes have an accurate movement cost and that the pathfinding choses the smallest node instantly and doesn`t stay to look for the most optimal path. I will run a debug to show the step by step.

Menu`s

There is different menu`s in the game the main menu, rest spot, shop, and the tressure room.

For the main menu I will need to check if all the buttons work which can be done at the start by clicking on them and showing that they can work.

For the rest spots I will need to check if the player can load into them and then if the player can either rest to leave the rest spot and continue with the game or if the player can enter the shop using their respective buttons.

The shop will be tested similarly to the rest spot in which I need to check if the shop exit button sends the player back to the rest spot. I will also need to check whether the player is able to purchase upgrades at the shop and if they affect the character and reduce their money.

The treasure room will be a menu in which the player receives a random item or some gold so I will need to check if the item received goes into the players inventory/ upgrades their stats and that when a button is pressed the game continues to the next section and that it returns to the level selection map.

Test Plan

I am going to test my project to make sure that players can`t cheat the system and use exploits to get past the levels and remove the difficulty of the game.

The testing video link: https://youtu.be/WK1QbwWC43k

Test No.	Purpose of test	Test Data	Expected Outcome	Actual outcome	Time Stamp
1.	To show that the player moves according to the inputs.	Keyboard inputs	The player will be able to move sneak and dash	The player was able to move as inputted	1:25
2.	To show that the player can attack.	Keyboard input R	The player will be able to do the attack input that is accepted and then the attack will be	When R is held down there are boxes to show the players hitbox	1:44

			seen by the user/player		
3.	Show that the players attacks deal damage	Keyboard input R	The player will attack an enemy and the enemy will die.	The enemy died shortly after being attacked	1:44
4.	Show how the player can die		Show how that if the player takes so much damage that the game will end.	After the enemy hits the player for a short amount of time the player dies, and the games quits	2:22
5.	Show that the dash has a cooldown	CTRL key	The player will be able to dash once and then not again until after a couple seconds	The player can dash towards the mouse and then is not able to do so for a short amount of time after	3:00
6.	Show that a dungeon is created		When loading into a level a dungeon would have generated	When the dungeon is selected it loads the level perfectly	1:25
7.	That a room does not generate of screen		When loading into the game the all rooms should be one screen	None of the rooms has generated off screen	1:25 2:21 2:59
8.	The rooms should not intersect		When generation is being done the rooms should generate a certain distance apart so that they do not intersect	None of the rooms intersected	1:25 2:21 2:59
9.	The indents being generated		When the indents are being generated it needs to make sure that the indents only go out to a length of 3 at max and that they never intersect with other rooms	The indents were always generated	1:25 2:21 2:59
10.	That the rooms are always connected		That there is always a gap to let the player traverse room to room that is enclosed so that	The rooms were always connected except for one instance in	Fully connected: • 1:25 • 2:59 Not fully: • 2:21

			the player cannot go out of bounds	which there was a singular gap	
11.	Show that the exit can be generated		It should only be generated once, and it should be generated in one of the rooms	The exit was generated every time	1:25 2:21 2:59
12.	Show that the exit works properly	Enter key	When all enemies have been defeated the exit will be able to be used to get to the next level.	After all of the enemies have been defeated the player can use the exit to get to the next level	1:46
13.	That the environmental pieces of the dungeon can be generated		There should be a few environmental pieces that are generated in each room.	Environmental pieces were generated in every level	1:25 2:21 2:59
14.	Show that the enemy only tracks the player when you are in its room.		When you leave a room, an enemy is it should stop tracking you and it should not instantly try to get to you when you load into the level	When the player is not in the same room the monster does not track the player or move but when the player enter the room the monster starts to track the player	2:22
15.	Show that the tracking the enemy does is A* algorithm		The code should follow the process of A* algorithm	The enemies track the player	2:24
16.	Show that the enemy can damage the player		The health of the player should go down	The players health goes down when the player is attacked	2:24
17.	Show that the enemy can die		The enemy after taking so much damage should delete itself and die.	After the enemy is attacked for a while, it dies and is removed from the game	1:42
18.	Show that the start menu button works	Mouse input	When you click “start game” the level selection should start	When clicked the main menu went away and the level selection screen is shown	0:24

19.	Show that the controls button works	Mouse Input	When you click onto the text “controls” it should take you to the control screen	When clicked it took me to the control screen	0:08
20.	Show that the quit button works	Mouse input	When you click the text “quit” the game should quit	When clicked the game quit	0:20
21.	Show that the level selection button works	Mouse input	When you click on the icon it should take to you to either a treasure room, rest point or the main game. The reason to test another button is because in the code they do work differently.	When clicked on a blue node it goes to the chest screen	0:37
22.	Show that the icons displayed is what the button takes you to in the level selection	Mouse input	When you click on the icon it should take you to the correct function	When clicked on a blue node it goes to the chest screen	Blue Node: 0:37
23.	Show that if you click a button, you do not yet have a path to nothing will happen		When you click the button, nothing will happen	When trying to click a node you are not at the level at yet you cannot enter them	0:28
24.	Show that the rest function can load		Show that when the rest spot is clicked that you can see it load onto the screen	The rest screen loads after the button have been pressed	0:46
25.	Show that the shop button works	Mouse input	When it is clicked the shop function should load	When the icon is clicked, we are sent to the shop screen	0:49
26.	Show that the player can purchase a health orb or damage orb.	Mouse input	When you purchase either your health or damage should go up.	When the player clicks onto the health orb their health goes up	0:53
27.	Show that the players money goes down when they purchase something	Mouse input	When a purchase is made the players, money should go down by the amount that the item is	When the player purchases a health orb their gold goes down	0:53

28.	Show that the exit button of the shop works	Mouse input	When the player clicks on the exit button the player will be sent back to the rest spot	When the go back button is pressed the player is sent back to the rest menu	1:16
29.	The treasure room function should be able to load	None	When selected in the level map the treasure room function should load	The treasure room can be loaded in	0:38
30.	Show when a button is pressed on the treasure room	Mouse input	The game should move on back to the level selection screen	When the take gold button is pressed we are sent back to the level selection screen	0:43

The code

Main.py

```
1  from  Generation import Map, Wall, Environmental, Exit, Level_map, Level_Node
2
3  #from Character import Character_class
4  #from Character import Monster
5  from Character import Node
6  from Character import C
7  import pygame
8  import random
9
10 class Line(pygame.sprite.Sprite):
11     def __init__(self, text, x, y, colour,background, size):
12         '''
13         It takes the parameters and renders the text with the colours and the text and size due to the parameters given with the font castellar and then the rect/hitbox is created and the position of the text is set according to the parameters
14         :param text:
15         :param x:
```

```
16      :param y:
17      :param colour:
18      :param background:
19      :param size:
20      '''
21      super().__init__()
22
23      self.__font = pygame.font.SysFont('castellar', size)
24      self.__text = self.__font.render(text, True, colour, background)
25      self.__text_rect = self.__text.get_rect()
26      self.__text_rect.x = x
27      self.__text_rect.y = y
28
29  def collide(self):
30      '''
31      Checks whether the mouse has colldied/ is inside of the rect/hitbox of the text
32      :return:
33      '''
34      mouse = pygame.mouse.get_pos()
35      if self.__text_rect.collidepoint(mouse[0],mouse[1]):
36          return True
37      else:
38          return False
39  def give_text(self):
40      '''
41      Returns the string variable text
42      :return:
43      '''
44      return self.__text
45  def give_rect(self):
46      '''
```

```
47        returns the rect of the text
48        :return:
49        '''
50        return self.__text_rect
51
52  class Rest_Screen:
53      def __init__(self,screen):
54          '''
55          Assigns the parameter screen to screen and then creates the text for the screen aswell as
loads the images they are then added to a pygame group
56          and all the variables needed for the class is assigned a variable count is set to 1
57          :param screen:
58          '''
59          self.__screen = screen
60          #pygame stuff
61          self.__writing = []
62          self.__writing.append(Line('Welcome to the camp !', 100,100,'black','light grey', 60))
63          self.__writing.append(Line('Rest',650,310,'black','light grey',60))
64          self.__writing.append(Line('Shop', 150, 310, 'black', 'light grey', 60))
65          self.__group = pygame.sprite.Group()
66          self.__rest_button= button(600,200, 'images/rest point button.png')
67          self.__shop_button = button(100,200, 'images/shop_button.png')
68          self.__group.add(self.__rest_button)
69          self.__group.add(self.__shop_button)
70          self.__over = False
71          self.__shop = False
72          self.__shop_object = Shop(self.__screen)
73          self.__count = 1
74
75
76      def run_rest(self,click,C):
```

```
77          '''
78          If the count is not 0 it checks wether it is 20 if it is then count is 0 if not 1 is added
79          if the count is 0 then it checks for wether the shop is true if it is then functions update_gold,
update_health, update_damage and run_shop are ran
80          If the string return from run_shop is quit then the shop is False otherwise the string is
returned.
81          Or the function button.colision for the rest button  is ran if that return is True and click is
True then over is set to True
82          The function button.colision for the shop button is ran and if they is a click from the mouse
then shop is set to True
83          :param click:
84          :param C:
85          :return:
86          '''
87          if self.__count != 0:
88              if self.__count == 20:
89                  self.__count = 0
90              else:
91                  self.__count += 1
92          if self.__count == 0:
93              if self.__shop == True:
94                  self.__shop_object.update_gold(C.player.return_gold())
95                  self.__shop_object.update_health(C.player.return_health())
96                  self.__shop_object.update_damage(C.player.return_damage())
97                  string = self.__shop_object.run_shop(click)
98                  if string == 'quit':
99                      self.__shop = False
100                 else:
101                     return string
102
103             else:
104                 if self.__rest_button.collision():
```

```
105                 if click:
106                     self.__over = True
107             if self.__shop_button.collision():
108                 if click:
109                     self.__shop = True
110
111
112         self.__screen.fill('light grey')
113         for i in self.__writing:
114             self.__screen.blit(i.give_text(), i.give_rect())
115         self.__group.draw(self.__screen)
116         self.__group.update(self.__screen)
117         pygame.display.update()
118
119     def return_over(self):
120         '''
121         Returns the bollean variable __over
122         :return:
123         '''
124         return self.__over
125
126     def reset_over(self):
127         '''
128         Boolean variable over is set to False
129         :return:
130         '''
131         self.__over = False
132
133     def return_cost(self):
134         cost = self.__shop_object.return_cost()
135         return cost
```

```
136
137 class button(pygame.sprite.Sprite):
138     def __init__(self,x,y,image):
139         '''
140         Runs the initializer of the pygame.sprite.Sprite and then loads the image of the string and creates a rect/hitbox for that image and sets the x and y to those integer parameters
141         :param x:
142         :param y:
143         :param image:
144         '''
145         super().__init__()
146         self.image = pygame.image.load(image)
147
148         self.rect = self.image.get_rect()
149         self.rect.x = x
150         self.rect.y = y
151     def collision(self):
152         '''
153         detects wether the mouse is over the button if it is it returns True if not then it returns False
154         :return:
155         '''
156         mouse_pos = pygame.mouse.get_pos()
157         if self.rect.collidepoint(mouse_pos[0], mouse_pos[1]):
158             return True
159         else:
160             return False
161
162 class Shop:
163     def __init__(self,screen):
164         '''
165         It creates the text of the shop with the Line class
```

```
166         Loads the images that are used on this screen
167         sets up the variables that the class uses
168         :param screen:
169         '''
170         self.__screen = screen
171         self.__cost = 5
172         self.__gold = 0
173         self.__count = 1
174         self.__writing = []
175         self.__writing.append(Line('''WelCome to my StOre''',410,5,'white',(112,146,190),40))
176         self.__writing.append(Line('''WhIch potion wOUld''', 410, 45, 'white', (112,146,190), 40))
177         self.__writing.append(Line('''you like:''', 410, 85, 'white', (112,146,190), 40))
178         self.__writing.append(Line('''The Potion of HeAlth''', 410, 125, 'white', (112,146,190), 40))
179         self.__writing.append(Line('''The pOtion of Damage''', 410, 165, 'white', (112,146,190), 40))
180         self.__writing.append(Line('''THey can be yours''', 410, 205, 'white', (112,146,190), 40))
181         self.__writing.append(Line('''with the right gold''''', 410, 245, 'white', (112,146,190), 40))
182
183         self.__writing.append(Line('Health + 1',100, 700,'black','light grey',40))
184         self.__writing.append(Line('Cost: '+str(self.__cost), 100,740,'black','light grey',40))
185
186         self.__writing.append(Line('Damage + 1', 400, 700, 'black', 'light grey', 40))
187         self.__writing.append(Line('Cost: ' + str(self.__cost), 400, 740, 'black', 'light grey', 40))
188
189         self.__writing.append(Line('Gold: ',400,400,'black','light grey', 40))
190         self.__writing.append(Line('Health: ',100,355,'black','light grey', 40))
191         self.__writing.append(Line('Damage: ',400,355,'black','light grey', 40))
192         self.__group = pygame.sprite.Group()
193         self.__shop_keep = button(0,0,'images/shop_keep.png')
194         self.__text_background = button(405,0,'images/text background shopkeep.png')
195         self.__Health_bottle = button(100,500,'images/Health Potion.png')
196         self.__damage_potion = button(400, 500,'images/Damage Potion.png')
```

```
197         self.__go_back = button(700,500,'images/Go Back Sign.png')
198         self.__group.add(self.__shop_keep)
199         self.__group.add(self.__text_background)
200         self.__group.add(self.__Health_bottle)
201         self.__group.add(self.__damage_potion)
202         self.__group.add(self.__go_back)
203
204     def run_shop(self,click):
205         '''
206         The main code for this class where to make sure no colision issues occor doesnt let the player input anything until the code is ran 100 times which is done quickly
207         this happens every time an icon is clicked so that it is only activated once. It checks whether the icons have been clicked and returns a string according to the icon clicked
208         it then updates and draws on the screen.
209
210         :param click:
211         :return:
212         '''
213         if self.__count != 0:
214             if self.__count == 400:
215                 self.__count = 0
216             else:
217                 self.__count += 1
218
219         if self.__count == 0:
220
221             if self.__Health_bottle.collision():
222                 if click and self.__gold - self.__cost > 0 :
223                     self.__count += 1
224                     self.cost_up()
225                     return 'health'
226
```

```
227             if self.__damage_potion.collision():
228                 if click and self.__gold - self.__cost > 0:
229                     self.__count += 1
230                     self.cost_up()
231                     return 'damage'
232 
233             if self.__go_back.collision():
234                 if click:
235                     self.__count += 1
236                     return 'quit'
237 
238         self.__screen.fill('light grey')
239         self.__group.draw(self.__screen)
240         self.__group.update()
241         for i in self.__writing:
242             self.__screen.blit(i.give_text(),i.give_rect())
243         pygame.display.update()
244 
245     def cost_up(self):
246         '''
247         Increases the class variable cost and updates the text in the class variable __writing
    according to the new cost
248         :return:
249         '''
250         self.__cost += 5
251         self.__writing[8] = Line('Cost: '+str(self.__cost), 100,740,'black','light grey',40)
252         self.__writing[10] = Line('Cost: ' + str(self.__cost), 400, 740, 'black', 'light grey', 40)
253 
254     def update_gold(self,gold):
255         '''
256         Updates the class variable gold with the parameter and updates the text in the shop
    accordingly
```

```
257         :param gold:
258         :return:
259         '''
260         self.__gold = gold
261         self.__writing[11] = Line(('Gold: '+str(gold)),410,300,'black','light grey',40)
262
263     def update_health(self,health):
264         '''
265         Takes the parameter and updates the text in the shop for the health of the player
266         :param health:
267         :return:
268         '''
269         self.__writing[12] = Line(('Health: '+str(health)),100,440,'black','light grey', 40)
270
271     def update_damage(self,damage):
272         '''
273         Takes the parameter and updates the text in the shop for the damage of the player
274         :param damage:
275         :return:
276         '''
277         self.__writing[13] = Line(('Damage: '+str(damage)),400,440,'black','light grey', 40)
278
279     def return_cost(self):
280         return self.__cost
281
282 class Start_menu:
283     def __init__(self,screen):
284         '''
285         Creates all the writing using the Line class for the main menu aswell as a seperate list for
the controls screen
286         :param screen:
```

```
287         '''
288         self.__screen = screen
289         self.__start = True #NEEDS TO BE SET TO TRUE
290         self.__quit = False
291         self.__controls = False
292         #MAIN MENU
293         self.__writing_main = []
294         self.__writing_main.append(Line('Monster Hunter', 200, 150, 'black', 'light grey', 60))
295         self.__writing_main.append(Line('Start Game', 350, 300, 'black', 'light grey', 40))
296         self.__writing_main.append(Line('Controls', 350, 400, 'black', 'light grey', 40))
297         self.__writing_main.append(Line('Quit', 350, 500, 'black', 'light grey', 40))
298         #CONTROL MENU
299         self.__writing_control = []
300         self.__writing_control.append(Line('GO BACK', 200,100,'black','light grey', 40))
301         self.__writing_control.append(Line('Use the WASD keys to move', 200, 200, 'black', 'light
grey', 30))
302         self.__writing_control.append(Line('Press R to attack', 200, 300, 'black', 'light grey', 30))
303         self.__writing_control.append(Line('Press CTRL to dash towards', 200, 400, 'black', 'light
grey', 30))
304         self.__writing_control.append(Line('the location of the mouse', 200, 430, 'black', 'light
grey', 30))
305         self.__writing_control.append(Line('Press SHIFT to slow walk', 200, 530 , 'black', 'light grey',
30))
306
307
308     def startup_menu(self, mouse_click):
309         '''
310         Updates and draws the text onto the screen as well as making the background light grey it
then checks for if any colisions have happend if they has a variable is set to True depending on which
one it was
311         :param mouse_click:
312         :return:
313         '''
```

```
314         self.__screen.fill('light grey')
315
316         for i in self.__writing_main:
317             self.__screen.blit(i.give_text(), i.give_rect())
318
319         if self.__writing_main[1].collide() and mouse_click:
320             self.__start = False
321
322         elif self.__writing_main[2].collide() and mouse_click:
323             self.__controls = True
324             self.__start = False
325
326         elif self.__writing_main[3].collide() and mouse_click:
327             self.__quit = True
328
329         pygame.display.update()
330
331     def controls_menu(self, click):
332         '''
333         Makes the bnackground of the screen light grey and then puts the text onto the screen and
checks for wether the text go back has had a collision if so then sets the Boolean variables
334         to say that the controls menu is false and the start menu is true
335         :param click:
336         :return:
337         '''
338         self.__screen.fill('light grey')
339         for i in self.__writing_control:
340             self.__screen.blit(i.give_text(), i.give_rect())
341
342         if self.__writing_control[0].collide() and click:
343             self.__controls = False
```

```
344             self.__start = True
345
346         pygame.display.update()
347
348
349     def run_menu(self, click):
350         '''
351         checks if the variable quit,controls and start is true if quit is true program ends if controls is true it runs the function Controls_menu and if start is true it runs the startup_menu function
352         :param click:
353         :return:
354         '''
355         if self.__quit == True:
356             quit()
357         elif self.__controls == True:
358             self.controls_menu(click)
359
360         elif self.__start == True:
361             self.startup_menu(click)
362
363
364     def check(self):
365         '''
366         If controls is false return start or return True this is used to check wether the start menu is in use
367         :return:
368         '''
369         if self.__controls == False:
370             return self.__start
371         else:
372             return True
373
```

```
374     def reset_main_menu(self):
375         self.__start = True
376 
377 class Level_map_creator:
378     def __init__(self,screen):
379         '''
380         Creates the variables the class uses and assigns the pygame screen to a variable
381         :param screen:
382         '''
383         self.__level = 1
384         self.__screen = screen
385         self.__level_buttons = None
386         self.__choice = None
387 
388     def create(self):
389         '''
390         Resets the buttons and deletes the previous ones and then creates a new level map class and adds the buttons of that class to pygame group level buttons. Then the difficulty variable is increased by 1.
391         :return:
392         '''
393         self.__level_buttons = pygame.sprite.Group()
394         self.__map = Level_map(self.__level, self.__screen)
395         map = self.__map.give_map()
396         for i in map:
397             for x in i:
398                 self.__level_buttons.add(x)
399         self.__level += 1
400 
401     def reset_choice(self):
402         '''
403         assings the class variable of chioce to None/nothing
```

```
404         :return:
405         '''
406         self.__choice = None
407
408     def running(self):
409         '''
410         Loops through every level node in the level map and checks if they return a choice if so that choice is assigned to the class variable choice
411         It then runs the function send_level and makes teh screen white and puts the buttons onto the screen and thgebn updates the display
412         :return:
413         '''
414         for i in self.__map.give_map():
415             for x in i:
416                 choice = x.run()
417                 if  choice != None:
418                     self.__choice =  choice
419
420         self.__map.send_level()
421         self.__screen.fill('white')
422         self.__level_buttons.draw(self.__screen)
423         self.__level_buttons.update()
424         pygame.display.update()
425
426     def return_choice(self):
427         '''
428         Returns the class variable choice
429         :return:
430         '''
431         return self.__choice
432
433 class Chest_screen:
```

```
434     def __init__(self,screen):
435         '''
436         Creates the text and image needed for the chest screen it also creates all the variables the class uses
437         :param screen:
438         '''
439         self.__screen = screen
440         self.__contents = None
441         self.__string_contents = ''
442         self.__writing = []
443         self.__writing.append(Line('You have Found a Chest', 200, 200, 'black', 'light grey', 40))
444         self.__writing.append(Line(('Inside the chest there is'+self.__string_contents), 200,300, 'black', 'light grey', 30))
445         self.__writing.append(Line('Take the Gold',200,700, 'black', 'grey', 30))
446         self.__group = pygame.sprite.Group()
447         self.__image = Chest_image()
448         self.__group.add(self.__image)
449         self.__count = 0
450         self.__spin = True
451
452
453     def chest(self,click):
454         '''
455         This is the main code for the chest screen it creates a new value for the chest 200 times then stops this creates the illusion that the amount in the text is spining to select an amount
456         it then checks for if a clolision and the mouse has clicked and then returns the value (amount of gold the chest has) and restocks the chest.
457         :param click:
458         :return:
459         '''
460         self.__screen.fill('light grey')
461         if self.__spin == True:
```

```
462             self.__restock()
463             self.__count += 1
464         if self.__count == 200:
465             self.__spin = False
466
467         if self.__writing[2].collide() and click:
468             self.__count = 0
469             self.__spin = True
470             value = self.__contents
471             self.__restock()
472             return value
473
474         for i in self.__writing:
475             self.__screen.blit(i.give_text(), i.give_rect())
476         self.__group.draw(self.__screen)
477         self.__group.update()
478
479         pygame.display.update()
480
481     def __restock(self):
482         '''
483         Creates a random number (0,100) this new value is now the new contents it then modifies
the text according to the new amount of gold the chest has
484         :return:
485         '''
486         num = random.randint(0, 100)
487         self.__contents = num
488         self.__string_contents = " "+str(num)+' Gold'
489         self.__writing[1] = Line(('Inside the chest there is'+self.__string_contents), 200,300, 'black',
'light grey', 30)
490
491  class Chest_image(pygame.sprite.Sprite):
```

```
492     def __init__(self):
493         '''
494         Loads the image chest and creates the rect/hitbox for the image and sets the coordinates of where the image is placed
495         '''
496         super().__init__()
497         self.image = pygame.image.load('images/chest.png')
498         self.rect = self.image.get_rect()
499         self.rect.x = 200
500         self.rect.y = 400
501
502
503 class Main_game():
504     def __init__(self):
505         '''
506         This function sets up everything that stays constant in the game and creates all the variables this class uses
507         '''
508         self.__Map = None
509         self.__map_list = []
510         self.__node_map = []
511         self.__exit_map = None
512         self.__C = C()
513         self.__player_placement = []
514         self.__screen = pygame.display.set_mode((1000,800))
515         self.__level_selection_map = Level_map_creator(self.__screen)
516
517         self.__walls = pygame.sprite.Group()
518         self.__objects = pygame.sprite.Group()
519         self.__players = pygame.sprite.Group()
520         self.__players.add(self.__C.player)
521         self.__weapon_hitboxes = pygame.sprite.Group()
```

```
522         for i in self.__C.player.give_weapon():
523             self.__weapon_hitboxes.add(i)
524 
525         self.__start_menu = Start_menu(self.__screen)
526         self.__rest_screen = Rest_Screen(self.__screen)
527         self.__level_selection = True
528         self.__Main = False
529         self.__rest = False
530         self.__chest = False
531         self.__chest_screen = Chest_screen(self.__screen)
532         self.__run = True
533         self.__restart = False
534 
535     def bootup(self):
536         '''
537         This resets the game this includes the level selection screen as well as the main game levels
538         :return:
539         '''
540         self.reset_main_game()
541         self.__level_selection_map.create()
542 
543     def reset_main_game(self):
544         self.__Map = Map()
545         self.__player_placement = self.__Map.return_player_placement()
546         self.__C.player.set_coords(self.__player_placement)
547         self.__Map.place_enemy(self.__C)
548         self.__walls = pygame.sprite.Group()
549         self.__objects = pygame.sprite.Group()
550         self.__map_list = self.__Map.give_map()
551         for i in range(len(self.__map_list)):
552             for x in range(len(self.__map_list[0])):
```

```
553             if self.__map_list[i][x] == 1:
554                 # slot = self.__map_list[i][x]
555                 x_cord = x * 20
556                 y_cord = i * 15
557                 wall = Wall(x_cord, y_cord)
558                 self.__walls.add(wall)
559 
560             elif self.__map_list[i][x] == 2:
561                 # slot = self.__map_list[i][x]
562                 x_cord = x * 20
563                 y_cord = i * 15
564                 object = Environmental(False, x_cord, y_cord)
565                 self.__objects.add(object)
566 
567             elif self.__map_list[i][x] == 3:
568                 # slot = self.__map_list[i][x]
569                 x_cord = x * 20
570                 y_cord = i * 15
571                 object = Environmental(True, x_cord, y_cord)
572                 self.__objects.add(object)
573 
574             elif self.__map_list[i][x] == 4:
575                 # slot = self.__map_list[i][x]
576                 x_cord = x * 20
577                 y_cord = i * 15
578                 self.__exit_map = Exit(x_cord, y_cord, self.__players)
579                 self.__objects.add(self.__exit_map)
580 
581         for i in range(len(self.__map_list)):
582             self.__node_map.append([])
583             for x in range(len(self.__map_list[0])):
```

```
584             self.__node_map[i].append(Node([i, x], [0, 0], 1, self.__map_list, self.__node_map,
None))
585             self.__node_map[i][x].create_neighbours()
586
587         for monster in self.__C.monsters:
588             self.__players.add(monster)
589             monster.recieve_map(self.__node_map)
590             monster.recieve_weapon_hitbox(self.__weapon_hitboxes)
591             monster.wallsetter(self.__walls)
592
593         self.__C.player.wall_setter(self.__walls)
594
595     def run_game(self):
596         '''
597         This function is the main loop of the game and where everything runs from
598         :return:
599         '''
600         for event in pygame.event.get():
601             if event.type == pygame.QUIT:
602                 self.__run = False
603
604
605         M_inputs = pygame.mouse.get_pressed()
606         if M_inputs[0]:
607             click = True
608
609         else:
610             click = False
611         if self.__start_menu.check():
612
613             self.__start_menu.run_menu(click)
```

```
614
615         elif self.__rest == True:
616             print('runs')
617             string = self.__rest_screen.run_rest(click,self.__C)
618             if string == 'health':
619                 self.__C.player.give_health()
620                 self.__C.player.take_gold(self.__rest_screen.return_cost())
621             if string == 'damage':
622                 self.__C.player.give_damage()
623                 self.__C.player.take_gold(self.__rest_screen.return_cost())
624
625             if self.__rest_screen.return_over():
626                 self.__rest = False
627                 self.__level_selection = True
628                 self.__level_selection_map.reset_choice()
629                 self.__rest_screen.reset_over()
630
631         elif self.__chest == True:
632             contents = self.__chest_screen.chest(click)
633             if contents != None:
634                 self.__C.player.give_gold(contents)
635                 self.__chest = False
636                 self.__level_selection = True
637                 self.__level_selection_map.reset_choice()
638
639         elif self.__level_selection == True:
640             self.__level_selection_map.running()
641             self.__choice = self.__level_selection_map.return_choice()
642             if self.__choice != None:
643                 if self.__choice == 'dungeon':
644                     print('going to a dungeon')
```

```
645                 self.__level_selection = False
646                 self.__chest = False
647                 self.__rest = False
648                 self.__Main = True
649
650             elif self.__choice == 'chest':
651                 print('going to a chest')
652                 self.__level_selection = False
653                 self.__chest = True
654                 self.__rest = False
655                 self.__Main = False
656
657             elif self.__choice == 'rest':
658                 print('rest on')
659                 self.__level_selection = False
660                 self.__chest = False
661                 self.__rest = True
662                 self.__Main = False
663
664             elif self.__choice == 'end':
665                 print('new level')
666                 self.__level_selection = False
667                 self.__chest = False
668                 self.__rest = False
669                 self.__Main = False
670                 self.__restart = True
671                 print('restarting')
672
673         elif self.__restart == True:
674             self.__restart = False
675             self.__level_selection_map.reset_choice()
```

```
676             self.bootup()
677             self.__level_selection = True
678 
679         elif self.__Main == True:
680             self.__main()
681             self.__exit_map.update_exit_clause(self.__players)
682             if self.__exit_map.entering() == True:
683                 self.__Main = False
684                 self.__level_selection = True
685                 self.__level_selection_map.reset_choice()
686                 self.reset_main_game()
687 
688         else:
689             # screen.fill('white')
690             font = pygame.font.Font('Comic sans', 32)
691             text = font.render('loading', True, 'green', 'blue')
692             text_rect = text.get_rect()
693             text_rect.center(400, 300)
694             self.__screen.blit(text, text_rect)
695             pygame.display.update()
696 
697     def __main(self):
698         '''
699         This function is used when in a main level of the game and updates and draws all pygame
    groups needed.
700         It also updates the display and runs the functions C.C.coords_recieve() C.coords_give()
    C.weapon_hitbox_recieve() C.damage_bits().
701         :return:
702         '''
703         if self.__Main == True:
704             # collisions(C.player, objects)
705             self.__screen.fill('light grey')
```

```
706         self.__objects.draw(self.__screen)
707         self.__objects.update()
708         self.__walls.draw(self.__screen)
709         self.__walls.update()
710         self.__players.draw(self.__screen)
711         self.__weapon_hitboxes.draw(self.__screen)
712         self.__weapon_hitboxes.update()
713         self.__players.update()
714         pygame.display.update()
715
716         self.__C.coords_recieve()
717         self.__C.coords_give()
718         self.__C.weapon_hitbox_recieve()
719         self.__C.damage_bits(self.__players)
720         self.__Map.check_room(self.__C)
721
722         for i in self.__C.monsters:
723             if i.player_colision(self.__C.player.rect):
724                 damage = i.return_damage()
725                 if self.__C.player.take_damage(damage):
726                     quit()
727
728
729         print(self.__C.player.return_health())
730
731     def return_run(self):
732         '''
733         Returns the class variable __run
734         :return: Boolean
735         '''
736         return self.__run
```

```
737
738
739
740 pygame.font.init()
741 pygame.display.init()
742 main_game = Main_game()
743 main_game.bootup()
744
745 run = True
746 while run:
747     main_game.run_game()
748     run = main_game.return_run()
749
750 pygame.quit()
```

Character.py

```
import pygame
2   #from Main import Weapon_hitboxes
3   import time
4
5   class Node:
6       def __init__(self, coords, target, cost, map, Node_map, parent):
7           '''
8           Assigns everything a variable and then checks whether it is traversable and it also splits up
the
9           1st parameter: coords into and x and y integer it then runs the self.__Heuristics function
10          Calculates the total distance and creates a variable to store it (movement_cost+heuristics)
11          creates a variable with a list and a variable to check when it has been traversed.
12          :param coords:
13          :param target:
14          :param cost:
15          :param map:
```

```
16        :param Node_map:
17        :param parent:
18        '''
19        self.__coords = coords
20        self.__parent = parent
21        self.__map = map
22        self.__Node_map = Node_map
23        if map[coords[0]][coords[1]] == 0: # checks if the space is traversable if 1 it is not
24            self.__traversable = True
25        else:
26            self.__traversable = False
27
28        self.__target = target
29        self.__y = self.__coords[0] # [y,x] because in the map the y direction is the first index
30        self.__x = self.__coords[1]
31        self.__movement_cost = cost
32        self.__Heuristics()
33        self.__Total_distance = self.__movement_cost + self.__heuristics
34
35        self.__Nieghbours = []
36        self.__traversed = False
37
38    def __Heuristics(self):
39        '''
40        Does the Heuristics equation which is (x + y) + ((2**0.5) * x or y) depending on which one
41        is smaller this is saved as one of the classes variables.
42        :return:
43        '''
44        distance_x = abs(self.__x - self.__target[1])
45        distance_y = abs(self.__y - self.__target[0])
46        if distance_x < distance_y:
```

```
47              self.__heuristics = (distance_x + distance_y) + ((2 ** 0.5) * distance_x)
48          else:
49              self.__heuristics = (distance_x + distance_y) + ((2 ** 0.5) * distance_y)
50
51      def create_neighbours(self):
52          '''
53          Creates a list of the coords of the neigbhours next to it
54          :return:
55          '''
56          Nieghbours = []
57
58
59          directions = [(1, 0, 1), (1,-1, 2**0.5), (0, 1, 1),  (1,1, 2**0.5),(-1, 0, 1), (-1,-1, 2**0.5),(0, -1, 1),
(-1,1, 2**0.5)]
60          checks = []
61          for i in directions:
62              if self.__y + i[0] > -1 and self.__y + i[0] < len(self.__map) and self.__x + i[1] > -1 and
self.__x + i[1] < len(self.__map[1]):
63                  checks.append(True)
64              else:
65                  checks.append(False)
66          for i in range(len(directions)):
67              if checks[i]:
68                  Nieghbours.append([self.__y + directions[i][0], self.__x + directions[i][1],
directions[i][2]])
69
70          self.__Nieghbours = Nieghbours
71
72
73
74
75      def check_neighbours(self):
```

```
76          '''
77          This is the A* algorithm. It first checks if it is the target if not it looks through its
78          neighbours. It then checks if it can be traversed if it can it checks that it gives a smaller
distance to the target
79          if it does it runs the function update node map appends its coords to the a list and then
returns that list the list
80          is the return of a check neighbours function of the neigbhour with a smaller distance.
81          :param travelled_distance:
82          :param new_distance:
83          :return:
84          '''
85          if self.__traversable:
86              self.Traveled()
87              new_distance = self.give_distance()# The new distance to find a smaller distance of
88
89              if self.__coords == self.__target:
90                  self.__traversed = False
91                  return [self.__coords] # send back a list of all the moves
92
93              for neighbour in self.__Nieghbours:
94
95                  neighbour_Node = self.__Node_map[neighbour[0]][neighbour[1]] # gets the Node class
ascosiated with those coords
96                  #if neighbour_Node.check_traverse(): # checks it can be traversed
97                  #sets variables
98                  neighbour_Node.set_parent(self.__coords)
99                  neighbour_Node.update_movement_cost(neighbour[2])
100                 neighbour_Node.update_target(self.__target)
101
102
103
```

```
104             if neighbour_Node.give_distance() < self.__Total_distance and not
neighbour_Node.check_traveled():
105             #This if statment checks if the distance is smaller between them and the target than
this node
106             #if they have been traversed yet
107
108             next_current = neighbour # sets next current to nighbour
109             self.__Node_map[neighbour[0]][neighbour[1]] = neighbour_Node # makes sure that
map has the updated node
110             for Node_line in self.__Node_map:
111                 for Node in Node_line:
112                     Node.update_Node_map(self.__Node_map) # sends out updadted node list to
other nodes
113             list = self.__Node_map[next_current[0]][next_current[1]].check_neighbours()
114             if list == None:
115                 list = []
116             self.__traversed = False # resets traversal
117             list.append(self.__coords) # puts their coords onto the list
118             return list
119
120
121     def set_parent(self, parent):
122         '''
123         Takes parameter and assigns it to a class variable
124         :param parent:
125         :return:
126         '''
127         self.__parent = parent
128
129     def update_target(self, target):
130         '''
131         Takes parameter and assigns it to a class (target) variable it then runs the __Heuristics
function
```

```
132         and then recalculates the class variable "total distance"
133         :param target:
134         :return:
135         '''
136         self.__target = target
137         self.__Heuristics()
138         self.__Total_distance = self.__movement_cost + self.__heuristics
139
140     def update_movement_cost(self, cost):
141         '''
142         Takes parameter and assigns it to class variable "Movement cost"
143         :param cost:
144         :return:
145         '''
146         self.__movement_cost = cost
147
148
149     def update_Node_map(self, map):
150         '''
151         Takes parameter and assigns it to the class variable "Node map"
152         :param map:
153         :return:
154         '''
155         self.__Node_map = map
156
157     def give_coords(self):
158         '''
159         return class variable "coords"
160         :return:
161         '''
162         return self.__coords
```

```
163
164     def give_neighbours(self):
165         '''
166         return class variable "Neighbours"
167         :return:
168         '''
169         return self.__Nieghbours
170
171     def check_traverse(self):
172         '''
173         Return class variable "traversable"
174         :return:
175         '''
176         return self.__traversable
177
178     def give_distance(self):
179         '''
180         returns class variable "total distance"
181         :return:
182         '''
183         return self.__Total_distance
184
185     def check_traveled(self):
186         '''
187         Returns class variable "traversed"
188         :return:
189         '''
190         return self.__traversed
191
192     def Traveled(self):
193         '''
```

```
194         Assigns True to class variable "traversed"
195         :return:
196         '''
197         self.__traversed =True
198
199
200 class C: # Connection
201     def __init__(self):
202         '''
203         Creates class Variables:
204         "coords" : None
205         "player" : object of character class
206         "map" : list
207         It then runs the function coords recieve and weapon hitbox recieve
208         '''
209         self.coords = None
210         self.player = Character_class()
211         self.monsters = []
212         self.map = []
213
214
215         self.coords_recieve()
216         self.weapon_hitbox_recieve()
217
218     def create_monsters(self, monsters_n, placement_coords, room):
219         '''
220         loops number of times for the 1st parameter and appends the class variable "monsters"
with an
221         instance of the monster class for so many times it loops for
222         :param monsters_n:
223         :param placement_coords:
```

```
224         :param room:
225         :return:
226         '''
227         for i in range(monsters_n):
228             self.monsters.append(Monster(self.coords, placement_coords, room))
229
230
231     def coords_recieve(self):
232         '''
233         sets the class variable "coords" to the return value of the object player`s return of give cord.
234         :return:
235         '''
236         self.coords = self.player.give_coord()
237
238     def player_room_active(self, room):
239         for i in self.monsters:
240             i.get_active_room(room)
241
242     def weapon_hitbox_recieve(self):
243         '''
244         sets the class variable "weapon hitbox" to the objects player return of give weapon group
245         :return:
246         '''
247         self.weapon_hitbox = self.player.give_weapon_group() # function returns pygame group
that is used as a hitbox with the collide sprite function
248     def coords_give(self):
249         '''
250         loops through class variable list of monsters and each loop it gives that monster the coords
251         via the function recieve coords from the monster class
252         :return:
253         '''
```

```
254         for i in self.monsters:
255             i.recieve_coords(self.coords)
256
257
258     def map_get(self, map):
259         '''
260         Sets parameter to class variable "map"
261         :param map:
262         :return:
263         '''
264         self.map = map
265
266
267     def map_give(self):
268         '''
269         loops through every monster in its list and gives them all a map via the recieve map function
270         from the monster class.
271         :return:
272         '''
273         for i in self.monsters:
274             i.recieve_map(self.map)
275
276     def damage_bits(self,players):
277         '''
278         gets 2 variables form the player class function give damage bits. It then loops through
279         monster list and runs function from monster class set damage bits and if they died it removes them from the game.
280         :return:
281         '''
282         damage,attacking = self.player.give_damage_bits()
283         for i in self.monsters:
```

```
284             i.set_damage_bits(attacking,damage)
285             if i.dying():
286                 self.monsters.remove(i)
287                 players.remove(i)
288
289
290 class Character_class(pygame.sprite.Sprite):
291     def __init__(self):
292         '''
293         This sets up the character class`s Varaibles and loads the image
294         that is a sprite sheet it also inizilises the weapon class as a variable
295         '''
296         super().__init__()
297         self.__Health = 50
298         self.__gold = 0
299         self.__weapon_type = 'sword'
300         self.__dashing = False
301         self.__dash_counter = 0
302         self.__invunrebility = False
303         self.__colour = (255, 255, 255)
304         self.__sheet = pygame.image.load('images/sprite_sheet.png')#.convert_alpha()
305         pygame.draw.rect(self.__sheet, self.__colour, self.__sheet.get_rect(),1)  # display of rect to
be taken out only used for testing
306         self.__width = 25
307         self.__height = 25
308         self.__frame = 0
309         self.__walking = False
310         self.image = pygame.Surface([self.__width,self.__height])
311         self.image.fill((1,0,0))
312         self.image.blit(self.__sheet, (0, 0), (0, 0, self.__width, self.__height))
313         self.image.set_colorkey((1,0,0))
```

```
314
315         self.rect = self.image.get_rect()
316         self.rect.x = 400
317         self.rect.y = 200
318         self.__walls = None
319
320         #set up collisions
321         self.last_move = []
322         pygame.draw.rect(self.__sheet, self.__colour, self.rect,1)  # display of rect to be taken out only used for testing
323
324         #WEAPON
325         self.__weapon = Weapon(1, self.__weapon_type, 1)
326         self.__weapon_group = pygame.sprite.Group()
327         self.__weapon_group.add(self.__weapon.give_hit_boxes()[0])
328         self.__weapon_group.add(self.__weapon.give_hit_boxes()[1])
329
330     def wall_setter(self, walls):
331         '''
332         Sets parameter to class variable "walls"
333         :param walls:
334         :return:
335         '''
336         self.__walls = walls
337     def give_coord(self):
338         '''
339         returns a list of its y coord and x coord
340         :return:
341         '''
342         return [self.rect.y,self.rect.x]
343     def set_coords(self, coords):
```

```
344         '''
345         sets the parameter[0] to the y coord and parameter [1] to the x coord
346         It also mutliplies the y coord by 15 and x coord by 20 so that it is the
347         coords of specific pixles.
348         :param coords:
349         :return:
350         '''
351         self.rect.y = coords[0] * 15
352         self.rect.x = coords[1] * 20
353     def give_weapon_group(self):
354         '''
355         Returns class variable "weapon group"
356         :return:
357         '''
358         return self.__weapon_group
359     def give_damage_bits(self):
360         '''
361         This returns the damage of the weapon from weapon.give damage
362         and returns if the player is attacking from weapon. attacking
363         :return: damage , attacking
364         '''
365         damage = self.__weapon.give_damage()
366         attacking = self.__weapon.attacking()
367         return damage,attacking
368     def __collisions(self):
369         '''
370         checks if a collision is made with the player and the wall
371         If it has it uses the class variable "last move" to determine which
372         direction the players icon is moved so that it is no longer coliding with the wall
373
374         :return:
```

```
375         '''
376         if pygame.sprite.spritecollide(self, self.__walls, False):
377             self.__colour = (0, 255, 0)
378             for i in self.last_move:
379                 if i == 'w':
380                     self.rect.y += 1
381 
382                 if i == 's':
383                     self.rect.y -= 1
384 
385                 if i == 'a':
386                     self.rect.x += 1
387 
388                 if i == 'd':
389                     self.rect.x -= 1
390     def __movement(self):
391         '''
392         creates a varibale of a boolean list of all inputs
393         it gets the position of the mouse
394         sets class variable invunrability to false
395         checks if player is dashing if it is dash counter goes up 1 and if dash counter is 100 then dashing is false dash counter is set to 0
396         then it checks if player is holding left control if so then the player dashes 30 pixles towards the mouse
397         after this it checks for every input of W A S or D and then moves the player in that respective direction (0.5 pixles if holding shift or 1 if not)
398         :return:
399         '''
400         inputs = pygame.key.get_pressed()
401         mouse = pygame.mouse.get_pos()
402         self.__invunrebility = False
403         if self.__dashing:
```

```
404             self.__dash_counter += 1
405             if self.__dash_counter == 1000:
406                 self.__dashing = False
407                 self.__dash_counter = 0
408
409
410         if inputs[pygame.K_LCTRL] and self.__dashing == False:
411             if self.rect.x < mouse[0]:
412                 amount = mouse[0] - self.rect.x
413                 if amount > 30:
414                     self.rect.x += 30
415                 else:
416                     self.rect.x += amount
417
418             elif self.rect.x > mouse[0]:
419                 amount =self.rect.x - mouse[0]
420                 if amount > 30:
421                     self.rect.x -= 30
422                 else:
423                     self.rect.x -= amount
424
425             if self.rect.y < mouse[1]:
426                 amount = mouse[1] - self.rect.y
427                 if amount > 30:
428                     self.rect.y += 30
429                 else:
430                     self.rect.y += amount
431
432             elif self.rect.y > mouse[1]:
433                 amount =self.rect.y - mouse[1]
434                 if amount > 30:
```

```
435                 self.rect.y -= 30
436             else:
437                 self.rect.y -= amount
438 
439         self.__dashing = True
440         self.__invunrebility = True
441 
442 
443     if inputs[pygame.K_LSHIFT]:
444         if inputs[pygame.K_w]:
445             self.last_move.append('w')
446             self.rect.y -= 0.5
447             self.__walking = True
448 
449         if inputs[pygame.K_s]:
450             self.last_move.append('s')
451             self.rect.y += 0.5
452             self.__walking = True
453 
454 
455         if inputs[pygame.K_a]:
456             self.last_move.append('a')
457             self.rect.x -= 0.5
458             self.__walking = True
459 
460 
461         if inputs[pygame.K_d]:
462             self.last_move.append('d')
463             self.rect.x += 0.5
464             self.__walking = True
465 
```

```
466
467         elif not inputs[pygame.K_LSHIFT]:
468             if inputs[pygame.K_w]:
469                 self.last_move.append('w')
470                 self.rect.y -= 1
471                 self.__walking = True
472
473
474
475             if inputs[pygame.K_s]:
476                 self.last_move.append('s')
477                 self.rect.y += 1
478                 self.__walking = True
479
480
481             if inputs[pygame.K_a]:
482                 self.last_move.append('a')
483                 self.rect.x -= 1
484                 self.__walking = True
485
486             if inputs[pygame.K_d]:
487                 self.last_move.append('d')
488                 self.rect.x += 1
489                 self.__walking = True
490
491
492         else:
493             self.__walking = False
494
495         if len(self.last_move) > 15:
496             self.last_move = []
```

```
497     def __animation(self):
498         '''
499         A function that swaps between pictures on the sprite sheet to make it look like the
    character is walking
500         :return:
501         '''
502         if self.__walking == True:
503             self.image = self.__get_image(self.__frame)
504             self.__frame += 1
505             if self.__frame == 5:
506                 self.__frame = 0
507     def update(self):
508         '''
509         Where every function that runs every loop is ran from thatare to do with the character class
510         :return:
511         '''
512         self.__movement()
513         self.__collisions()
514         self.__weapon.coords([self.rect.y,self.rect.x])
515         self.__weapon.input()
516
517     def give_gold(self, gold):
518         '''
519         takes the variable and adds it the character class variable "gold".
520         :param gold:
521         :return:
522         '''
523         self.__gold += gold
524
525     def take_gold(self, gold):
526         '''
```

```
527         takes the parameter and takes it away from the vaaribale gold
528         :param gold:
529         :return:
530         '''
531         self.__gold -= gold
532
533     def return_gold(self):
534         return self.__gold
535
536     def return_health(self):
537         return self.__Health
538
539     def give_health(self):
540         self.__Health +=5
541
542     def give_damage(self):
543         self.__weapon.increase_damage()
544
545     def return_damage(self):
546         damage = self.__weapon.give_damage()
547         return damage
548
549     def __get_image(self,frame):
550         '''
551         selects the image of the sprite sheet of the specific frame is needed in that animation and
puts it on as the one that is being used
552         :param frame:
553         :return:
554         '''
555         image = pygame.Surface((self.__width, self.__height)).convert_alpha()
556         image.fill((1,0,0))
```

```
557         image.blit(self.__sheet, (0, 0), ((frame * self.__width), 0, self.__width, self.__height))
558         image.set_colorkey((1,0,0))
559         return image
560 
561     def give_weapon(self):
562         '''
563         Returns the sprites of the hitboxes of the weapon
564         :return:
565         '''
566         return self.__weapon.give_hit_boxes()
567 
568     def take_damage(self, damage):
569         self.__Health -= damage
570         if self.__Health <= 0:
571             return True
572         else:
573             return False
574 
575 
576 class Weapon():
577     def __init__(self, damage, type, size):
578         '''
579         Sets up all the variables of the weapon class and initilises to instances of the weapon sprite
class
580         :param damage:
581         :param type:
582         :param size:
583         '''
584         # variables
585         self.__size = size
586         self.__damage = damage
```

```
587         self.__type = type
588         self.__Attacking = False
589         self.__count = 0
590 
591         #pygame
592         self.__Side_hitbox = Weapon_sprite(self.__size, (0,0), True)
593         self.__Top_Bottom_hitbox = Weapon_sprite(self.__size, (0, 0), False)
594 
595     def input(self):
596         '''
597         Checks if the R key has been pressed and if "count" is 0
598         if it has it sets "attacking" to true
599         and "count" is set to 100
600         while Attacking is False and "count" is not 0
601         1 is taken away from "count"
602         :return:
603         '''
604         inputs = pygame.key.get_pressed()
605         if inputs[pygame.K_r] and self.__count == 0:
606             self.__Attacking = True
607             self.__count = 100
608         else:
609             self.__Attacking = False
610             if self.__count !=0:
611                 self.__count -= 1
612 
613     def coords(self, coords):
614         '''
615         Sets the coords of the 2 weapon sprite classes inside of weapon by running
616         the .coords function of the weapon sprite class
617         :param coords:
```

```
618         :return:
619         '''
620         self.__Side_hitbox.coords(coords)
621         self.__Top_Bottom_hitbox.coords(coords)
622 
623     def give_hit_boxes(self):
624         '''
625         Returns the 2 instances of the class weapon sprites
626         :return:
627         '''
628         return [self.__Side_hitbox, self.__Top_Bottom_hitbox]
629 
630     def give_damage(self):
631         '''
632         return the damage of the weapon
633         :return:
634         '''
635         return self.__damage
636 
637     def attacking(self):
638         '''
639         If "attacking" is true then the weapon sprtie function attacking is run
640         If count is 0 the hitboxes reset with the reset function of weapon sprite
641         Then "attack" is returned
642         :return:
643         '''
644         if self.__Attacking:
645             self.__Side_hitbox.attacking()
646             self.__Top_Bottom_hitbox.attacking()
647         elif self.__count == 0:
648             self.__Side_hitbox.reset()
```

```
649             self.__Top_Bottom_hitbox.reset()
650         return self.__Attacking
651
652     def increase_damage(self):
653         self.__damage += 1
654
655
656 class Weapon_sprite(pygame.sprite.Sprite):
657     def __init__(self, size, coords, side):
658         '''
659         It sets the weapon Sprites classes variables then it checks the Boolean parameter "side"
660         if it is True or False it will take a different surface of the inverse of eachother
661         This creates the two hitboxes
662         :param size:
663         :param coords:
664         :param side:
665         '''
666         super().__init__()
667         #Variables
668         self.__size = size
669         self.__coords = coords
670         self.__side = side
671         #pygame bits
672         if self.__side:
673             self.image = pygame.Surface([60 * size, 20 * size])
674
675         else:
676             self.image = pygame.Surface([20 * size, 60 * size])
677
678         self.image.fill(('lightgrey'))
679
```

```
680         self.rect = self.image.get_rect()
681         self.rect.x = 0
682         self.rect.y = 0
683 
684     def coords(self, coords):
685         '''
686         It checks where the mouse is on the screen
687         It sets the list parameter "coords" to the coordinates of the rect
688         It then changes the coords of the Sprite to face to where the mouse is.
689         :param coords:
690         :return:
691         '''
692         mouse_pos = pygame.mouse.get_pos()
693         if self.__side: # if it is the vertical hitboxes
694             if mouse_pos[1] < coords[0]:
695                 self.rect.x = coords[1] - 20 * self.__size
696                 self.rect.y = coords[0] - 15 * self.__size
697 
698             elif mouse_pos[1] > coords[0]:
699                 self.rect.x = coords[1] - 20 * self.__size
700                 self.rect.y = coords[0] + 25 * self.__size
701 
702         else:
703             if mouse_pos[0] < coords[1]:
704                 self.rect.x = coords[1] - 25
705                 self.rect.y = coords[0] - 20 * self.__size
706 
707             elif mouse_pos[0] > coords[1]:
708                 self.rect.x = coords[1] + 25
709                 self.rect.y = coords[0] - 20 * self.__size
710 
```

```
711     def attacking(self):
712         '''
713         Depending on the "side" boolean varibale it cheanges the colour of the rect to
714         red or yellow.
715         :return:
716         '''
717         if self.__side:
718             pygame.draw.rect(self.image, (255, 0, 0), self.image.get_rect())
719 
720         else:
721             pygame.draw.rect(self.image, 'yellow', self.image.get_rect())
722 
723     def reset(self):
724         '''
725         Fills the image with the colour light grey
726         :return:
727         '''
728         self.image.fill(('lightgrey'))
729 
730 class Monster (pygame.sprite.Sprite):
731     def __init__(self, target, coords, room):
732         '''
733         Sets up the varibales of the Monster Class
734         :param target:
735         :param coords:
736         :param room:
737         '''
738         super().__init__()
739         self._multiplier = 1
740         self.__health = self._multiplier * 10
741         self.__Damage = 1 * self._multiplier
```

```
742         self.image = pygame.image.load('images/monster.png')
743         self.rect = self.image.get_rect()
744         self.rect.x = coords[1]
745         self.rect.y = coords[0]
746         self.__player_coord = [0,0]
747         self.__target_x = None
748         self.__target = target
749         self.__target_y = None
750         self.__t_y_node = 0
751         self.__t_x_node = 0
752         self.__count = 0
753         self.__Same_target = False
754         self.__taking_damage = False
755         self.__player_damage = 0
756         self.__weapon_hitboxes = None
757         self.__room = room
758         self.__active_room = 0
759         self.__walls = None
760         self.__last_move = []
761         self.__movement_list = []
762         self.__damage_count = 0
763
764     def update(self):
765         '''
766         Where all the function of the monster class are ran from each loop
767         :return:
768         '''
769         if self.__room_check():
770             self.__detection()
771             self.__movement()
772             self.__collision()
```

```
773             self.__take_damage()
774
775     def recieve_coords(self, coords):
776         '''
777         it takes the "coords" list paremeter and checks if coords are the same
778         if they are a boolean variable "Same Target" is set as True
779         if not "target coords" are aset as coords and "Same Target" is set to False
780         and integer variable "count" is set to 0
781         :param coords:
782         :return:
783         '''
784         if coords == self.__target:
785             self.__Same_target = True
786         else:
787             self.__target = coords
788             self.__Same_target = False
789             self.__count = 0
790
791     def recieve_map(self,map):
792         '''
793         Sets 2d list parameter "map" to the class varibale "map"
794         :param map:
795         :return:
796         '''
797         self.__map = map
798
799     def recieve_weapon_hitbox(self, group):
800         '''
801         Assiigns the object parameter "group" to the class variable "weapon hitboxes"
802         :param group:
803         :return:
```

```
804         '''
805         self.__weapon_hitboxes = group
806
807     def __detection(self):
808         '''
809         It checks if it has the same target if not it runs the check neighbour function of the class
node to detect where the player is.
810         :return:
811         '''
812
813
814         starting_x = self.rect.x // 100  # 20
815         starting_y = self.rect.y // 100  # 15
816         #Discover patha
817
818         #set nodes
819         #new_t_y_node = self.__target[0] // 100
820         #new_t_x_node = self.__target[1] // 100
821
822         self.__t_y_node = self.__target[0] // 100
823         self.__t_x_node = self.__target[1] // 100
824
825         target = [self.__t_y_node, self.__t_x_node]
826
827         current = self.__map[starting_y][starting_x]
828
829         current.update_target(target)
830
831         if current.give_coords() == target:
832             self.__target_x = self.__target[1]
833             self.__target_y = self.__target[0]
```

```
834
835         if self.rect.x > self.__target_x:
836             self.__target_x += 15
837
838         else:
839             self.__target_x -= 15
840
841         if self.rect.y > self.__target_y:
842             self.__target_y += 15
843
844         else:
845              self.__target_y += 15
846
847       elif self.__Same_target == False:
848           movement_list = current.check_neighbours()
849           if movement_list != None:
850               self.__movement_list = movement_list
851               self.__movement_list.reverse()
852
853           self.__target_x = self.__movement_list[0][1] * 100# give coords of first node to travel too
854           self.__target_y = self.__movement_list[0][0] * 100
855
856       else:
857           self.__target_x = self.__movement_list[self.__count][1] * 100  # give coords of first node
to travel too
858           self.__target_y = self.__movement_list[self.__count][0] * 100
859
860           if starting_x == self.__target_x // 100 and starting_y == self.__target_y // 100 and
self.__count != len(self.__movement_list) - 1:
861               self.__count += 1
862
863     def __room_check(self):
```

```
        '''
        Make a variable that tells the monster what room it is in (self variable)
        Make a variable that tells what rooms the player is in (self variable of a player variable)
        if this number is the same then they are in the same room therefore detection function should start
        :return:
        '''
        if self.__active_room == self.__room:
            return True
        else:
            return False

    def get_active_room(self,room):
        self.__active_room = room

    def __take_damage(self):
        '''
        It checks if a collision has occured and if so then it checks wether
        it should be taking damage if so it takes damage up to the damage of the characters damage
        :return:
        '''
        if pygame.sprite.spritecollide(self,self.__weapon_hitboxes,False):
            if self.__taking_damage:
                self.__health -= self.__player_damage

    def set_damage_bits(self,taking,damage):
        '''
        Assigns the parameters to "taking damage" a Boolean varibale to check
        wether the monster should be taking damage
        and "player damage" and integer varibale that stores how much damage the player does
        :param taking:
```

```
894         :param damage:
895         :return:
896         '''
897         self.__taking_damage = taking
898         self.__player_damage = damage
899 
900     def dying(self):
901         '''
902         If health drops bellow 0 the monster has died so it returns True
903         This will then start the monster to be deleted from the game so that it has died
904         :return:
905         '''
906         if self.__health <= 0:
907             return True
908         else:
909             return False
910 
911     def __movement(self):
912         '''
913         Moves the monster toward the target coordinates by adding or subtracting its y and x
values
914         :return:
915         '''
916         if self.__map[self.__target_y//100][self.__target_x//100].check_traverse:
917             if self.rect.x < self.__target_x:
918                 self.rect.x += 1
919                 self.__last_move.append('d')
920 
921             else:
922                 self.rect.x -= 1
923                 self.__last_move.append('a')
```

```
924
925             if self.rect.y < self.__target_y:
926                 self.rect.y +=1
927                 self.__last_move.append('s')
928             else:
929                 self.rect.y -= 1
930                 self.__last_move.append('w')
931
932         if len(self.__last_move) > 7:
933             self.__last_move = []
934
935     def __collision(self):
936         if self.__walls != None:
937             if pygame.sprite.spritecollide(self, self.__walls, False):
938                 for i in self.__last_move:
939                     if i == 'w':
940                         self.rect.y += 1
941
942                     if i == 's':
943                         self.rect.y -= 1
944
945                     if i == 'a':
946                         self.rect.x += 1
947
948                     if i == 'd':
949                         self.rect.x -= 1
950
951     def wallsetter(self, walls):
952         self.__walls = walls
953
954     def player_colision(self, player):
```

```
955         if self.__damage_count == 0:
956             if self.rect.colliderect(player):
957                 self.__damage_count += 1
958                 return True
959             else:
960                 return False
961         else:
962             self.__damage_count += 1
963             if self.__damage_count == 90:
964                 self.__damage_count = 0
965
966     def return_damage(self):
967         return self.__Damage
```

Generation.py

```
1   import pygame
2   import random
3   from Character import C
4   class Wall(pygame.sprite.Sprite):
5       def __init__(self,x,y):
6           '''
7           Loads the image of wall.png into the self.image class variable and sets the
8           coordinates according to the integer parameters
9           :param x:
10          :param y:
11          '''
12          super().__init__()
13
14          self.image = pygame.image.load('images/wall.png')
15          self.rect = self.image.get_rect()
16          self.rect.y = y
```

```
17          self.rect.x = x
18
19  class Environmental(pygame.sprite.Sprite):
20      def __init__(self, environment, x, y):
21          '''
22          Depending on the Boolean parameter it will load the image of either rock1 or rock2 .png
23          it then sets the coords of these images according to the integer parameters
24          :param environment:
25          :param x:
26          :param y:
27          '''
28          super().__init__()
29          self.image = pygame.Surface([20, 15])
30          self.image.fill((1,0,0))
31          if environment:
32              self.__sheet = pygame.image.load('images/rock_1.png')
33
34          else:
35              self.__sheet = pygame.image.load('images/rock_2.png')
36
37          self.image.blit(self.__sheet, (0, 0), (0, 0, 20, 15))
38          self.image.set_colorkey((1, 0, 0))
39
40          self.rect = self.image.get_rect()
41          self.rect.y = y
42          self.rect.x = x
43
44  class Exit(pygame.sprite.Sprite):
45      def __init__(self, x, y, players):
46          '''
```

```
47      The inmage hatch is loaded and the rect/hitbox is created of the image and the coordinates are set using the parameters and the other variables thge class uses are created
48      :param x:
49      :param y:
50      :param players:
51      '''
52      super().__init__()
53      self.image = pygame.Surface([20, 15])
54      self.__sheet = pygame.image.load('images/Hatch.png')
55      self.image.fill((1,0,0))
56      self.image.blit(self.__sheet, (0,0), (0,0, 20,15))
57      self.image.set_colorkey((1,0,0))
58      self.rect = self.image.get_rect()
59      self.rect.y = y
60      self.rect.x = x
61      self.__players = players
62      self.__Enter_clause_is_complete = False
63
64  def entering(self):
65      '''
66      Checks for a colision between the player character and the rect/hiotbox of the image
67      It then checks for if the enter clause is complete and if the etner key has been pressed
68      if that is all True then it returns True if not then it returns false
69      :return:
70      '''
71      inputs = pygame.key.get_pressed()
72      colision = pygame.sprite.spritecollide(self, self.__players, False)
73      if colision and inputs[pygame.K_RETURN] and self.__Enter_clause_is_complete:
74          return True
75      else:
76          return False
```

```
77
78      def update_exit_clause(self,players):
79          '''
80          It checks whether the length of the parameter object players is 1 which means only the player is now alive
81          If that is true the enter clause is set to being True.
82          :param players:
83          :return:
84          '''
85          if len(players) == 1:
86              self.__Enter_clause_is_complete = True
87
88
89
90  class Map:
91      def __init__(self):
92          '''
93          It creates a 2d list of the size 70 by 70 by determining a random number 1 or 0 it selects which side the second room will come from
94          the first room is created using the room class room 2 is then created with the starting coordinates either below or next to room 1
95          room 3 is then created with the opposite direction to room 2 than room 2 was to room 1. The coords of each room are then assigned to variables.
96          '''
97          self._map = []
98          for i in range(0, 70):
99              self._map.append([])
100             for x in range(0, 70):
101                 self._map[i].append(0)
102
103         self.__dimensions = [0, 0]
104         self.__rooms = []
```

```
105         'random.randint(0,1)'
106         if  random.randint(0,1) == 1:
107             side1 = True
108             side2 = False
109         else:
110             side1 = False
111             side2 = True
112         height = random.randint(10, 20)
113         width = random.randint(13, 20)
114
115         room1 = Room(width, height, [self.__dimensions[0] + 3, self.__dimensions[0] + 3],
self._map, side2,1)
116         self.__rooms.append(room1)
117         self._map = room1.give_map_back()
118         self.__dimensions = room1.give_dimensions_back()
119
120
121         height = random.randint(10, 20)
122         width = random.randint(13, 20)
123
124         room2 = Room(width, height, self.__dimensions, self._map, side1,2)
125         self.__rooms.append(room2)
126         self._map = room2.give_map_back()
127         self.__dimensions = room2.give_dimensions_back()
128
129         room3 = Room(width, height, self.__dimensions, self._map, side2,3)
130         self.__rooms.append(room3)
131         self._map = room3.give_map_back()
132         test = room3.give_map_back()
133         self.__dimensions = room3.give_dimensions_back()
134         self.__connections()
```

```
135         self.__exit()
136
137         self.__players_room = random.randint(0,2)
138         self.__player_placement = self.__rooms[self.__players_room].player_placement()
139
140         self.__room1_coords = room1.return_room_coords()
141         self.__room2_coords = room2.return_room_coords()
142         self.__room3_coords = room3.return_room_coords()
143
144
145     def __connections(self):
146         '''
147         A hallway is created betwee 2 rooms in the direction that thery are next to eachother by this is done by
148         changing the 1's and 0's to make the shape of a hallway onto the map that is one block wide which is enough room for the player.
149         It also checks for if there are any indents and then changes the where the hallway is put.
150         :return:
151         '''
152         for i in range(len(self.__rooms) - 1):
153             print('running')
154             room = self.__rooms[i]
155             conneting = self.__rooms[i+1]
156             gap = 3
157             room_width0_height1 = room.give_width_height()
158             zero_room = room.give_zero_width_height()
159             zero_connecting = conneting.give_zero_width_height()
160             connecting_width0_height1 = conneting.give_width_height()
161
162             if room.give_side():
163
164                 #clear out space
```

```
165
166
167             loop = True
168             count = 1
169             while loop:
170                 if self._map[zero_room[1] + 4][zero_room[0] + room_width0_height1[0] + count] ==
0:
171                     count += 1
172                 else:
173                     loop = False
174
175             if room_width0_height1[1] > connecting_width0_height1[1]:
176                 half_height = connecting_width0_height1[1] // 2
177             else:
178                 half_height = room_width0_height1[1] // 2
179             print(half_height)
180             for i in range(gap):
181                 self._map[zero_room[1] + half_height + i][zero_room[0] + room_width0_height1[0]]
= 0
182
183                 self._map[zero_connecting[1] + half_height + i][zero_connecting[0]] = 0
184
185             #Upper
186             for i in range(count):
187                 self._map[zero_room[1] + half_height - 1][zero_room[0] + room_width0_height1[0]
+ i] = 1
188                 #lower
189                 self._map[zero_room[1] + half_height + gap][zero_room[0] +
room_width0_height1[0] + i] = 1
190
191
192
```

```
193         else:
194             #checking for indent length
195             room_B_working_room = room.give_B_working_room()
196             if room_width0_height1[0] < connecting_width0_height1[0]:
197                 half_width = room_width0_height1[0] // 2
198             else:
199                 half_width = connecting_width0_height1[0] // 2
200             #geting the count
201             count = 1
202             loop = True
203             while loop:
204                 if self._map[zero_room[1] + room_width0_height1[1] + room_B_working_room + count][zero_room[0] + half_width] == 0:
205                     count += 1
206                 else:
207                     loop = False
208 
209             count += room_B_working_room
210 
211             #clear out space
212             for i in range(gap):
213                 for x in range(count):
214                     self._map[zero_room[1] + room_width0_height1[1] + x][zero_room[0] + half_width + i] = 0
215                     self._map[zero_connecting[1] - x][zero_connecting[0] + half_width + i] = 0
216 
217             #Add in walls
218             for i in range(count):
219                 #left
220                 self._map[zero_room[1] + room_width0_height1[1] + i][zero_room[0] + half_width - 1] = 1
221                 #right
```

```
222                 self._map[zero_connecting[1] - i][zero_connecting[0] + half_width + gap] = 1
223
224     def __exit(self):
225         '''
226         It choses a room at random it then runs the function create exit on the room that was chosen
227         :return:
228         '''
229         room = random.randint(0,len(self.__rooms) - 1)
230         for i in range(len(self.__rooms)):
231             if room == i:
232                 self.__rooms[i].create_exit()
233
234     def place_enemy(self, C):
235         '''
236         It loops through all of the rooms if the player doesnt spawn in that room an enemy is placed by the place_enemy function
237         :param C:
238         :return:
239         '''
240         for i in range(len(self.__rooms)):
241             if i != self.__players_room:
242                 self.__rooms[i].place_enemy(C)
243
244     def check_room(self, C):
245         '''
246         Checks if the plkayer is in any of the rooms by looking at the coordinates of the player and looking at thye coordinates of which the rooms span
247         This is then put into the function player_active_room
248         :param C:
249         :return:
250         '''
```

```
251         coords = C.player.give_coord()
252         coords[0] = coords[0]//15
253         coords[1] = coords[1]//20
254         room = None
255         #room 1
256             #y coord check
257         if coords[0] > self.__room1_coords[1] and coords[0] < (self.__room1_coords[1] +
self.__room1_coords[3]):
258             #x coord check
259             if coords[1] > self.__room1_coords[0] and coords[1] < (self.__room1_coords[0] +
self.__room1_coords[2]):
260                 room = 1
261         # room 2
262         # y coord check
263         if coords[0] > self.__room2_coords[1] and coords[0] < (self.__room2_coords[1] +
self.__room2_coords[3]):
264             # x coord check
265             if coords[1] > self.__room2_coords[0] and coords[1] < (self.__room2_coords[0] +
self.__room2_coords[2]):
266                 room = 2
267         # room 2
268         # y coord check
269         if coords[0] > self.__room3_coords[1] and coords[0] < (self.__room3_coords[1] +
self.__room3_coords[3]):
270             # x coord check
271             if coords[1] > self.__room3_coords[0] and coords[1] < (self.__room3_coords[0] +
self.__room3_coords[2]):
272                 room = 3
273
274         C.player_room_active(room)
275
276     def give_map(self):
277         '''
```

```
278         returns the list map
279         :return:
280         '''
281         return self._map
282
283     def return_player_placement(self):
284         '''
285         returns the integer player placement
286         :return:
287         '''
288         return self.__player_placement
289
290 class Room:
291     def __init__(self, width, height, zero, map, side, room_no):
292         '''
293         Creates a random height and width this is then put onto the map with the posistions having the starting coords added onto it
294         Through a random number being generated a corner is chosen for each corner and then the orignal corner is removed and a new one is put onto the 2d list
295         It then checks if there are space for indents and if there is space if they is it dcoes an indent to how much space it has after this calculates
296         the starting coords of the next room.
297         :param width:
298         :param height:
299         :param zero:
300         :param map:
301         :param side:
302         :param room_no:
303         '''
304         self.__side = side
305         self._map = map
306         self.__height = height
```

```
307         self.__width = width
308         self.__zero_height = zero[1]
309         self.__zero_width = zero[0]  # variable that will change for rooms where they start from
310         self.__room_number = room_no
311         # Set outline
312         height_slot = self.__zero_height + self.__height
313         width_slot = self.__zero_width + self.__width
314
315         loop = True
316
317         for i in range(self.__height):
318             self._map[i + self.__zero_height][0 + self.__zero_width] = 1
319             self._map[i + self.__zero_height][self.__width + self.__zero_width] = 1
320
321         for i in range(self.__width):
322             self._map[0 + self.__zero_height][i + self.__zero_width] = 1
323             self._map[self.__height + self.__zero_height][i + self.__zero_width] = 1
324
325         self._map[self.__height + self.__zero_height][self.__width + self.__zero_width] = 1
326         # Make the corners curved
327         '''
328         start
329         [1][1][1][1]
330         [0][0][0][1]
331         [0][0][0][1]
332         [0][0][0][1]
333
334         idea 1
335         [1][1][0][0] remove 2 closet to corner
336         [0][1][0][0]add one bellow remaining 1 and remove one at side
337         [0][1][1][0]
```

```
338         [0][0][1][1]
339
340         idea 2
341         [1][1][0][0]
342         [0][1][1][0]
343         [0][0][1][0]
344         [0][0][1][1]
345
346         idea 3
347         [1][0][0][0]
348         [0][1][0][0]
349         [0][0][1][0]
350         [0][0][0][1] does not look good
351         '''
352
353         # corner_type = 2
354         # top left
355         corner_type = random.randint(0, 1)
356         if corner_type == 0:
357             self._map[0 + self.__zero_height][0 + self.__zero_width] = 0
358             self._map[0 + self.__zero_height][1 + self.__zero_width] = 0
359
360             self._map[1 + self.__zero_height][2 + self.__zero_width] = 1
361             self._map[1 + self.__zero_height][0 + self.__zero_width] = 0
362
363             self._map[2 + self.__zero_height][0 + self.__zero_width] = 0
364             self._map[2 + self.__zero_height][2 + self.__zero_width] = 1
365             self._map[2 + self.__zero_height][1 + self.__zero_width] = 1
366
367             self._map[3 + self.__zero_height][1 + self.__zero_width] = 1
368         elif corner_type == 1:
```

```
369             self._map[0 + self.__zero_height][0 + self.__zero_width] = 0
370             self._map[0 + self.__zero_height][1 + self.__zero_width] = 0
371
372             self._map[1 + self.__zero_height][2 + self.__zero_width] = 1
373             self._map[1 + self.__zero_height][0 + self.__zero_width] = 0
374             self._map[1 + self.__zero_height][1 + self.__zero_width] = 1
375
376             self._map[2 + self.__zero_height][0 + self.__zero_width] = 0
377             self._map[2 + self.__zero_height][1 + self.__zero_width] = 1
378
379             self._map[3 + self.__zero_height][1 + self.__zero_width] = 1
380
381         # bottom left
382         corner_type = random.randint(0, 1)
383         if corner_type == 0:
384             self._map[self.__height + self.__zero_height][0 + self.__zero_width] = 0
385             self._map[self.__height + self.__zero_height][1 + self.__zero_width] = 0
386
387             self._map[self.__height - 1 + self.__zero_height][2 + self.__zero_width] = 1
388             self._map[self.__height - 1 + self.__zero_height][0 + self.__zero_width] = 0
389
390             self._map[self.__height - 2 + self.__zero_height][0 + self.__zero_width] = 0
391             self._map[self.__height - 2 + self.__zero_height][2 + self.__zero_width] = 1
392             self._map[self.__height - 2 + self.__zero_height][1 + self.__zero_width] = 1
393
394             self._map[self.__height - 3 + self.__zero_height][1 + self.__zero_width] = 1
395
396         elif corner_type == 1:
397             self._map[self.__height + self.__zero_height][0 + self.__zero_width] = 0
398             self._map[self.__height + self.__zero_height][1 + self.__zero_width] = 0
399
```

```
400             self._map[self.__height - 1 + self.__zero_height][2 + self.__zero_width] = 1
401             self._map[self.__height - 1 + self.__zero_height][0 + self.__zero_width] = 0
402             self._map[self.__height - 1 + self.__zero_height][1 + self.__zero_width] = 1
403
404             self._map[self.__height - 2 + self.__zero_height][0 + self.__zero_width] = 0
405             self._map[self.__height - 2 + self.__zero_height][1 + self.__zero_width] = 1
406
407             self._map[self.__height - 3 + self.__zero_height][1 + self.__zero_width] = 1
408
409         # top right
410         corner_type = random.randint(0, 1)
411         if corner_type == 0:
412             self._map[0 + self.__zero_height][self.__width + self.__zero_width] = 0
413             self._map[0 + self.__zero_height][self.__width - 1 + self.__zero_width] = 0
414
415             self._map[1 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
416             self._map[1 + self.__zero_height][self.__width + self.__zero_width] = 0
417
418             self._map[2 + self.__zero_height][self.__width + self.__zero_width] = 0
419             self._map[2 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
420             self._map[2 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
421
422             self._map[3 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
423
424         elif corner_type == 1:
425             self._map[0 + self.__zero_height][self.__width + self.__zero_width] = 0
426             self._map[0 + self.__zero_height][self.__width - 1 + self.__zero_width] = 0
427
428             self._map[1 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
429             self._map[1 + self.__zero_height][self.__width + self.__zero_width] = 0
430             self._map[1 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
```

```
431
432         self._map[2 + self.__zero_height][self.__width + self.__zero_width] = 0
433         self._map[2 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
434
435         self._map[3 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
436
437     # bottom right
438     corner_type = random.randint(0, 1)
439     if corner_type == 0:
440         self._map[self.__height + self.__zero_height][self.__width + self.__zero_width] = 0
441         self._map[self.__height + self.__zero_height][self.__width - 1 + self.__zero_width] = 0
442
443         self._map[self.__height - 1 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
444         self._map[self.__height - 1 + self.__zero_height][self.__width + self.__zero_width] = 0
445
446         self._map[self.__height - 2 + self.__zero_height][self.__width + self.__zero_width] = 0
447         self._map[self.__height - 2 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
448         self._map[self.__height - 2 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
449
450         self._map[self.__height - 3 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
451     elif corner_type == 1:
452         self._map[self.__height + self.__zero_height][self.__width + self.__zero_width] = 0
453         self._map[self.__height + self.__zero_height][self.__width - 1 + self.__zero_width] = 0
454
455         self._map[self.__height - 1 + self.__zero_height][self.__width - 2 + self.__zero_width] = 1
456         self._map[self.__height - 1 + self.__zero_height][self.__width + self.__zero_width] = 0
457         self._map[self.__height - 1 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
458
459         self._map[self.__height - 2 + self.__zero_height][self.__width + self.__zero_width] = 0
460         self._map[self.__height - 2 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
461
```

```
462         self._map[self.__height - 3 + self.__zero_height][self.__width - 1 + self.__zero_width] = 1
463
464     # add indents
465
466     # check for valid sides
467     # checks for top
468     if self.__zero_height != 0:
469         top = True
470     elif self._map[self.__zero_height - 1][self.__zero_width + 5] == 1:
471         top = False
472     else:
473         top = False
474
475     # checks for bottom
476     if self.__zero_height + self.__height != 50:
477         bottom = True
478     elif self._map[self.__zero_height + self.__height + 1][self.__zero_width + 5] == 1:
479         bottom = False
480     else:
481         bottom = False
482
483     # top indent
484     if top == True:
485
486         width_start = 0
487         start = self.__width // 8
488         for i in range(self.__zero_width, len(self._map[0])):
489             if self._map[self.__zero_height][i] == 1:
490                 width_start = i
491                 break
492         self.__Top_working_room = 0
```

```
493         i = 0
494         while self.__Top_working_room < 3:
495             if self._map[self.__zero_height - i][self.__zero_width + 4] == 0:
496                 self.__Top_working_room += 1
497             i += 1
498             if self.__zero_height - i == -1:
499                 self.__Top_working_room = 3
500 
501         # clear out space
502         gap = self.__width // 2 + self.__Top_working_room - (self.__width // 8) * 2
503 
504         for i in range(gap):
505             self._map[self.__zero_height][i + width_start + start] = 0
506 
507         # rise
508         for i in range(self.__Top_working_room):
509             self._map[self.__zero_height - i][width_start + i + start] = 1
510             self._map[self.__zero_height - i][width_start + i + 1 + start] = 1
511 
512         # solid in middle
513         wall = gap - (self.__Top_working_room * 2) + width_start + self.__Top_working_room
514 
515         for i in range((width_start + self.__Top_working_room), wall):
516             self._map[self.__zero_height - self.__Top_working_room][i + start] = 1
517 
518         # fall
519         for i in range(self.__Top_working_room):
520             self._map[self.__zero_height - self.__Top_working_room + i][wall + i + start] = 1
521             self._map[self.__zero_height - self.__Top_working_room + i][wall + i + 1 + start] = 1
522 
523     # bottom indent
```

```
524
525         if bottom == True:
526             width_start = 0
527             start = self.__zero_width + self.__width - (self.__width // 8)
528             for i in range(self.__zero_width, len(self._map[0])):
529                 if self._map[self.__zero_height][i] == 1:
530                     width_start = i
531                     break
532             self.__Bottom_working_room = 0
533             i = 0
534
535             while self.__Bottom_working_room < 3:
536                 if self._map[height_slot + i][self.__width + 4] == 0:
537                     self.__Bottom_working_room += 1
538                 i += 1
539                 if height_slot + i == 70:
540                     self.__Bottom_working_room = 3
541
542             # create the gap
543             gap = self.__width // 2 + self.__Bottom_working_room - (self.__width // 8) * 2
544             for i in range(gap):
545                 self._map[height_slot][start - i] = 0
546
547             # rising
548             for i in range(self.__Bottom_working_room):
549                 self._map[self.__zero_height + self.__height + i][start - i] = 1
550                 self._map[self.__zero_height + self.__height + i][start - i - 1] = 1
551
552             # solid in middle
553             wall = gap - (self.__Bottom_working_room * 2) + 2
554             for i in range(wall):
```

```
555                 self._map[self.__zero_height + self.__height + self.__Bottom_working_room][start - self.__Bottom_working_room - i] = 1
556
557             # fall
558             for i in range(self.__Bottom_working_room):
559                 self._map[self.__zero_height + self.__height + self.__Bottom_working_room - i][start - wall - 1 - i] = 1
560                 self._map[self.__zero_height + self.__height + self.__Bottom_working_room - i][start - wall - 1 - i - 1] = 1
561
562
563         if side == True:
564             self.__dimensions = [width_slot + 4, self.__zero_height]
565         else:
566             self.__dimensions = [self.__zero_width, height_slot + self.__Bottom_working_room + 5]
567
568         self.place_items(3)
569
570     #extra generation after the inisialastion
571     def place_items(self, amount):
572         '''
573         Loops amount of times as the parameter amount and randomly choses the coordinates of the object to be placed it then either places a 2 or a 3 on the 2d list
574         :param amount:
575         :return:
576         '''
577         amount -=1 # for loop starts at 0
578         for i in range(amount):
579             x_cord = 0
580             y_cord = 0
581             new_x = 0
582             new_y = 0
```

```
583         while new_x == x_cord or new_y == y_cord or new_x == x_cord - 1 or new_x == x_cord + 1 or new_y == y_cord - 1 or new_y == y_cord + 1:
584             new_x = random.randint(self.__zero_width + 3, self.__zero_width + self.__width - 3)
585             new_y = random.randint(self.__zero_height + 1, self.__zero_height + self.__height - 1)
586         x_cord = new_x
587         y_cord = new_y
588         if random.randint(0,1) == 1:
589             self._map[y_cord][x_cord] = 2
590         else:
591             self._map[y_cord][x_cord] = 3
592
593     def create_exit(self):
594         '''
595         Loops until the coords chosen to place the exit is an empty slot and if it is then on the 2d list it is represented as a 4
596         :return:
597         '''
598         loop = True
599         while loop:
600             new_x = random.randint(self.__zero_width + 3, self.__zero_width + self.__width - 3)
601             new_y = random.randint(self.__zero_height + 1, self.__zero_height + self.__height - 1)
602             if self._map[new_y][new_x] == 0:
603                 loop = False
604
605         self._map[new_y][new_x] = 4
606
607     def place_enemy(self, C):
608         '''
609         places a monster in the middle of the room using Connection.create monsters
610         :param C:
611         :return:
612         '''
```

```
613         x_cord = (self.__zero_width + (self.__width//2)) * 20
614         y_cord = (self.__zero_height + (self.__height//2)) * 15
615         C.create_monsters(1, [y_cord, x_cord], self.__room_number)
616
617     #returning items
618     def give_map_back(self):
619         '''
620         Returns the 2d list map
621         :return:
622         '''
623         return self._map
624
625     def give_dimensions_back(self):
626         '''
627         returns the list of the dimesnions of the room
628         :return:
629         '''
630         return self.__dimensions
631
632     def return_room_coords(self):
633         '''
634         Returns a list of the starting width and height and the width and height of the room
635         :return:
636         '''
637         return [self.__zero_width,self.__zero_height,self.__width,self.__height]
638
639     def give_B_working_room(self):
640         '''
641         Returns the amount of space the room has underneath it
642         :return:
643         '''
```

```
644         return self.__Bottom_working_room
645
646     def give_T_working_room(self):
647         '''
648         Returns the amount of spoace the room has above it
649         :return:
650         '''
651         return self.__Top_working_room
652
653     def give_width_height(self):
654         '''
655         returns a list of the width and height
656         :return:
657         '''
658         return [self.__width,self.__height]
659
660     def give_zero_width_height(self):
661         '''
662         Returns a list of the starting width and height
663         :return:
664         '''
665         return [self.__zero_width, self.__zero_height]
666
667     def give_side(self):
668         '''
669         Returns the Boolean value side
670         :return:
671         '''
672         return self.__side
673
674     def player_placement(self):
```

```
675         '''
676         Returns a list of the coords of the middle of the room to place the player there
677         :return:
678         '''
679         placementx = self.__zero_width + (self.__width//2)
680         placementy = self.__zero_height + (self.__height // 2)
681         return [placementy, placementx]
682
683 class Level_map:
684     def __init__(self, difficulty,screen):
685         '''
686         creates all the variables the class uses then it loops for 4*difficulty and it starts by creating one node on each layer but it increases by 1 each time until the last node
687         '''
688         self.__level_map = [] # initialise map
689         self.__dimensions = screen.get_size()
690         #Node creation
691         self.__difficulty = difficulty
692         layers = 4*self.__difficulty
693         self.__sectors_height = self.__dimensions[0]//layers
694         self.__sectors_width = self.__dimensions[1]//layers
695         self.__level_at = 1
696         height_list = []
697         for i in range(layers):
698             height_list.append(i*self.__sectors_height)
699         height_list.reverse()
700         self.__level_map = []
701         for i in range(1,layers): # not plus one so tyhat it leaves enough room for boss node to go
702             self.__level_map.append([])
703             num = i + 1
704             for x in range(1,num):
```

```
705                 self.__level_map[i-1].append(Level_Node((height_list[i-1],(x*self.__sectors_width)), False, i, self, self.__difficulty))
706
707         self.__level_map[0].append(Level_Node((0,400), True,layers , self, self.__difficulty))
708
709     def give_map(self):
710         '''
711         returns the 2d list level map
712         :return:
713         '''
714         return self.__level_map
715
716     def send_level(self):
717         '''
718         it loops through every level node in level map and run the function level_at_recieve where the level at variable is the parameter
719         :return:
720         '''
721         for i in self.__level_map:
722             for x in i:
723                 x.level_at_recieve(self.__level_at)
724
725     def recieve_new_level(self, level):
726         '''
727         assigns parameter to level at variable
728         :param level:
729         :return:
730         '''
731         self.__level_at = level
732
733 class Level_Node(pygame.sprite.Sprite):
734     def __init__(self, pos, end, level, father, total_level):
```

```
735         '''
736         checks if it is the last node if it is the its type is end and its colour is balck with its width and height being 100 by 40 pixles
737         if not then it is randomly assigned as either a dungeon, chest or rest node with the size always being 35 / total level halved squares
738         if its a dungeon then its red, a chest then blue and a rest then yellow
739         it then creates its rect/hitbox and assigns the position coordinates and creates other variables needed for the class
740         :param pos:
741         :param end:
742         :param level:
743         :param father:
744         :param total_level:
745         '''
746         super().__init__()
747         if end:
748             self.image = pygame.Surface([100,40])
749             self.__type ='end'
750             self.__colour = 'black'
751         else:
752             #Node bits
753             self.image = pygame.Surface([35//(total_level/2), 35//(total_level/2)])
754             num = random.randint(1,3)
755             if num == 1:
756                 self.__type = 'dungeon'
757                 self.__colour = 'red'
758             elif num == 2:
759                 self.__type = 'chest'
760                 self.__colour = 'blue'
761
762             elif num == 3:
763                 self.__type = 'rest'
```

```
764             self.__colour = 'yellow'
765
766         # Pygame bits
767         self.__position = pos
768
769         self.rect = self.image.get_rect()
770         pygame.draw.rect(self.image, self.__colour,self.rect )
771         self.__coords = (self.__position[0], self.__position[1])
772         self.rect.x = self.__position[1]
773         self.rect.y = self.__position[0]
774
775         self.__level = level
776         self.__current_level = 1
777         self.__father = father
778
779     def __colision(self):
780         '''
781         runs the __level_check function and if it returns True then it checks whether a colision is
made between the mouse and the node
782         if so its checks if a mouse click has happend if it has the __update_level method is ran and it
returns True
783         :return:
784         '''
785         if self.__level_check():
786             mouse_pos = pygame.mouse.get_pos()
787             click = pygame.mouse.get_pressed()
788             if self.rect.collidepoint(mouse_pos):
789                 print('collision')
790                 if click[0]:
791                     print('click')
792                     self.__update_level()
793                     self.__colour = 'purple'
```

```
794                 return True
795
796     def __update_level(self):
797         '''
798         new_level is assigned as the level + 1 and then runs the function recieve_new_level to the father object with the new_level as the parameter
799         :return:
800         '''
801         new_level = self.__level + 1
802         self.__father.recieve_new_level(new_level)
803
804     def level_at_recieve(self,level):
805         '''
806         assigns the parameter to current_level
807         :param level:
808         :return:
809         '''
810         self.__current_level = level
811
812     def __level_check(self):
813         '''
814         Checks if the level at is this nodes level if so then Return TRue if not Return False
815         :return:
816         '''
817         if self.__level == self.__current_level:
818             return True
819         else:
820             return False
821
822     def run(self):
823         '''
```

```
824         Checks whether a collision has been made from the function __colisision and if it returns
True then it returns the string of its type
825         :return:
826         '''
827         if self.__colision():
828             if self.__type == 'dungeon':
829                 return 'dungeon'
830
831             elif self.__type == 'chest':
832                 return 'chest'
833
834             elif self.__type == 'end':
835                 return 'end'
836
837             elif self.__type == 'rest':
838                 return 'rest'
839
840             else:
841                 return None
842
843
844
```

Evaluation

Objectives

Objective	Was completed	Evaluation
Character Class		
A simplistic character creator	No	I was not able to implement this aspect into the game however I do not feel that it affects the gameplay that much and one of the features I had to drop on the final program due to time constraints.
Allow the player to move on the screen	Yes	The player can move on the screen with no issue and their rect/hitbox follows them when they do
Allow the player to attack	Yes	The player can attack the enemy without any issue. However, an image for the attack was never made so it is still red and yellow blocks so there is a visualisation of the attack but it would have been better to see an image that looked like an attack instead of just the hitbox
Add a dash that the player can do	Yes	The player can dash when the left control is pressed this places the player towards the mouse to make it looks like it has dashed there. It would be better if there was an animation to show the player dashing. There is then a cooldown of about 10 seconds in which the player cannot dash.
Level Generation		
Walls class	Yes	The wall class was completed, and it was able to hold the position. There was image that was created for the wall, and it had no issues. The player cannot go through the wall however that function does not occur in the wall class, and it happens in the character in monster class however it does not affect the gameplay.

The floor will be the background of the level	Yes	The background is a light grey to imitate the floor of the cave the player is in
Environmental Class	Partially	The environment class can use positional arguments to put them on a specific area on the map. There are 2 designs that it can chose from, 2 different types of rocks. However, they do not have any collisions.
Exit class	Yes	The exit class does have a unique image of a hatch. IT is always looking for if a collision has been made and if it has then it looks for an enter input if the exit clause has been fulfilled then the player will go to the level selection level to choose the next level
The exit clause	Partially	There is a clause for if all enemies has been killed however no puzzles have been put into the game so it there is no clause for them.
Map Class	Yes	The map class works, and all objectives were successful. However, it would have been better if there wasn't a set number of rooms as there is always 3. The function that connects the room together works fine. However, there is sometimes a missing block that is not large enough for the player to walk through, but it still does affect how the game looks.
Room Class	Partially	Most of the Room class objectives were completed however there were some that were not completed the left and right indents were not put into the game however I don't think this takes much away as the top and bottom indents are enough to make it look like a cave/organic room and adding the left and right may have been too much. There was not implementation of Dijkstra's algorithm to make

		sure that it can be completed as the way that the map itself is made it will always be able to be completed so I didn't implement that test as it would of just made the process longer.
Monsters AI		
Implementation of A* algorithm	Partially	A node map is made which is a 2d list that imamates the map array/2d list. When pathfinding is being done a target node is set and then an algorithm, check neighbours, is started and they look at all their neighbours. If they look at a node to that one that node is then travelled to. This means that there is not an unvisited list. The visited list is created at the end, and it is the coordinates of the node from the target node to the starting node appended onto a list. Even though the unvisited list was not implemented it was not needed in the way I coded it.
Node Class	Partially	To check if a node is traversable for the enemy pathfinding was not fully implemented instead, I made it so that the enemy only path finds if it is in the same room as the player this means that the enemy would not have to go outside of the room it is in. Other than that, feature and the unvisited list the Node objectives were all fulfilled.
Allowing the enemy to attack	Partially	The enemy can attack the player if it collides with the player. However, there is not an animation to show that this attack has been made and it does not check with a mask of the player. Using the collisions with the rects of the player does not affect gameplay that much but not having an animation makes it hard to tell

		when the enemy is attacking so it would have been good if that had been implemented as it would of made the game easier and more enjoyable to play.
So that they take damage and die	Partially	The enemy can die if it takes enough damage from the player however, they just disappear when this happens. It would have been good to implement the death animation feature as it would have made killing a more rewarding however it does not affect gameplay greatly. The enemy does not drop loot when it dies no inventory system was made for the player so the enemy dropping loot was not added but it would of gave more of a reason to kill the enemies rather than to just get to the next level.
Have the spawn rate of the monsters drop depending on how many have been killed	No	This was not implemented at all instead the rate increases with the levels of the game so when you reach the end of the level selection map you are on o the next on there will be more enemies. Which I think fits better as the further you go through the game the more challenging it is. However, I think that this feature would be great if there were different types of monsters and that the spawn rate of each type changed so that you had different monsters on each level and if you killed one more than the others it would spawn less.

End User Feedback

End User Review

I asked to get a review of what they thought of the game before I asked questions so that the questions could be more tailored to why they didn`t like certain aspects and why liked others more.

The positives of "Monster Hunter" include the procedurally generated dungeon levels stop the game from ending up feeling repetitive, as some games do, though occasionally the walls will generate with small gaps in them, this doesn't impact gameplay to a significant degree. The dynamic shop system helps with balancing, as the price increases every time you purchase a new upgrade, meaning, as you progress, upgrades feel more impactful, as you have needed to work harder to reach it. The branching path feature adds another layer of interactivity, as hands control over to the player in a section that would otherwise result in railroading. However, the game does unfortunately suffer some pitfalls, the enemies themselves are also quite simple, with no projectiles. This is compounded by the fact that the enemies don't reward the player in a tangible way, like money or upgrades, meaning the only way to obtain money, and subsequently upgrades, are the randomised chests that make up the third type of level. These chests on their own are not a bad idea by any means, but they should function as a supplementary method to obtain money, not the only way.

Questions

Are there other ways in which this game stops itself from feeling repetitive?

Other than the procedurally generating dungeons there is the branching paths which are randomly generated. They make the gameplay feel unique and as nothing ever happens in the same order. However, there are only 3 different types which after a while of playing you've played them all so I feel that this game could benefit from more different types of levels to make the gameplay more interesting.

What do you think of the level selection screen?

The level selection screen does its job for selecting levels. However, it can be hard to determine what each button does for the first time as there is nothing that tells you what each colour means. So, I think that it would be better if they had icons or if there was a key to tell you what each colour meant. After you learn how to use the level selection screen it is easy to use. But it would be good to have an indicator to tell you what row you are on. I think that it would be a good idea to show your path on the level selection screen so that you can look back on what levels you chose.

Do you feel that adding projectiles to the players and monsters would help the combat in the game?

As I said previously, I do think that the monsters should have projectiles as it would make them more challenging and more fun to fight against. If they had multiple different projectile patterns like other games, have it would make it more engaging. However, I do believe that the players abilities do not need to be changed as if you gave the player projectile attacks it would keep the difficulty at the same level which would not be improving the game and with the players movement, I don`t think that they would need to have a ranged attack.

What do you think of the players attacks?

They do their job, but I feel that could do more. They don`t tell the player when they have made an attack you only the hitboxes. There designs are not good as they are just 2 different coloured blocks and I feel that if they had an actual design, it would make it more immersive.

What should be taken away from the Feedback?

From the review it can be taken that there should be more ways to get gold from the game so that it makes it more easily able to get gold and that the monsters should be more interesting and maybe have multiple attacks. There are also some issues with the generation that need to be fixed. The enemies should also offer more of a reward other than being needed to progress through the game. There are some positives taken from this with the branching pathways and the generation of the dungeon.

From the responses I feel that there should be more level types and that I could do different types of dungeons. If the addition of the puzzle room was met, I feel that this would of helped however I think that different number of rooms in each dungeon will also help the less repetitiveness of the game.

The level selection screen needs a lot of work to be done to it so that it can be easily understood. I think that putting icons on the symbols will be the better way to make it more understandable. I should also add an arrow to point at which row the player has to select from. I could shade the icons that the player has travelled through to make it so that they look back on their path as well however I don`t think that is such of an important feature of the other faults in the level selection menu.

The players attacks could also do with having a gif of slashing to show that the player is attacking so that the user knows when an attack is made and the hitbox`s would look more interesting and be more immersive for the game which would help it feel more fun and comfortable to play.

Future Improvements

Gold Counter

A future improvement could be that the enemies give the player gold upon killing a monster as that would offer another way for the player to get gold and give the monster another reason to be killed. This can be implemented easily as I would use the same function that the chest class uses to add gold to the player just when a monster dies with an amount of gold 2*difficulty of the level. I would need to implement a pop up or something that tells the player that gold has been added to their character.

Here is a design of what a gold counter could look like:



The font used is castellar as the rest of the game. The reason I chose to make it yellow was to make it more interesting and be the same colour as the gold. The background of the icon is a darker grey so that the yellow text is easier to read and there is a black outline to make it pop and be noticeable.

This would work the same way that it does in the shop where it counts the gold in the shop. Which is where it gets the gold the player has from the return gold function in the player class then it changes the recreates the Line class that creates.

Health Bar

There is also an issue where the player cannot know what their health is at a time. So, adding in a health bar would be good addition to the game. I would make the design a health bar where there is a red bar that goes down to represent the players health. That could look like this:



This would be in the top right of the screen and be able to scale with the players health so that it would remove less if the player had more health. So that it does not get too large if the player has lots of health. I think a simplistic design like this is better as it will not distract from the game itself and if they player has played other games with health bars before they will be able recognise what it is.

The way you could do this is by using the variable health in the player class to determine the length of a red rectangle from pygame. This would be around a rectangle with a black outline but no colour in the middle of it, this rectangle would be at a set length. To calculate how much smaller to make the red rectangle you would divide the length of the red rectangle when it is full by the amount of max health the player has (call this size x). Then when the player loses health, the rectangle variable would get reassigned to that a red rectangle with a length with size x removed for a few times due to

the amount of health lost. You would be able to get the size of the rect through a function to get the private attribute of the sizes.

Vectorising Player Movement

I could also vectorise the movement of the player so that the player does not move quicker or slower on different screens. So that the experience will be the same on different computers/screen sizes. It would also be able to achieve a more smooth and more realistic movement and the player wouldn't just stop instantly instead skid a little before stopping like real life.

In the pygame documentation there is a class pygame.math.vector2 which is used for 2 dimensional vectors. With this you must define the position and velocity however the velocity starts at (0,0) this is the same for acceleration which you also need to define. These are the Kinematic vectors. You must also define some kinematic Constants which are Horizontal acceleration and horizontal friction. Now in the update function when you press a key it will update the acceleration due to the direction that the player is moving in. The amount would be due to the Horizontal acceleration. Then after the keys have been pressed new kinematic vectors will be calculated. So, the acceleration.x = (velocity.x * horizontal friction) – acceleration.x and velocity = velocity + acceleration and position = velocity + (0.5*acceleration) + position. This is due to the physics equation $S = UT + 0.5AT^2$ since T = 1 you can remove T from the equation. T = 1 because this calculation is being done per second/loop of the code. After this we still need to update the rect since the vector is not apart of it. You can do this by using on of the pygame methods for example bottomleft which is to define the bottom left of the rect with a vector. So, we will be able to put the variable position as a parameter into pygame.bottomleft to change the position of the rect.